

University of Stuttgart
Institute of Industrial Automation and
Software Engineering

Model Based Systems Engineering and Adversarial Multi-Agent Reinforcement Learning

Master Thesis 3564

Submitted at the University of Stuttgart by
Ahmed Mahfouz

INFOTECH

Examiner: Jun. -Prof. Dr.-Ing. Andrey Morozov

Supervisor: Joachim Grimstad, M.Sc.Eng

02.12.2023



Abstract

This thesis combines Model-Based Systems Engineering, Discrete-Time Markov Chains, Adversarial Multi-Agent Reinforcement Learning, and game theory to analyze system safety and resource optimization. It introduces fundamental concepts and a competitive game model where two agents compete to control system states. A Discrete-Time Markov Chain is transformed into a system object. The system object is transformed into a game object integrated into an environment object. The Proximal Policy Optimization model is trained using neural networks. Ten experiments validate the concept and prototype. A path elimination algorithm is presented, with four experiments introduced. This unique framework offers a promising avenue for system safety analysis and resource optimization.

Keywords: Model-Based Systems Engineering, Discrete-Time Markov Chains, game theory, Adversarial Multi-Agent Reinforcement Learning, Path elimination.

die Zusammenfassung

Diese Arbeit kombiniert modellbasiertes Systems Engineering, zeitdiskrete Markov-Ketten, adversariales Multi-Agenten-Verstärkungslernen und Spieltheorie zur Analyse von Systemsicherheit und Ressourcenoptimierung. Es werden grundlegende Konzepte und ein kompetitives Spielmodell vorgestellt, bei dem zwei Agenten um die Kontrolle von Systemzuständen konkurrieren. Eine zeitdiskrete Markov-Kette wird in ein Systemobjekt umgewandelt. Das Systemobjekt wird in ein Spielobjekt umgewandelt, das in ein Umgebungsobjekt integriert ist. Das Proximal Policy Optimization Modell wird mit Hilfe neuronaler Netze trainiert. Zehn Experimente validieren das Konzept und den Prototyp. Ein Algorithmus zur Pfadeliminierung wird vorgestellt und vier Experimente werden erläutert. Dieser einzigartige Rahmen bietet einen vielversprechenden Weg für die Analyse der Systemsicherheit und die Optimierung von Ressourcen.

Schlüsselwörter: Modellbasierte Systemtechnik, zeitdiskrete Markov-Ketten, Spieltheorie, Adversarial Multi-Agent Reinforcement Learning, Pfadeliminierung.

Preface

At the end of this academic journey, I am filled with both excitement and gratitude. This thesis represents the culmination of extensive research, critical analysis, and countless hours of dedication. It is the product of not only my efforts but also the support and encouragement of my supervisors, and colleagues.

I wish to express my heartfelt gratitude to my thesis advisor, Joachim Grimstad, M.Sc., whose unwavering guidance, invaluable insights, and endless patience have been instrumental in shaping the direction of this study. Your mentorship has been a source of inspiration, and I am truly fortunate to have had the opportunity to learn from you.

I would like to extend my appreciation to the University of Stuttgart and the IAS Institute for providing the resources and conducive environment that made this research possible. Additionally, I want to acknowledge the financial support provided by DAAD, and the Sony Awards Program, which allowed me to focus on my research without undue financial burdens.

The research journey has not only been about academic growth but also personal development. I wish to acknowledge the support and great love of my family, my father, Mahmoud, my mother, Amaal, my brother, Muhammad, my sister Eman, my wife Dina, and my future son Youssef who will be born a few weeks after the thesis submission. Their unwavering support, understanding, and encouragement. Your belief in me has been a constant source of motivation.

The research presented in this thesis represents my earnest effort to contribute to the body of knowledge in my field. I hope that this work will serve as a valuable resource for future researchers and a testament to the power of perseverance and dedication in academic pursuits.

Table of Contents

Abstract	i
die Zusammenfassung	ii
Preface	iii
List of Figures	viii
List of Tables	x
1 Introduction	1
1.1 Project Definition	2
2 Basis	3
2.1 Environment and Constraint Conditions	3
2.2 Game Theory Approach	3
2.2.1 Description of Game Theory	3
2.2.2 Advantages of Game Theory	3
2.2.3 Disadvantage of Game Theory	4
2.3 Basic Decisions	4
2.4 Technological Basics	5
2.4.1 Model-Based Systems Engineering	5
2.4.2 Discrete-Time Markov Chain	9
2.4.3 Game Theory	13
2.4.4 Reinforcement Learning	15
2.4.5 Algorithms of Reinforcement Learning	18
2.4.6 Multi-Agent Reinforcement Learning	20
2.4.7 PettingZoo API for Multi-Agent Reinforcement Learning	26
3 Conception	31
3.1 Description of the Conception	31
3.1.1 DTMC Model Parsing	31
3.1.2 System Object	32
3.1.3 Game Object	32
3.1.4 Environment Object	32
3.1.5 PPO Model	32
3.1.6 System Software Architecture	33

3.2	Game Description	34
3.2.1	Player Strategy	34
3.2.2	Game Termination Conditions	34
3.3	Environment Description	35
3.3.1	Observation Space	35
3.3.2	Action Space	35
3.3.3	Action Masking	35
3.3.4	Step Function	35
3.3.5	Reward Function	36
3.4	Summary	36
4	Prototype Description	37
4.1	Description of the System Components	38
4.1.1	Parser Object	38
4.1.2	System Object	38
4.1.3	DTMC Object	38
4.1.4	Game Object	38
4.1.5	Environment Object	38
4.1.6	PPO Model	38
4.1.7	DTMC Model Generator	38
5	Experiments and Results	39
5.1	Experiment 1: Results Difference	39
5.2	Experiment 2: Episodes Increase	41
5.3	Experiment 3: State Desirability	42
5.4	Experiment 4: DTMC Solution	44
5.5	Experiment 5: Number of Transitions	45
5.6	Experiment 6: Initial Resources	46
5.7	Experiment 7: Udesirableness threshold	48
5.8	Experiment 8: Changing Order of Moves	49
5.9	Experiment 9: Transitions Add/Remove	50
5.10	Experiment 10: Changing Order of Moves without Retrain	52
6	Thesis Specialization	53
6.1	Path Elimination Algorithm	53
6.2	Optimization Problem	54

6.3	Experiments and Results	54
6.3.1	Experiment 1	54
6.3.2	Experiment 2	55
6.3.3	Experiment 3	56
6.3.4	Experiment 4	56
6.4	Experiments Conclusion	57
6.5	Other Optimization Techniques	57
6.5.1	Reinforcement Learning with Constraints	57
6.5.2	Soft Actor-Critic with Soft Constraints	58
6.5.3	Proximal Policy Optimization with Penalty Terms	58
6.5.4	Constrained Optimization with Lagrange Multipliers	58
6.5.5	Trust Region Policy Optimization with KL Divergence Constraints	59
6.5.6	Action Space Constraints	59
6.5.7	Resource-Adaptive State Space	60
6.5.8	Resource Allocation Networks	60
6.5.9	Dynamic Resource Allocation Strategies	60
6.5.10	Hybrid Control and Reinforcement Learning	60
6.5.11	Reward Shaping with Resource Costs	60
6.5.12	Multi-Agent Reinforcement Learning with Communication	60
6.5.13	Model Predictive Control	61
6.5.14	Hierarchical Reinforcement Learning	61
6.5.15	Supervised Learning for Resource Constraint Enforcement	61
7	Conclusion and Future Work	62
	Bibliography	64
	Appendix	66
A	Prototype Implementation	66
A.1	Parser Object	66
A.2	System Object	67
A.3	DTMC Object	69
A.4	Game Object	70
A.5	Environment Object	72
A.6	PPO Model	73
A.7	DTMC Model Generator	76

B	DTMCs of Different Experiments	77
C	Experiment Results	85
C.1	Experiment 1	85
C.2	Experiment 2	89
C.3	Experiment 3	91
C.4	Experiment 4	94
C.5	Experiment 5	95
C.6	Experiment 6	98
C.7	Experiment 7	103
C.8	Experiment 8	107
C.9	Experiment 9	110
C.10	Experiment 10	113
C.11	Path Elimination: Experiment 1	114
C.12	Path Elimination: Experiment 2	115
C.13	Path Elimination: Experiment 3	117
C.14	Path Elimination: Experiment 4	119

List of Figures

1	Example of a Markov Chain	9
2	Example of Absorbing DTMC	10
3	Mean First Passage Time for an ergodic Markov	13
4	Mean Recurrence Time for Ergodic Chains	13
5	Example of Extensive Form Game	14
6	Agent-environment interaction loop	15
7	Adversarial interference in landmark covering problem	25
8	MARL under adversarial state perturbations	25
9	An example of the basic usage of Gym	26
10	The AEC diagram of chess	28
11	The AEC diagram of chess	29
12	System Software Architecture	31
13	PPO algorithm pseudocode	33
14	Action Masking Example	35
15	Prototype Implementation	37
16	General Structure of the DTMCs used for the experiments	39
17	Results of Experiment 1	40
18	Results of Experiment 2	42
19	Results of Experiment 3	43
20	Results of Experiment 4	44
21	Results of Experiment 5	46
22	Results of Experiment 6	47
23	Results of Experiment 7	49
24	Results of Experiment 8	50
25	Results of Experiment 9	51
26	Results of Experiment 10	52
27	Path Elimination Algorithm	53
28	Path Elimination Algorithm: Results of Experiment 1	55
29	Path Elimination Algorithm: Results of Experiment 2	55
30	Path Elimination Algorithm: Results of Experiment 3	56
31	Path Elimination Algorithm: Results of Experiment 4	57
32	System Software Architecture	62
33	Transition matrices for experiments 1 and 2	77

34	Transition matrices for experiments 3 and 4	78
35	Transition matrices for experiments 5.1 and 5.2	79
36	Transition matrices for experiments 5.3 and 6	80
37	Transition matrices for experiments 7 and 8.	81
38	Transition matrices for experiments 9 and 10.	82
39	Transition matrices for resource optimization experiments 1 and 2.	83
40	Transition matrices for resource optimization experiments 3 and 4.	84

List of Tables

1	Example of Normal Form Games	14
2	PPO Characteristics	33

1 Introduction

Model-Based Systems Engineering (MBSE) is defined as a procedure supporting design, and analysis in the development of complex systems. MBSE is used to reduce safety hazards before developing the system so that systems are safe in the early design, instead of adding safety features during the development process (Schvchenko, 2020).

In a model, the system should be represented in less detail by the system architect in which the system's structure and behavior are clear, while the system's complexity is managed. The safety and reliability of the designed system are critical aspects of the system.

Adversarial Multi-Agent Reinforcement Learning (Adv-MARL) is one of the approaches used recently to investigate the decision-making problem in a multi-agent environment. Adv-MARL considers the interaction of a team of agents training in an environment and learning optimal policies via the trial-and-error process.

Using the concepts of Adv-MARL, the purpose of this project is to transform a model-based system, a Discrete-Time Markov Chain (DTMC) in this thesis work, into a zero-sum game containing two agents, and train the opponent agents to win the game, so we can redesign the system based on the game results.

The following list will give a short description of each chapter included in this thesis work.

- **Chapter 1** will introduce the project's different phases and their related tasks.
- **Chapter 2** will introduce basic definitions of game theory, MBSE, DTMC, and Adv-MARL.
- **Chapter 3** will introduce the developed concept of converting a DTMC into a game, and how different RL agents are trained to win this game within an environment, using the Proximal Policy Optimization (PPO) algorithm.
- **Chapter 4** will introduce a prototype implementation of the software components (System model, System object, Game object, Environment Object, and PPO model).
- **Chapter 5** will be an experimental chapter introducing different experiments that investigate the results of the game and the behavior of the agents under different game conditions.
- **Chapter 6** will introduce an algorithm for optimizing the initial resources provided to each agent to restrict the actions selected by each agent so that the most improbable DTMC paths are excluded. It will also introduce other different techniques of resource optimization that can be considered in future implementations.
- **Chapter 7** will introduce a thesis summary and possible future work.

1.1 Project Definition

This project mainly consists of four phases:

1. Definition Phase

The definition phase includes two main tasks:

- Defining the system requirements discussed with the supervisor and summarizing them in the System Requirements Specification document.
- Understanding the basic concepts of game theory, MBSE, DTMC, and Adv-MARL.

2. Conception Phase

The conception phase includes two main tasks:

- Designing a concept for converting a DTMC into a Zero-Sum Game.
- Designing a system software architecture for the implemented prototype.

3. Prototype Phase

The prototype implementation phase includes one main task:

- Implementing the software components for the developed concept.

4. Experiment and Specialization Phase

The experiment and specialization phase includes two main tasks:

- Conducting different experiments to test the developed prototype under different conditions.
- Develop a path elimination algorithm and introduce other algorithms and techniques from the literature.

2 Basis

2.1 Environment and Constraint Conditions

This thesis project investigates the idea of MBSE, as a procedure supporting design and analysis in developing complex systems. The thesis picks one of the engineering models, which is the DTMC, and aims to convert it into a zero-sum game. The game contains two agents, and the opponent agents are trained to win the game using the concepts of Adv-MARL. Then, the game theory concepts will be used to evaluate the behavior of the trained agents through their action steps, and decision taken to win the game. Through behavior evaluation and analysis, the system can be redesigned, so it is safer. The following list includes the constraints and conditions for this thesis project:

- Python must be used as a programming language for implementation.
- The learning environment must be implemented based on the PettingZoo library.
- The implementation must properly follow Object-Oriented Programming (OOP).
- The project must implement the modules: system object, game object, learning environment, and learning algorithms.

2.2 Game Theory Approach

2.2.1 Description of Game Theory

The ideas in game theory offer a common language for describing, organizing, analyzing, and ultimately understanding various strategic scenarios. Game theory typically examines how agents interact and make decisions in conflict situations. In game theory, a game involves a finite number of agents following specific rules. These agents could be people, teams, organizations, businesses, associations, etc. They are interdependent because their interactions will influence both the individual agents and the entire group of agents. To be more specific, a game is defined by a group of agents and their potential to play the game by the rules, or more specifically, by their collection of strategies. The following sections will introduce the advantages and disadvantages of the game theory (Hotz, 2006).

2.2.2 Advantages of Game Theory

The advantages of game theory are introduced in the following subsections.

2.2.2.1 Enhances Reasoning and Decision-Making Skills

Understanding the game theory strategies can enhance one's reasoning and decision-making skills, particularly in complex situations. Game theory provides a systematic framework for analyzing strategic interactions, allowing individuals to make more informed choices (Picardo, 2022).

2.2.2.2 Interdisciplinary Approach

Game theory brings together various disciplines such as mathematics, psychology, philosophy, economics, and political science. This interdisciplinary nature enables the application of game theory in a wide range of fields, facilitating cross-pollination of ideas and insights (Picardo, 2022).

2.2.2.3 Recognized Contribution

The fact that several leading economists and scientists have been awarded the Nobel Prize in Economic Sciences for their contributions to game theory demonstrates its importance and impact on modern analysis and decision-making. This recognition highlights the significance of game theory in advancing knowledge and understanding (Picardo, 2022).

2.2.3 Disadvantage of Game Theory

The disadvantages of game theory can be listed in the following subsections.

2.2.3.1 Assumption of Rationality

According to game theory, players make logical decisions and always act in their own best interests. A number of variables, such as emotions, prejudices, and incomplete knowledge, can affect human behavior and cause it to diverge from reason. (Askari, Gordji and Park, 2019).

2.2.3.2 Complexity and Simplifications

Many real-world scenarios are highly complex and involve numerous inter-dependencies, making it challenging to model them accurately in game theory. Simplifications and assumptions are often made to make the analysis feasible, which may overlook important aspects of the situation.

2.2.3.3 Information Requirements

Game theory assumes that players have perfect or complete information about the game, its rules, and the strategies chosen by other players. However, in practice, information may be limited, uncertain, or asymmetric, impacting the accuracy of predictions and outcomes (Walliser et al., 2008).

2.2.3.4 Lack of Behavioral Dynamics

Game theory typically assumes static and fixed strategies, disregarding the potential for players to adapt or learn from previous interactions. Players may adjust their strategies based on feedback and changing circumstances, leading to dynamic and evolving behaviors.

2.3 Basic Decisions

The main reason for choosing game theory as the general approach of this thesis project is that game theory has a lot of concepts such as zero-sum games that could be used in Reinforcement Learning (**RL**) and Artificial Intelligence (**AI**). In general, the game theory could be used to do the following:

- **Evaluating** what the agent is doing.
- **Evaluating** how the agent is winning the game through the established mathematics of game theory.
- **Determining** what is happening in the Neural Network architecture.
- **Determining** which parts are responsible for which actions.
- **Quantifying** how robust and accurate the system is. This point is not as strong in AI as it is in game theory.

2.4 Technological Basics

2.4.1 Model-Based Systems Engineering

MBSE is defined as a procedure supporting design and analysis in developing complex systems. MBSE places models at the center of system design, in contrast to document-based systems engineering (DBSE). The adoption of MBSE increased due to the increased adoption of digital-modeling environments in the recent past (Schvchenko, 2020).

MBSE is used to reduce safety hazards before developing the system so that systems are as safe in the early design, instead of adding safety features during the development process. Systems engineers can perform threat-modeling analysis of the system earlier when system attributes are captured in models. Based on that, they can integrate hazard reduction strategies into the system design, so the system's overall safety-based hazards are reduced (Schvchenko, 2020).

2.4.1.1 Concepts of Model-Based Systems Engineering

MBSE integrates three concepts introduced in the following subsections.

1. Model

It is a simple version of a system. It can be a graph, a mathematical equation, or a physical representation abstracting the real complex system (Hart, 2015). In a model, the system should be represented in less detail by the system architect in which the system's structure and behavior are clear, while the system's complexity is managed. A perfect model fully represents the system which should confirm its model (Long and Scott, 2012).

2. Systems Thinking

It is a method that considers the investigated system as a part of a larger system. In contrast to the systematic following of perfect methods, or gathering statistics, the system engineer observes the top view of the system and inspects its context, lifecycle, boundaries, behavior, and clear patterns. This helps in clarifying the system problems, such as effort duplication, absence of interaction, process step, or chance of automation. It also handles the system's complexity (Schvchenko, 2020). Even though the system must be divided into parts by the system engineer to be analyzed and the connection between parts is clarified, the system's parts are later synthesized back as one consistent part. Parts connections not only guarantee the system's proper work but also the parts' dependency on each other (Schvchenko, 2020). By systems thinking, the system's interconnectedness is highlighted. The system's sub-parts activities drive the system's behavior, and feedback loops and patterns causes are identified by the system engineer. In general, systems thinking balance and manage the system's complexity (Acaroglu, 2017).

3. Systems Engineering

It is a combining approach that uses system principles, concepts, and scientific, and technological techniques to enable the perfect realization, usage, and retirement of engineered systems. It combines several techniques to guarantee that all system requirements are satisfied by design. It studies the system's architecture, analysis, implementation, integration, and management during its lifecycle. It also studies the system's software, hardware, processes, and disciplined aspects (INCOSE, 2023).

2.4.1.2 Benefits of Model-Based Systems Engineering

MBSE has more advantages than DBSE in a digital modeling environment. For instance, in a document-based technique, the system's design is captured from many documents generated by different authors to satisfy the different views of stakeholders regarding the system's hardware, software, behavior, security, safety, and other aspects. The digital modeling environment provides a single source of truth for the system where the same model elements create the specific views of the system (Schvchenko, 2020).

The digital modeling environment also provides a mutual standards-based technique for documenting the system, this technique can be automatically validated to reduce irregularities within the models and guarantees the use of all stakeholders' standards. The system's analysis and the reduction of defects caused by the document-based technique are improved by the common modeling environment. It also provides a consistent propagation of corrections and integration of new information and design decisions to all stakeholders. An overall reduction of development hazards is guaranteed when MBSE is done correctly (Schvchenko, 2020).

The following subsections will introduce the benefits of the MBSE.

1. Integration Time and Effort

Due to the interconnectedness of a model-based system, all dependents are automatically updated whenever a model is modified, which decreases the update time of MBSE. However, in DBSE, a change applied to one document means all dependents should also be updated (Desmar, 2022).

2. Communication

Models give a better understanding and a more detailed level of communication between team members. All stakeholders can easily understand the system through diagrams and pictures (Desmar, 2022).

3. Fewer Documents

A challenge of information and product quality was highlighted after the use of documents, which should not contain errors to avoid rework, delays, and additional costs. Error propagation between documents is another problem, which needs proper reviewing of documents. This can add additional delay in the absence of a proper document management system (Desmar, 2022).

4. Fixed Quality Standards

A standard and quality level are set, and all teams should follow them without any lower or higher quality standards. MBSE guarantees following these quality standards, even though different teams are contributing to one project. Fixed standards reduce flexibility, which can have pros and cons based on how they are used (Desmar, 2022).

5. Developmental Risks

MBSE provides a single source of truth, which decreases the arising possibility of risk problems during the system lifecycles. This also provides the stakeholders the ability to use a fixed standard removing inconsistencies during the system documentation process (Desmar, 2022).

2.4.1.3 Disadvantages of Model-Based Systems Engineering

MBSE has some challenges and disadvantages that are introduced in the following.

1. Usability Issues

MBSE needs more effort to be used. If it is better than project management, or systems engineering approaches, it should be easier to navigate and utilize (Desmar, 2022).

2. Rigid Framework

MBSE has a basic framework on which all other things are built. The team can make modifications; however, it is hard to change this basic framework. A problem in the foundation can affect the entire system which can lead to a restart or cancellation of the entire project if corrections are needed (Desmar, 2022).

3. Flashy Not Always Functional

With MBSE, the system is represented with interactive components, which makes the use and understanding easier for the people inside the organization, however, it can be confusing for people from outside. The results might not match what is happening in the system, which can lead to wrong decisions by stakeholders since they are not understanding the process (Desmar, 2022).

4. Model Complexity

MBSE needs a proper understanding of the internal model works to give the best use of it. MBSE can only be used by experts, which can cause misunderstanding, put more responsibility on them, and overload their work (Desmar, 2022).

5. New Concepts

Many people still do not know much about MBSE because it is still quite new, which can cause problematic and fatal failures. Trial-and-error models should be applied to the new models which can be less effective than the old ones (Desmar, 2022).

6. Stakeholders' Buy-in

All stakeholders should agree on MBSE to be successful. Not all Stakeholders will have the ability to learn or adapt to a new way of working. Additionally, learning and using MBSE is still difficult for some stakeholders (Desmar, 2022).

7. The System is The Limit

In contrast to DBSE, documents can not be imported into a model-based system. Using external technology offering new ways to design, calculate, and visualize system behavior, is not allowed because the system is model-based by nature (Desmar, 2022).

8. The Complexity of System and Software Updates

Models are hosted in software, so if this software is updated, the models will also require updates. If multiple models need updates, this can take a long time. In contrast to document systems, updating software does not require updating the documents (Desmar, 2022).

9. Static Overtime

The model becomes static over time because it is built based on a static frame. This reduces the model's flexibility and ability to adapt to environmental changes (Desmar, 2022).

10. Lack of Standardization

MBSE still has no standard. This means that different organizations can use different approaches. Standards are necessary to guarantee quality, the ability to build on previous works, and knowledge transfer (Desmar, 2022).

11. Scaling Issues

The complexity of the MBSE model increases when the size of the project increases. Sometimes, the model can not manage the project's complexity. In this case, the model should be redesigned which can affect the entire project (Desmar, 2022).

12. The Effort to Use

Using the approach requires more work, and simple characteristics might be too complex. This may cause frustration and a sense that the strategy is failing. Additionally, the added effort might not be useful, especially when simpler methods are sufficient (Desmar, 2022).

13. Snowball Effect

A problem with the foundation will affect the entire system (Desmar, 2022).

14. Review Issues

All changes need to be reviewed because the model is the base of the project, which can affect the project's progress. Reviews are also necessary to guarantee, that the changes will not affect the project negatively (Desmar, 2022).

15. Incorrect Assumptions

The correctness of the assumptions on which the model is based might affect the accuracy of the project's results. Validation of assumptions is very important (Desmar, 2022).

16. Different and Unknown Risks

MBSE has more risks than DBSE. These risks should be considered by the project managers and risk managers. Risks related to the process, stakeholders, and the models themselves, might appear because MBSE is still new which makes it hard to plan risk mitigation (Desmar, 2022).

17. Costly to Implement

MBSE needs a team of experts, and a dedicated system to be implemented and maintained which can be costly. Sometimes, this high cost is not justified when traditional methods can do the job. MBSE obligates organizations to work and develop one system, in contrast to DBSE, in which the system can use different resources and insert them quickly (Desmar, 2022).

In the next section, DTMC will introduce the selected model-based system to be focused on in this thesis work.

2.4.2 Discrete-Time Markov Chain

This section will introduce the main concepts of Markov Chains, specifically the DTMC.

2.4.2.1 Markov Chains

As shown in figure 1, a Markov Chain process can be described as a set of states, $S = \{s_0, s_1, \dots, s_r\}$. One of these states is considered a start state from which the process moves sequentially between states via steps. The process moves from the current state s_i to the next state s_j with a transition probability p_{ij} , which is independent of what the previous chain states through which the process took before the current state s_i . A probability p_{ii} denotes the probability of the process staying in the same state. An initial probability distribution and initial state are defined on S (Grinstead and Snell, 2006).

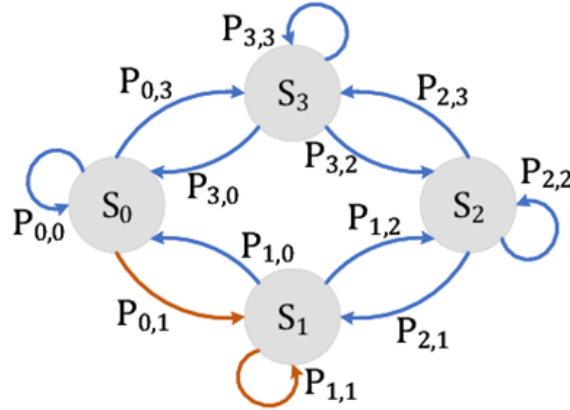


Figure 1: Example of a Markov Chain (Salamat et al., 2019)

2.4.2.2 Transition Matrix of Markov Chains

$$P = \begin{matrix} & \begin{matrix} R & N & S \end{matrix} \\ \begin{matrix} R \\ N \\ S \end{matrix} & \begin{pmatrix} 0.50 & 0.25 & 0.25 \\ 0.50 & 0.00 & 0.50 \\ 0.25 & 0.25 & 0.25 \end{pmatrix} \end{matrix} \quad (1)$$

This is an example of the transition matrix for a Markov Chain representing the weather change probabilities between rain R , nice day N , and snow S in the Land of Oz in a day. For example, the first row of the matrix P includes the probabilities for a day following a rainy day. The $p_{ij}^{(2)}$ denotes the probability of currently being in state i and being in state j after two steps (Grinstead and Snell, 2006). From conditional probability rules, $p_{ij}^{(2)}$ of a Markov Chain with r states can be written as:

$$p_{ij}^{(2)} = \sum_{k=1}^r p_{ik} p_{kj} \quad (2)$$

From the above observation, the following general theorem can be proved.

Theorem 1: In a Markov Chain, if P is the transition matrix of a Markov Chain, element $p_{ij}^{(n)}$ in the matrix P^n represents the probability of starting in state i and being in state j after n steps (Grinstead and Snell, 2006).

2.4.2.3 Probability Distribution of Markov Chains

$$P^6 = \begin{matrix} & \begin{matrix} R & N & S \end{matrix} \\ \begin{matrix} R & N & S \end{matrix} & \begin{pmatrix} 0.40 & 0.20 & 0.40 \\ 0.40 & 0.20 & 0.40 \\ 0.40 & 0.20 & 0.40 \end{pmatrix} \end{matrix} \quad (3)$$

By computing P^6 of matrix P , it is obvious that the prediction values do not depend on today's weather, which means it is a **regular** Markov Chain. Taking the long-term behavior of the Markov Chain into consideration, it can start with a probability distribution of the set of states called **probability vector $\mathbf{u}$** . It is a row vector with r nonnegative entries where $\sum_{i=1}^r r_i = 1$. The i th element of $\mathbf{u}$ represents the probability that the chain starts with state s_i (Grinstead and Snell, 2006). Based on that definition, the following theorem can be proved:

Theorem 2: In a Markov Chain, if P is the transition matrix of a Markov Chain, and $\mathbf{u}$ is the probability vector representing the distribution of the chain start, then the probability of reaching state s_i after n steps is the i th element of $\mathbf{u}$, then $\mathbf{u}^{(n)} = \mathbf{u}P^n$ (Grinstead and Snell, 2006).

From this theorem, the behavior of a Markov Chain with the assumption that it starts with a certain state s_i can be examined by setting the i th element of $\mathbf{u}$ to 1 and all other elements to 0.

2.4.2.4 Special Types of Markov Chains

Considering special types of Markov Chains is the best way to study Markov Chains. This section will introduce some special types of Markov Chains.

1. Absorbing Markov Chains

If there is a state s_i of a Markov Chain that is impossible to leave ($p_{ii} = 1$), it is called an absorbing state. If there is at least one **absorbing** state that can be reached from every other state in the Markov Chain, the whole chain is called absorbing. Non-absorbing states are called **transient** (Grinstead and Snell, 2006).

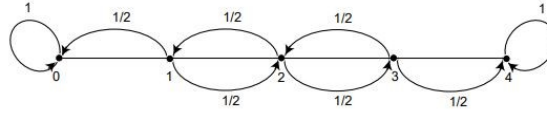


Figure 2: Example of Absorbing DTMC (Salamat et al., 2019)

Figure 2 shows an example of an absorbing DTMC with two absorbing states $\{0, 4\}$, and three transient states $\{1, 2, 3\}$.

$$P = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \end{matrix} \quad (4)$$

If the process is in a transient state, it has an even probability of moving to the left or right. The process continues till it reaches one of the absorbing states, either 0 or 4, then it stays there. The transition matrix is as in equation 4.

The following chain properties will depend on the starting state and the transient state.

- The probability that the process will end up in each state.
- The average time it will take for the process to be absorbed.
- The average number of times the process will be in each transient state.

Canonical Form and Probability of Absorption

For an arbitrary absorbing Markov Chain, if it has r absorbing states and t transient states, the transition matrix P can be put in the following canonical form:

$$TR. \text{ ABS.}$$

$$P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix} \quad (5)$$

where $I_{r \times r}$ is the identity matrix, $0_{r \times t}$ is the zero matrix, $R_{t \times r}$ is a nonzero matrix, and $Q_{t \times t}$ matrix. The first t states are transient, however, the last r states are absorbing.

Based on **Theorem 1** introduced in section 2.4.2, P^n of absorbing Markov Chain can be written as:

$$TR. \text{ ABS.}$$

$$P^n = \begin{pmatrix} Q^n & * \\ 0 & I \end{pmatrix} \quad (6)$$

where $*_{t \times r}$ is the matrix in the upper right-hand corner of P^n . Q^n contains the probabilities of being in a transient state after n steps for each possible starting state. As the probability of being in a transient state after n steps approaches zero, when n reaches infinity, the values of every entry of Q^n reach zero.

Theorem 3: The probability of an absorbing Markov Chain to be absorbed is 1 ($Q^n \rightarrow 0$ as $n \rightarrow \infty$) (Grinstead and Snell, 2006).

The Fundamental Matrix

Theorem 4: The Matrix $I - Q$ for an absorbing Markov Chain P has an inverse matrix N called the fundamental matrix. $N = I + Q + Q^2 + \dots$ When a chain starts in state s_i , the element n_{ij} of the matrix N represents the expected number of times the chain will be in state s_j . The initial state is counted if $i = j$ (Grinstead and Snell, 2006).

By calculating the N matrix of the absorbing Markov Chain example in section 2.4.2:

$$N = (\mathbf{I} - \mathbf{Q})^{-1} = \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{pmatrix} 1.5 & 1 & 0.5 \\ 1 & 2 & 1 \\ 0.5 & 1 & 1.5 \end{pmatrix} \end{matrix} \quad (7)$$

For example, if the chain is started in state s_2 , the expected number to get in states 1, 2, and 3 is 1, 2, 1 as indicated by the middle row of N (Grinstead and Snell, 2006).

Time to Absorption

Theorem 5: If the absorbing Markov Chain starts in state s_i , t_i is the expected number of steps before absorption, and $\mathbf{t}$ is a column vector whose i th element is t_i , then $\mathbf{t} = \mathbf{N}\mathbf{c}$. Where $\mathbf{c}$ is a column vector with all elements equal to 1 (Grinstead and Snell, 2006).

Absorption Probabilities

Theorem 6: If the absorbing Markov Chain starts in a transient state s_i and will be absorbed in state s_j with a probability of b_{ij} , then the matrix $B_{t \times r}$ including the elements b_{ij} , then $\mathbf{B} = \mathbf{NR}$. Where $\mathbf{N}$ is the fundamental matrix, and $\mathbf{R}$ is introduced in canonical form 2.4.2.

By calculating the $\mathbf{B}$ matrix of the absorbing Markov Chain example in section 2.4.2:

$$\mathbf{B} = \begin{matrix} & \begin{matrix} 0 & 4 \end{matrix} \\ \begin{pmatrix} 0.75 & 0.25 \\ 0.50 & 0.50 \\ 0.25 & 0.25 \end{pmatrix} \end{matrix} \quad (8)$$

For example, if the chain is started in state s_1 , there is an absorption possibility of 0.75 in state s_0 , and 0.25 in state s_4 (Grinstead and Snell, 2006).

2. Ergodic Markov Chains

Ergodic Markov Chain is another type of Markov Chain, which is defined as the Markov chain in which it is possible to move from every state to every state in one step or more. It is also called an irreducible Markov Chain (Grinstead and Snell, 2006).

Regular Markov Chains

Regular Markov Chains are a type of Markov Chains, which are defined as the Markov chains, which have some power of the transition matrix and have only positive elements. All regular Markov Chains are ergodic but not all ergodic Markov Chains are regular (Grinstead and Snell, 2006).

Theorem 7: If $\mathbf{P}$ is the transition matrix of a regular Markov Chain, then

$$\mathbf{W} = \lim_{n \rightarrow \infty} \mathbf{P}^n \quad (9)$$

Where $\mathbf{W}$ is a matrix with all rows equal to a row vector $\mathbf{w}$. For example, the matrix $\mathbf{P}$ in section 2.4.2 is a regular matrix as $\mathbf{P}^6$ is in the form of a $\mathbf{W}$ matrix. The row vector $\mathbf{w}$ is fixed for the $\mathbf{P}$ matrix and can be obtained from the equation: $\mathbf{w}\mathbf{P} = \mathbf{w}$ (Grinstead and Snell, 2006).

Mean First Passage Time for Ergodic Chains

Mean First Passage Time for an ergodic Markov Chain from s_i to s_j , m_{ij} , is the expected number of steps to reach state s_j for the first time if the chain starts in state s_i . By convention, $m_{ii} = 0$. As shown in figure 3, it can be computed by making s_j an absorbing state and calculating the time to absorption as introduced in section 2.4.2 (Grinstead and Snell, 2006).

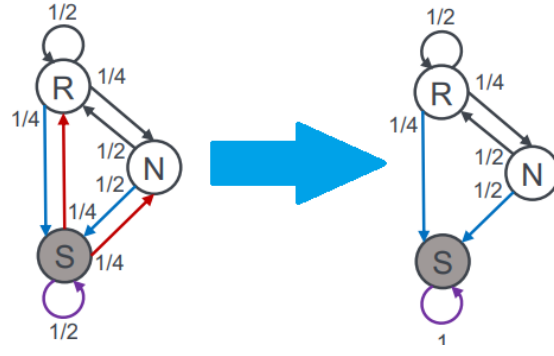


Figure 3: Mean First Passage Time for an Ergodic Markov Chain(Morozov, 2022)

Mean Recurrence Time for Ergodic Chains

The Mean Recurrence Time for an ergodic Markov Chain r_i is the expected number of steps to return to state s_i for the first time if the chain starts in state s_i . As shown in figure 4, it can be computed by adding an absorbing state to the Markov Chain as a copy of state s_i and calculating the time to absorption as introduced in the section 2.4.2.

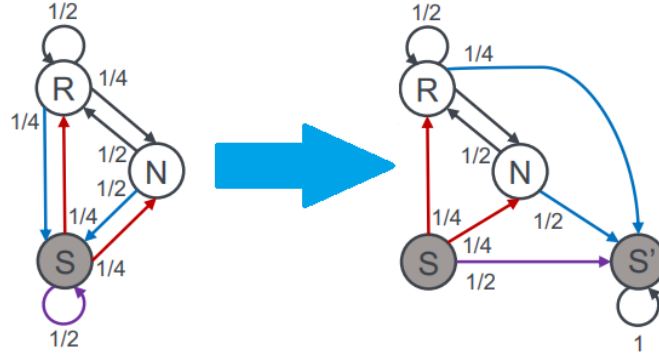


Figure 4: Mean Recurrence Time for Ergodic Chains Markov (Morozov, 2022)

2.4.3 Game Theory

Game Theory This section will introduce the main concepts of game theory such as normal-form games, extensive-form games, and Nash equilibrium (Han et al., 2022).

2.4.3.1 What is Game Theory?

A common language for characterizing, categorizing, evaluating, and ultimately comprehending different strategic scenarios is provided by the concepts of game theory. Typically, game theory studies how agents interact and choose their course of action when faced with conflict. In the context of game theory, a game consists of a predetermined number of players who interact according to predetermined rules. These participants may consist of individuals, groups, companies, associations, associations, and so on. Because their interactions will affect each player individually as well as the group as a whole, they are interdependent. A group of players and their ability to follow the rules more precisely, their collection of strategies define a game, to put it more precisely (Hotz, 2006).

Player1/Player2	L	R
U	1,3	2,4
D	1,0	3,3

Table 1: Example of Normal Form Games

2.4.3.2 Where is Game Theory applied?

Section 2.4.3 mentioned that game theory is a branch of mathematics. These games can be described using a common language provided by mathematics. Game theory can be applied when there is competition for a resource to explain current behavior or to enhance tactics (Hotz, 2006).

2.4.3.3 Normal Form Games

The main three components of the game in a normal form are:

- A limited number of participants.
- Each participant has been given a set of strategies.
- A payoff function that determines each participant's payoff based on his or her strategy and the strategies of the other participant.

The result of the payoff function can be represented in a matrix, the so-called payoff matrix, which shows the two players, their strategies, and their payoffs, if the number of players is restricted to two and their sets of strategies only have a few components (Hotz, 2006).

2.4.3.4 Extensive Form Games

As shown in figure 5, the rules of an extensive form game are set out so that the agents of the game carry out their moves sequentially, in contrast to the normal form game. A game tree is used to represent this game, with each node representing a different playing phase. The initial node is a special node that indicates the beginning of the game (Hotz, 2006).

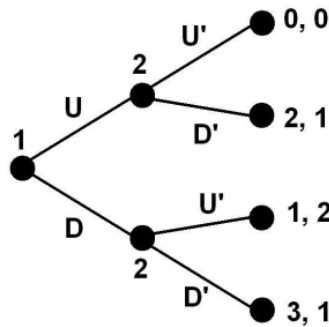


Figure 5: Example of Extensive Form Game (Han et al., 2022)

A terminal node is any node that has only one edge connected to it. It indicates the conclusion of the game (and a strategy profile). Every non-terminal node has a player attached to it in the sense that it represents a phase of the game where that player is making a move. Every edge represents a potential move that a player might make. Every terminal node has a payoff for each player connected to it. If all the necessary steps are taken to get to the terminal node, then these are the rewards for each player (Hotz, 2006).

2.4.3.5 Nash Equilibrium

The Nash equilibrium in game theory is a type of solution concept for a game involving two or more players, where no player has anything to gain by changing only his own strategy. It is named after John Nash, who first described it. The current set of strategy options and the associated payoffs constitute a Nash equilibrium if each player has chosen a strategy and no player can benefit from altering his strategy while the other players maintain theirs (Hotz, 2006).

2.4.3.6 Best Response

The best response, considering other players' strategies, is the strategy (or strategies) that results in the best immediate result for the current player. With the help of the payoff matrix and this definition, the Nash equilibrium can be determined in a normal-form game (Hotz, 2006).

2.4.3.7 Mixed Strategies

Nash equilibrium has a feature that is frequently unclear to beginners, which is highlighted by mixed strategy equilibrium. Nash equilibrium does not require a convincing reason to use the equilibrium strategy (Hotz, 2006).

Player 2 will eventually predict the opponent's strategy if, for instance, player 1 chooses to play Head with a probability of 80% and play Tail with a probability of 20%. He will therefore always play Tail as a result. Due to this, player 2 will receive a payoff of $0.8 \cdot 1 + 0.2 \cdot (-1) = 0.6$ per game, while player 1 will receive a payoff of -0.6 . In a fair zero-sum game like this, the best possible payoff is 0, which can be obtained by playing $\frac{1}{2}\text{Head} + \frac{1}{2}\text{Tail}$ (Hotz, 2006). A mixed strategy is simply a linear combination of pure strategies, to use mathematics.

2.4.4 Reinforcement Learning

Reinforcement Learning (RL) is a machine-learning technique that allows an agent to learn by interacting with an environment using a trial-and-error process and feedback from its experiences, and actions' impact on the learning environment (OpenAI, 2018).

2.4.4.1 Concepts of Reinforcement Learning

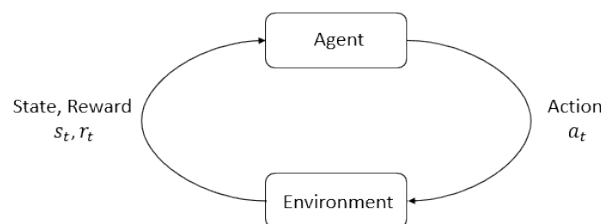


Figure 6: Agent-environment interaction loop (OpenAI, 2018)

As shown in figure 6, **the agent** and **environment** are the main components of RL. Agent lives in and interacts with the environment. Agent observes the state of the environment and decides which action to take at each step of the interaction. The environment changes based on the agent acting on it, however, it may have self-changes. The agent receives a feedback signal from the environment in the form of a **reward**. A reward is a number indicating how the agent's action was good or bad to the environment at the action time. RL methods are techniques through which the agent can learn behaviors to achieve goals.

The following sub-sections will introduce the main concepts of RL (OpenAI, 2018).

1. States and Observations

A **state** s completely describes the state of the environment with no missing information. An **observation** o partially describes the state of the environment with some omitted information. In deep RL, the states and observations are represented by a real-valued vector, matrix, or high-order tensor. For example, the RGB matrix can represent a visual observation, while joint angles and velocities can represent the state of a robot. If the agent can completely observe the state of the environment, the environment is **fully observed**. If the agent can partially observe the state of the environment, the environment is **partially observed** (OpenAI, 2018).

2. Action Spaces

There are different actions allowed by different environments. **Action space** is the set of all valid actions allowed by an environment. There are two types of action spaces, either **discrete action spaces** that allow only a finite number of actions such as **Atari** or **Go**, or **continuous action spaces** that allow real-valued vectors of actions as controlling a robot in a physical world (OpenAI, 2018).

3. Policies

A Policy is defined as the rule that an agent follows to decide the taken actions in different environmental states. There are two types of policies.

- **Deterministic policies** denoted by μ : $a_t = \mu(s_t)$.
- **Stochastic policies** denoted by π : $a_t \sim \pi(\cdot|s_t)$. The most popular types of stochastic policies in deep RL are **categorical policies** and diagonal **Gaussian policies**.

It is common to replace the word agent with the word policy because the policy is considered as the agent’s brain (OpenAI, 2018).

4. Trajectories

A **trajectory** τ is defined as a sequence of states and actions in the environment:

$$\tau = (s_0, a_0, s_1, a_1, \dots).$$

The initial state s_0 is randomly sampled from the start-state distribution p_0 : $s_0 \sim p_0(\cdot)$.

State transitions from (t, s_t) to $(t+1, s_{t+1})$ are controlled by the natural laws of the environment, and they only depend on the most recent action. They might be deterministic, $s_{t+1} = f(s_t, a_t)$, or stochastic, $s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)$ (OpenAI, 2018).

5. Reward and Return

A reward function R returns the reward value given to the agent based on the action it took. It is dependent on the current state, the taken action, and the next state of the environment: $r_t = R(s_t, a_t, s_{t+1})$. Usually, it is reduced to only $r_t = R(s_t, a_t)$. The agent aims at maximizing the cumulative value of reward over a trajectory $R(\tau)$. A return function can be either a **finite-horizon undiscounted return**, which is simply the sum of all obtained rewards overall steps: $R(\tau) = \sum_{t=0}^T r_t$, or **infinite-horizon discounted return**, which is the sum of all obtained rewards discounted by discount factor $\gamma \in (0, 1)$ over all steps: $R(\tau) = \sum_{t=0}^T \gamma^t r_t$. The discount factor is

used so the agent can choose between whether it is better to have near rewards or future rewards, so the value of infinite-horizon converges to a certain sum. A discount factor that is close to 0 leads to an agent driven by short-term rewards, while a discount factor that is close to 1 leads to an agent driven by long-term high rewards (OpenAI, 2018).

2.4.4.2 The Problem of Reinforcement Learning

Regardless of choosing any of the return functions introduced in reward and return 2.4.4, or choosing any of the policies introduced in the policies 2.4.4, the RL aims at selecting a policy that can maximize the expected return if the agent follows such a policy. Before talking about expected return, probability distributions over trajectories should be introduced. If the environmental transitions and policy are stochastic, the probability of a T-step trajectory is (OpenAI, 2018):

$$P(\tau|\pi) = p_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1}|s_t, a_t) \pi(a_t|s_t) \quad (10)$$

Regardless of choosing any of the return functions, the expected return $J(\pi)$:

$$J(\pi) = \int_{\tau} P(\tau|\pi) R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi}[R(\tau)] \quad (11)$$

The main optimization problem of RL can be defined by: $\pi^* = \arg \max J_{\pi}(\pi)$, with π^* as the optimal policy.

2.4.4.3 Value Functions

Defining the **value** of a state or state-action pair is useful in RL. It can be defined as the **expected return** value when an agent starts in that state, or a state-action pair, and follows a certain policy forever. **Value functions** are used in most of the RL algorithms (OpenAI, 2018).

There are four main value functions:

- The **On-Policy Value Function**, $V^{\pi}(s)$. It gives the expected return when the agent starts in state s and follows the policy π : $V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s]$ (OpenAI, 2018).
- The **On-Policy Action-Value Function**, $Q^{\pi}(s, a)$. It gives the expected return value when the agent starts in state s , takes a random action a , and follows a certain policy π : $Q^{\pi}(s, a) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s, a_0 = a]$ (OpenAI, 2018).
- The **Optimal Value Function**, $V^*(s)$. It gives the expected return when the the agent starts in state s and follows the optimal policy in the environment: $V^*(s) = \max_{\pi} \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s]$ (OpenAI, 2018).
- The **Optimal Action-Value Function**, $Q^*(s, a)$. It gives the expected return if the agent starts in state s , takes a random action, and follows the optimal policy of the environment: $Q^*(s, a) = \max_{\pi} \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s, a_0 = a]$ (OpenAI, 2018).

2.4.4.4 The Optimal Q-Function and the Optimal Action

The optimal policy in s will select an action that can maximize the expected return if the agent starts in s . From that, we can have the optimal action as $a^*(s) = \arg \max Q^*(s, a)$. In case more than one optimal action exists, the policy randomly selects one of them (OpenAI, 2018).

2.4.4.5 Bellman Equations

The value functions introduced in 2.4.4 follow equations called Bellman equations. The idea is the value of state s is the expected reward of being in s added with the value of the next state s' (OpenAI, 2018).

For the on-policy value functions, the Bellman Equations are:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi, s' \sim P}[r(s, a) + \gamma V^\pi(s')] \\ Q^\pi(s, a) &= \mathbb{E}_{s' \sim P}[r(s, a) + \gamma \mathbb{E}[Q^\pi(s', a)]] \end{aligned} \tag{12}$$

where $s' \sim P$ is the reduced notation of $s' \sim P(\cdot|s, a)$, which shows that the next state s' is obtained from the transition rules of the environment; a^π is the reduced notation for $a \sim \pi(\cdot|s)$; and $a \sim \pi$ is the reduced notation for $a \sim \pi(\cdot|s')$ (OpenAI, 2018).

For the optimal value functions, the Bellman Equations are:

$$\begin{aligned} V^*(s) &= \max_a \mathbb{E}_{s' \sim P}[r(s, a) + \gamma V^*(s')] \\ Q^*(s, a) &= \mathbb{E}_{s' \sim P}[r(s, a) + \gamma \max_{a'} Q^*(s', a')] \end{aligned} \tag{13}$$

The main difference between the Bellman equations in these two cases is that there is no max in the first case. This indicates that if the agent wants to act optimally, it must select the action that can lead to the highest value (OpenAI, 2018).

2.4.5 Algorithms of Reinforcement Learning

This section will introduce and compare theoretically between different RL algorithms, so one of them is selected and implemented in the prototype chapter 4. This section will include Vanilla Policy Gradient (VPG), Trust Region Policy Optimization (TRPO), Proximal Policy Optimization (PPO), Deep Deterministic Policy Gradient (DDPG), and Soft Actor-Critic (SAC) algorithms. They are core deep RL algorithms, which represent the recent evolution of the RL field. The two fundamental deep RL algorithms in this package, PPO and SAC, provide sample efficiency and reliability while also highlighting trade-offs in the development and application of deep RL algorithms (OpenAI, 2018).

2.4.5.1 The On-Policy Algorithms

An On-Policy algorithm is defined as the agent's value of the policy, along with the steps involved in the exploration process, according to an on-policy learner. An early deep learning algorithm (VPG), dates back to the late 1980s and early 1990s. Stronger algorithms like TRPO and PPO are a result of its founding principles. These algorithms compromise stability for sample efficiency and are on-policy, with a focus on policy performance. The development of techniques, however, aids in making up for this shortage (OpenAI, 2018).

2.4.5.2 The Off-Policy Algorithms

An Off-Policy algorithm is defined as the value of the best course of action discovered by an off-policy learner separate from the agent's actions. DDPG is a fundamental algorithm that simultaneously learns a Q-function and a policy. It is closely related to Q-learning algorithms. Bellman's equations enable effective data reuse in these off-policy algorithms. Nevertheless, there are no assurances of excellent policy performance, which could leave them fragile and unstable. The off-spring of DDPG, and SAC, employ a variety of strategies to address these problems (OpenAI, 2018).

2.4.5.3 Vanilla Policy Gradient

By raising and decreasing action probabilities, policy gradients seek to maximize return and produce the best possible policy. MPI parallelization is supported by the on-policy algorithm VPG. Environments with either discrete or continuous action spaces can use VPG. Using the most recent version of the policy, VPG samples actions to train a stochastic policy on the policy. The training process and initial conditions determine how random the action selection process is. The less randomness in policy could lead to local optima being trapped (OpenAI, 2018).

2.4.5.4 Trust Region Policy Optimization

TRPO updates policies by maximizing performance gains while adhering to the KL-Divergence constraint on policy distance. This is not the same as a regular policy gradient, which maintains the proximity of new and old policies in parameter space. TRPO prevents collapse and achieves rapid, steady performance improvements. MPI parallelization is supported by the on-policy algorithm TRPO. Environments with either discrete or continuous action spaces can use TRPO. Using the most recent version of the policy, TRPO samples actions to train a stochastic policy on the policy. The training process and initial conditions determine how random the action selection process is. Overtraining, the policy becomes less random, possibly trapping in local optima (OpenAI, 2018).

2.4.5.5 Proximal Policy Optimization

The goal of the PPO family of first-order techniques is to enhance policies without leading to a collapse in performance. It functions similarly to TRPO and is easier to implement. The two main variations are PPO-Clip and PPO-Penalty. While PPO-Clip employs specialized clipping to eliminate incentives for new policies to diverge significantly from old ones, PPO-Penalty penalizes KL-divergence in the objective function. PPO is an on-policy algorithm that can be parallelized with MPI and is appropriate for both discrete and continuous action spaces. Using the most recent version of the policy, PPO samples actions to train a stochastic on-policy. The training process and initial conditions determine how random the action selection process is. Less randomness in policy could lead to local optima being trapped (OpenAI, 2018).

2.4.5.6 Deep Deterministic Policy Gradient

Using off-policy data and the Bellman equation, the Deep Deterministic Policy Gradient (DDPG) algorithm simultaneously learns a policy and a Q-function. It is designed for environments with continuous action spaces and has close ties to Q-learning. Specifically designed for continuous action spaces, DDPG interleaves learning an approximator to $a^*(s)$ with learning an approximator to $Q^*(s, a)$. This is because an effective, gradient-based learning rule for a policy can be created. After all, it is assumed that the function $Q^*(s, a)$ is differentiable concerning the action argument. The use of DDPG is restricted to environments with continuous action spaces, as it is an off-policy algorithm. Agents can investigate on-policy more successfully when DDPG trains a deterministic policy off-policy. Actions are trained with noise, preferably uncorrelated Gaussian noise, to improve exploration. Higher-quality data can be obtained by reducing the noise scale during training. The efficiency of the policy is not evaluated during testing (OpenAI, 2018).

2.4.5.7 Soft Actor-Critic

By combining DDPG-style methods with stochastic policy optimization, the SAC algorithm optimizes stochastic policies off-policy. Its intrinsic stochasticity allows it to incorporate the clipped double-Q trick and to benefit from target policy smoothing. Entropy regularization, the main component of SAC, maximizes a trade-off between entropy and expected return while encouraging exploration and averting premature convergence. Only settings with continuous action spaces can

be used. To manage explore-exploit tradeoffs, SAC employs a stochastic policy with entropy regularization, where a higher α indicates more exploration and a lower α indicates more exploitation. Mean action at test time is used to assess the performance of the policy after stochasticity has been eliminated (OpenAI, 2018).

In the conception chapter 3, a decision regarding selecting one of these algorithms will be introduced.

2.4.6 Multi-Agent Reinforcement Learning

In contrast to the single-agent approach in which only one agent is considered, and all other agents are considered as a part of the environment, MARL, is a type of RL in which the optimal policy of an agent depends on the policies of other agents. In this case of a decentralized system, the coordination between agents and modeling the conflict between them is critical (Lourentzou, 2016).

2.4.6.1 Markov Decision Processes

The mathematical framework for modeling the decision-making of a single-agent interacting with a fixed environment is given by Markov Decision Processes (MDPs). So, the secondary agents are not explicitly modeled but can be considered as a part of the environment (Lourentzou, 2016). A Markov decision process can be defined by a tuple $\{S, A, T, R\}$ where:

- S is the set of states.
- A is the set of actions.
- $T : S \times A \rightarrow \mathbb{P}(S)$ is the transition function defining the probability distribution over the next states based on the current state and the selected action.
- $R : S \times A \rightarrow \mathbb{R}$ is the reward function determining the reward the agent receives when it selects a certain action in a given state.

Algorithms for solving MDPs can be classified into two groups based on the transition function. If there is a known transition function from one state to another state, it is a **Dynamic Programming** setting such as **Value iteration** and **Policy iteration** algorithms. If there is an unknown transition function, it is the RL in which algorithms such as **Q-Learning** require online updates of sampling sequences from MDP to find a solution to the optimization problem. The following subsections will introduce the **Value iteration** and the **Q-Learning** and discuss their extension to the multi-agent case (Lourentzou, 2016).

1. Value Iteration

The process starts at time step $t = 0$, the value function is randomly initialized, and the V_{t+1} is repeatedly computed for all states using the equations (Lourentzou, 2016):

$$\begin{aligned} Q(s, a) &= R(s, a) + \gamma \sum_{s' \in S} T(s, a, s') V(s') \\ V(s) &= \max_{a' \in A} Q(s, a') \end{aligned} \tag{14}$$

The Optimal policy can be computed as (Lourentzou, 2016):

$$a^* = \arg \max_{a' \in A} Q(s, a') \tag{15}$$

2. Q-Learning

In case of an unknown transition function, the agent performs an update once it receives a reward r after a transition from s to s' via an action a (Lourentzou, 2016):

$$Q(s, a) \leftarrow Q(s, a) + \eta (r + \gamma \max_{a'} Q(s', a') - Q(s, a)) \quad (16)$$

or,

$$Q(s, a) \leftarrow (1 - \eta)Q(s, a) + \eta(r + \gamma V(s')) \quad (17)$$

where η is the learning rate with an initial value of 1.0 and decays over time.

The next section will introduce the framework for Markov Games and algorithms for MARL.

2.4.6.2 Stochastic Games

The recommended framework for simulating multiple adaptive agents with interacting or competing goals is stochastic or Markov games. State transitions are still assumed to be Markovian in Markov games, but in the single-agent MDP framework, each agent has a distinct set of actions (Lourentzou, 2016). For n agents, the MDP process is extended as follows:

- Joint-action space $A = A_1 \times A_2 \times \dots \times A_n$.
- State transition $T : S \times A_1 \times \dots \times A_n \rightarrow \mathbb{P}(S)$.
- Reward functions $R_i : S \times R_1 \times \dots \times R_n \rightarrow \mathbb{R}$, which depend on the joint action of all agents.

Agent i is attempting to maximize their expected reward under a joint policy $\pi = (\pi_1, \dots, \pi_n)$, which is similar to the MDP objective. A policy π_i is assigned to each agent i by the joint policy π (Lourentzou, 2016):

$$V_i^\pi = \mathbb{E} \left[\sum_{j=0}^{\infty} \gamma^j r_{i,t+j} \right] \quad (18)$$

In contrast to the MDP case, it is possible that an optimal stationary deterministic policy does not exist, which means that an optimal stationary policy could sometimes be probabilistic. This optimal stationary policy maps states to discrete probability distributions over actions $\pi^* : S \rightarrow \mathbb{P}(A)$. The best response and Nash equilibrium in 2.4.3 concepts can be applied to these games: a policy π_i is considered as the best response if there is no other policy for agent i that provides better expected future reward when the policies of other agents are fixed (Lourentzou, 2016).

The Markov game is reduced to an MDP when only one agent is assumed or when other agents have a fixed policy. The Markov game is reduced to a repeated normal-form game if it has only one state. In this game, a common benchmark for MARL is where players simultaneously pick an individual action, perform it, and receive a reward based on their joint action. Also, no delayed reward is an important property of RL. However, if other agents' behavior or policies are considered, it is still a problem in non-stationary environments. So, the methods of this category need to adopt policies that are dependent on the actions of other agents (Lourentzou, 2016).

2.4.6.3 Algorithms for General Markov Games

Assumptions

In the general RL, the reward function and the expected reward of performing a joint action are assumed to be unknown for the agents. It is also assumed that not the same deterministic reward is given to each agent after performing joint actions. For example, there are stochastic payoffs. So, a repeated sampling for these joint actions is important. Actions of all participating agents

or rewards are assumed by most MARL methods to be observed from all agents because of the dependence on other agents' policies. This could be useful in modeling components and learning over joint actions. It could be impractical if common knowledge is assumed in distributed systems in which this information is mostly not available (Lourentzou, 2016l). MRAL algorithms can be classified into two categories:

- **Independent Learning:** It considers other agents as part of the environment.
- **Joint Action Category:** It tries to model other agents.

The next subsections will introduce some algorithms from both categories.

1. Independent Learning

In this MARL algorithms category, the multi-agent case is reduced to a single-agent learning problem. The interaction between other agents is considered noise. **SARSA** and **Q-learning** algorithms can be used in this case, however, convergence is not guaranteed because of the stochastic behavior of other agents within the environment. Additionally, the coordination mechanism, which is highly important for decentralized systems is missing (Lourentzou, 2016).

2. Joint Action Learning

In this MARL algorithms category, other agents are considered by learning in the joint action space. Therefore, the coordination between agents can be modeled. However, there is an exponential growth of the algorithm complexity with the number of agents. Making assumptions by agents about the opponent's strategies to predict their future actions is another disadvantage of this category of algorithms. When **Q-Learning** is extended to MARL, $Q(s, a)$ is estimated by the agent for selecting the joint action $a = a_1, \dots, a_n$ in the state s . The agent must keep estimates for each combination of his action a_i and the joint action a_{-i} played by the other agents because the actions of other agents may vary. As a result, the agent cannot have a single estimate for the reward it will receive by taking action a_i in state s . So, the value of the state is calculated by considering the actions of other agents. It is obvious that the higher the number of agents and their actions, the more the complexity of the algorithm (Lourentzou, 2016).

The following subsections will introduce some algorithms from the Joint Action Learning category.

3. Joint Action Learner

This algorithm estimates other agents' policies, so the expected probability with each different joint action is determined. The value of a state is calculated using the probability distribution of joint actions. In the Joint Action Learner (**JAL**) algorithm, an agent counts the instances in which the other agents in a given state play a joint action pair indicated as $c(s, a_{-i})$. To determine the value of a state, frequencies that have been normalized are used (Lourentzou, 2016):

$$V_i(s) = \max Q(s, a_i) = \sum_{c(s, a_{-i})} \frac{n(s, a'_{-i})}{a_{-i} \in A_{-i}} Q(s, a_i, a_{-i}) \quad (19)$$

Where A_{-i} is the possible action set for all other agents, and $Q(s, a_i, a_{-i})$ is the Q-value in state s for the joint action in which agent i performs a_i and other agents perform a joint action a_{-i} . The standard single-agent Q-learning algorithm's other features of the Q-learning update are the same. The low additional complexity due to storing and updating these counts is the advantage of this method. The algorithm, however, only performs well for deterministic tasks, where each agent eventually decides to take the same action in a particular state s (Lourentzou, 2016).

4. Minimax Q

By applying the minimax principle ("behave so that your reward is maximized in the worst case"), the minimax-Q algorithm prefers conservative strategies in two-player zero-sum games. In the minimax-Q algorithm, it is assumed that the agent adversary will perform the action that will minimize the agent's payoff, so the optimal agent's minimum expected reward is intended to be maximized. The optimal strategy in the case of two-player zero-sum games is (Lourentzou, 2016):

$$\pi^* = \arg \max_{\pi \in \mathcal{P}(A)} \min_{o \in O} \sum_{a \in A} R_{o,a} \pi_a \quad (20)$$

Where $R_{o,a}$ is a reward for the agent acting with a , and the opponent taking action o .

By replacing the minimax value for the max operator in single-agent Q-learning, other algorithms, including Value Iteration and Q-learning, can be easily extended. For example, Value Iteration can be extended as (Lourentzou, 2016):

$$\begin{aligned} Q_i(s, a, o) &= R_i(s, a, o) + \gamma \sum_{s' \in S} T(s, a, o, s') V_i(s') \\ V_i(s) &= \max_{\pi \in \mathcal{P}(A)} \min_{o \in O} \sum_{a \in A} Q_i(s, a, o) \pi_a \\ \pi^* &= \arg \max_{\pi \in \mathcal{P}(A)} \min_{o \in O} \sum_a Q_i(s, a, o) \pi_a \end{aligned} \quad (21)$$

Additionally, Q-learning can be extended as:

$$\begin{aligned} Q_i(s, a, o) &\leftarrow (1 - \eta) Q_i(s, a, o) + \eta (r + \gamma V_i(s')) \\ V_i(s) &= \min_{o \in O} \sum_{a' \in A} \pi_i^*(s, a') Q_i(s, a', o') \end{aligned} \quad (22)$$

$$\pi_i^*(s, a) \leftarrow \arg \max_{\pi_i(s, a')} \min_{o \in O} \sum_{a' \in A} \pi_i(s, a') Q_i(s, a', o') \quad (23)$$

Linear programming can be used to solve the optimization problem in equation 23. Even if the minimax optimization has multiple solutions, any of them will, regardless of what the opponent does, produce at least the minimax return. Furthermore, it shows that we might achieve better results when the adversary is suboptimal (does not always select the optimal action). Convergence is only assured in two-player zero-sum games and under the condition that the other agent performs each action infinitely frequently. Although convergence also holds when the other agent does not converge to the equilibrium, it offers an opponent-independent method for learning an equilibrium solution. Minimax, on the other hand, is extremely cautious because it aims to maximize performance under the worst-case scenario. In certain scenarios, it might be a better choice to change the risk taken to increase the reward relative to the opponent's actions (Lourentzou, 2016).

5. Friend-or-Foe Q-learning

Other agents are divided into two categories in this setting: friends (coordination) with updates like Q-learning or foes (opponents) with updates similar to minimax-Q. The authors show the updates for the two-player game even though it is defined for any number of players:

Friend: $V_1(s) \leftarrow \max Q_1(s, (a_1, a_2))$, where $a_1 \in \mathcal{A}_1, a_2 \in \mathcal{A}_2$.

Foe: $V_1 \leftarrow \max_{\pi_1 \in \prod(\mathcal{A}_1)} \min \sum_a \pi_1(a_1) Q_1(s, (a_1, a_2))$, where $\pi_1 \in \prod(\mathcal{A}_1), a_2 \in \mathcal{A}_2$.

Friend-or-Foe Q-learning converges in general-sum Markov games, but equilibrium policies are constrained in these two classes of games (Lourentzou, 2016).

2.4.6.4 Drawbacks of Multi-Agent Reinforcement Learning

The drawbacks of MARL are introduced in the following subsections.

1. Exponential Growth of State-Action Space

This is known as the curse of dimensionality. The complexity of MARL becomes exponential in the number of agents, making it infeasible for large Markov Games and environments with a high number of agents (Lourentzou, 2016).

2. Limited Scalability

MARL algorithms are typically evaluated on small problems, and scaling up to larger or continuous state and action spaces is challenging. This limitation makes the application of MARL in real-world scenarios that involve complex environments with numerous agents more difficult (Lourentzou, 2016).

3. Non-Stationarity

In MARL, the best policy for an agent changes as the policies of other agents change. This non-stationarity introduces additional challenges for achieving convergence in the general Markov game case. The dynamic nature of multi-agent interactions makes it difficult to find a stable equilibrium policy (Lourentzou, 2016).

4. Incomplete or Uncertain Observations

Limited research has been done in the case of incomplete or uncertain observations in MARL. Partially observable states or games with incomplete information introduce complexities in modeling the belief over the state variable. Dealing with unbounded state space and nested beliefs becomes challenging, limiting the practical application of MARL in such scenarios (Lourentzou, 2016).

5. Tractability of Optimal Policies

Computing optimal policies in general Partially Observed Stochastic Games (POSGs) is not known to have a tractable solution. While some approximate probabilistic inference approaches have shown convergence in certain cases, there is no known general solution. This limitation restricts the applicability of MARL in complex games with partial observability (Lourentzou, 2016).

6. Insufficiency of Q-values

Some academics contend that learning an equilibrium strategy in general sum games may not be possible simply by using Q-values. The limitations of Q-values imply that alternative approaches, such as evolutionary game theory, may be required to improve MARL performance. However, more research in this area is necessary (Lourentzou, 2016).

2.4.6 Adversarial Multi-Agent Reinforcement Learning

With the assumption that agents' policies are based on accurate state information, many MARL techniques have been developed. **Deep Reinforcement Learning** (DRL) -learned policies, however, are susceptible to adversarial state perturbation attacks. This paperwork makes the first

attempt to investigate the basic characteristics of MARL under state uncertainties and proposes a **State Adversarial Markov Game (SAMG)**. This study shows that SAMGs often lack robust **Nash equilibrium** and **optimal agent policy** solution concepts. It considers a new idea for a solution called robust agent policy to get around this problem, where agents try to maximize the worst-case expected state value (Han et al., 2022).

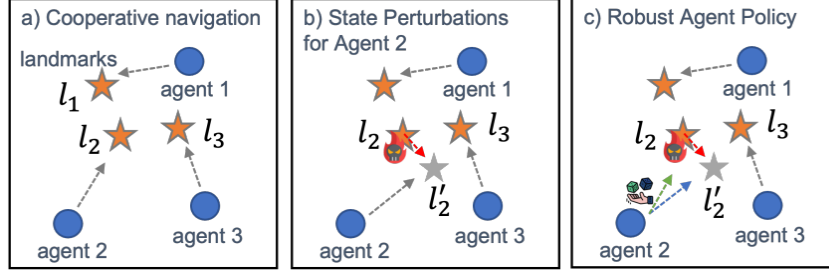


Figure 7: Adversarial interference in landmark covering problem (Han et al., 2022)

In figure 7, the agents' objective in this scenario is to occupy and cover every landmark, so choosing which landmark to cover will require cooperation. The ideal target landmark for each agent in the absence of adversarial interference is shown in figure (a). However, in figure (b), an adversary messes with agent 2's state observation ($l_2 \rightarrow l'_2$), sending it in the wrong direction. This research demonstrates how adversarial state perturbations can quickly corrupt conventional agent policies. It suggests a robust agent policy that maximizes average performance under the worst possible state perturbations to avoid this problem (Han et al., 2022).

2.4.6.1 State-Adversarial Markov Game

The State-Adversarial Markov Game (SAMG) with n agents is formulated as:

- The Agent Set $N = \{1, \dots, n\}$. Each agent i is associated with an action $a_i \in \mathcal{A}_i$.
- The global joint action $a = (a^1, \dots, a^n) \in \mathcal{A}$, where $\mathcal{A} = \mathcal{A}^1 \times \dots \times \mathcal{A}^n$.
- The global joint state $s \in \mathcal{S}$.
- The stage-wise reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, shared by all agents.
- The total expected reward $\sum \gamma^t r_{t+1}(s_t, a_t)$, where γ is a discount factor.

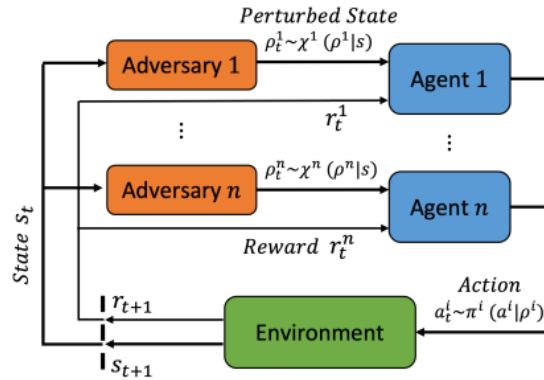


Figure 8: MARL under adversarial state perturbations (Han et al., 2022)

It is assumed that each agent has a corresponding adversary, as depicted in figure 8. As the corresponding agent's perturbed knowledge or observation of the global state, each adversary

determines a perturbed $p_i \in \mathcal{S}$ state for it. The joint perturbed state is designated as $p := [p_i]_{i \in N}$. This research models the difficulties of obtaining accurate states for multi-agent systems like connected and autonomous vehicles and multi-robot systems using the admissible perturbed state as a task-specific "neighboring" state of s , such as the bounded sensor measurement errors (Han et al., 2022).

To limit the perturbed state to only falling within a specific subset of states $p \in P_s$, an admissible perturbed state set P_s is defined as follows:

Admissible Perturbated State Set: The set of admissible perturbated states for agent i as $P_s^i \subseteq \mathcal{S}$ is considered. The joint admissible perturbed state set at state s as P_s is defined as: $P_s := P_s^1 \times \dots \times P_s^n$.

The admissible perturbated state set includes the true state ($s \in P_s^i$) for any $i \in N$. For example, a 2-agent 3-state system with $\mathcal{S} = \{s_1, s_2, s_3\}$. If the current true state is s_1 for both agents, the state observation of agent 1 is perturbed by adversary 1 within $P_{s_1}^1 = \{s_1, s_2\}$, while the state observation of agent 2 is perturbed by adversary 2 within $P_{s_1}^2 = \{s_1, s_3\}$.

2.4.7 PettingZoo API for Multi-Agent Reinforcement Learning

To improve access to MARL research and act as a multi-agent **Gym** substitute, the **PettingZoo** library and API were developed. Before PettingZoo, the many single-use MARL APIs almost exclusively derived their design from the two most well-known mathematical models of games in the MARL literature. **Partially Observable Stochastic Games** (POSGs) and **Extensive Form Games** (EFGs). It was found that these game models are not clear for multi-agent games from the implementation point of view. As a solution, the **Agent Environment Cycle** (AEC) was introduced as a new game model that can be used as the basis of the PettingZoo API. AEC can also represent the standard POSG model, and POSG can represent the AEC model (Terry et al., 2021).

The Gym API uses the Partially Observable Markov Decision Process (POMDP) conceptualization of RL and is a relatively simple Python API. The pervasive POMDP model, which is used as the pervasive mental and mathematical model of RL, naturally complements the API because of its simplicity and conceptual clarity. This makes it simpler for anyone with knowledge of the RL framework to comprehend Gym's API in its entirety (Terry et al., 2021).

```
import gym
env = gym.make('CartPole-v0')
observation = env.reset()
for _ in range(1000):
    action = policy(observation)
    observation, reward, done, info = env.step(action)
```

Figure 9: An example of the basic usage of Gym (Terry et al., 2021)

2.4.7.1 Partially Observable Stochastic Games and RLlib

The **POMDP** model used in single-agent reinforcement learning serves as the universal mental and mathematical model for MARL. This model is comparable to multi-agent MDPs (Boutilier, 1996), Dec-POMDPs (Bernstein et al., 2002), and stochastic **Markov** games (Shapley, 1953) while being strictly more general. In a POSG, all agents move in parallel, observe in parallel, and receive parallel rewards (Terry et al., 2021).

Applying single-agent RL techniques to multi-agent environments has become much simpler because of this model (Terry et al., 2021). However, this model has the following two immediate issues:

- It is necessary to constantly pass dummy actions for inactive agents (or employ similar techniques) to support strictly turn-based games like chess (Terry et al., 2021).

-
- It is awkward to change the number of agents for agent death or creation because learning code requires adapting to lists that are suddenly different in size (Terry et al., 2021).

2.4.7.2 OpenSpiel and Extensive Form Games

Extensive Form Games (EFG) are typically used by MARL researchers to mathematically model strictly turn-based games for which POSG models are inadequate (such as Chess). Every actionable combination is explicitly represented as a root-to-leaf path in the EFG’s representation of games as a tree. By including a “Nature” player sometimes referred to as “Chance” who makes decisions based on a predetermined probability distribution, stochastic aspects of a game (or MARL environment) are captured. Methods like game theoretic analysis and tree search have been used to solve the theory of mind problems using the EFG model (Terry et al., 2021).

However, the EFG model raises three immediate issues for application in general MARL problems:

- This environment API is significantly more complex than Gym’s API or RLLib’s POSG API, for example, is not suitable for beginners the way Gym is. The model and the corresponding API are also very complex compared to that of POSGs. The EFG model’s complexity prevents RL researchers from using it as frequently as they do the POSG or POMDP model as a mental representation of games (Terry et al., 2021).
- While RL frequently needs frequent rewards, the formal definition only includes rewards after games. Although there are ways to get around this in the API implementation, it is not ideal (Terry et al., 2021).
- Although this was a decision and not a requirement of the EFG model, the OpenSpiel API does not support continuous actions (a typical and significant case in RL) (Terry et al., 2021).

2.4.7.3 PettingZoo Design Goals

The following objectives served as the foundation for both the general library and API that we developed for PettingZoo (Terry et al., 2021).

1. Be like Gym

Gym’s simplicity, ubiquity, and universality were all advantages maintained in PettingZoo (Terry et al., 2021). As a result, there are two specific objectives:

- Make the API feel and look like a Gym, and in connection, make the API simple and Pythonic.
- Add numerous game reference implementations to the main package (Terry et al., 2021).

Using as many of Gym’s design metaphors as possible will make it easier for PettingZoo’s enormous user base to comprehend its API right away. As Gym did, an API needs to support a wide range of useful environments to draw users and encourage adoption if it is to become widely used.

2. Be a Universal API

A Gym-like MARL API must be capable of supporting all use cases and environment types if one is to exist (Terry et al., 2021). Consequently, several technically challenging cases require careful consideration (Terry et al., 2021):

- Environments with many agents.
- Environments that allow for the creation and death of agents.

- Environments in which various agents may be selected to take part in each episode.
- Methods for learning that demand access to specialized low-level features.

2.4.7.4 The Agent Environment Cycle Games Model

AEC game was created as a result of issues encountered when POSG and EFG models were attempted to be applied to MARL APIs. In this model, agents sequentially see their observations, agents act, other agents emit rewards, and the agent who will act next is selected. This is essentially a POSG model that steps sequentially (Terry et al., 2021).

There are many advantages to modeling multi-agent environments sequentially for APIs (Terry et al., 2021):

- It enables more precise attribution of rewards to various sources, enabling various learning advancements.
- It prevents programmers from including ambiguous and simple race conditions.
- It more accurately simulates the way that code is used to run computer games.
- Although it formally permits rewards after each step as is necessary for real life, the EFG model does not typically include it.
- Because learning code does not involve considering lists that are constantly changing in size, changing the number of agents for agent death or creation is less awkward.
- When compared to simultaneous stepping, it is the universal API option that is least undesirable.
- In contrast to sequentially stepping agents who could all act simultaneously and queuing up their actions, simultaneous stepping needs the use of no-op actions, which are very challenging to manage.

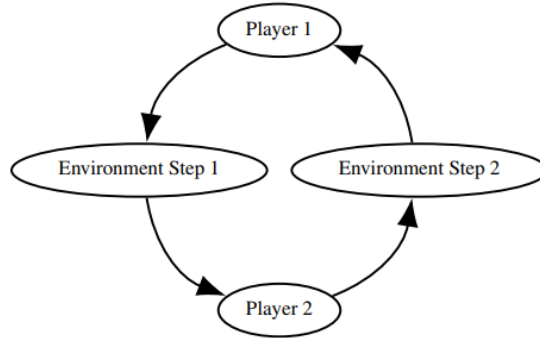


Figure 10: The AEC diagram of chess (Terry et al., 2021)

The AEC games model differs from the POSG model by including the "environment" agent, which is comparable to the Nature agent from EFGs. When this agent acts as the model, it denotes that the environment is being updated, realizing, and responding to submitted agent actions. This enables a discussion of games with odd updating rules and race conditions as well as a more thorough attribution of rewards, causes of agent death, and discussion of those games. Figure 10 illustrates a chess transition, where the term "agent environment cycle" originated (Terry et al., 2021).

2.4.7.5 API Design

The PettingZoo API is depicted in figure 11, and it should be clear from this comparison how closely it resembles the Gym API (figure 9): each agent submits an action to a step function, and in return, they are given the return values observation, reward, done, and info (Terry et al., 2021).

```
from pettingzoo.butterfly import pistonball_v0
env = pistonball_v0.env()
env.reset()
for agent in env.agent_iter(1000):
    env.render()
    observation, reward, done, info = env.last()
    action = policy(observation, agent)
    env.step(action)
env.close()
```

Figure 11: The AEC diagram of chess (Terry et al., 2021)

The same space objects that are used in the gym are also used in the observation and state spaces. The render and close methods perform similarly to Gym’s in that they always display a visual frame that represents the environment on the screen. The observation and state spaces utilize the same space objects as the Gym. A current visual frame representing the environment is displayed on the screen whenever the render and close methods are called, the same as the Gym does. The reset method performs the same function as Gym in that it restores the environment to its initial configuration after being used. PettingZoo only deviates from the standard Gym API in two ways: the last and agent_iter methods and their associated iteration logic (Terry et al., 2021).

1. The agent_iter Method

The next agent that the environment will act upon is returned by the environment’s agent_iter method, which is a generic method. The next agent that the environment will act upon is returned by the environment’s agent_iter method, which is a generic method. Any problems relating to shifting agent orders, agent generation, and agent death are cleanly abstracted away because the environment provides the next agent to act. This generation operates similarly to the next agent function in the AEC games model. This approach enables the support of every possible variation of the set of agents changing while allowing for one agent to act simultaneously (Terry et al., 2021).

2. The last method

The fact that one agent sees the other agents as elements of the environment is an odd feature of multi-agent environments. In the case of a multi-agent system, an agent must wait for all other agents to act before its observation, reward, done, and information can be fully determined. In the case of a single agent, the observation and rewards can be given right away. The last method provides these values as a result, and they can then be used to select an action by being passed into a policy. Features like altering agent orders (similar to the reverse card in Uno) would not be possible with less reliable implementations (Terry et al., 2021).

3. Additional API Feature

In the PettingZoo framework, researchers are provided with access to lower-level attributes and methods, in addition to the standard higher-level API. These features allow for more flexibility and enable researchers to conduct novel and unconventional experiments in the field of MARL (Terry et al., 2021):

- **Agents Attribute:** It provides a list of all agents in the environment, represented as strings.

-
- **Rewards, Dones, Infos Attributes:** These attributes are dictionaries keyed by agent names, storing the instantaneous rewards, done flags, and additional information for each agent. They allow access to agent properties throughout a trajectory, regardless of which agent is selected.
 - **action_space(agent) and observation_space(agent) Methods:** These functions return the static action and observation spaces, respectively, for a specified agent. Researchers can use these to understand the action and observation capabilities of individual agents.
 - **observe(agent) Method:** By passing the name of an agent as an argument, this method provides the observation specifically for that agent. It is particularly useful when observing an agent in an unconventional context.
 - **State Method:** This optional method returns the global state of the environment. It is specifically designed to support centralized critical methods that require access to the overall state information.
 - **Agent_selection Method:** This method returns the agent that is currently able to be acted upon in each agent iteration.

The PettingZoo framework aims to be universal in an emerging field, offering a straightforward set of lower-level features in addition to the standard API. It includes an optional attribute called "possible_agents" that lists all the agents that might exist in the environment at any given point. For environments that need to include environment agents following the formal AEC games model, the standard practice is to include an agent named "env" in the "agents" list with None as its action (Terry et al., 2021).

4. Environment Creation and the Parallel API

PettingZoo environments use primitive methods like reset, seed, step, observe, render, and close, along with attributes like agents, rewards, dones, infos, state, and agent_iter. These environments make it easier to create new ones and reduce code duplication. They also make it simple to switch to different APIs by writing a new wrapper. For default environments, a "parallel API" was added via a wrapper similar to the RLlib POSG-based API. This secondary API reduces Python function calls in environments with high agent density, enhancing runtime (Terry et al., 2021).

3 Conception

3.1 Description of the Conception

In this chapter, the concept of converting a DTMC model into a zero-sum game in which AI agents are trained to win this game is introduced. From the agent's behavior through the game, the game theory can be used to investigate the safety analysis of the system. The diagram in figure 12 shows the system software architecture to be implemented through this project. The following subsections will introduce each block of the diagram.

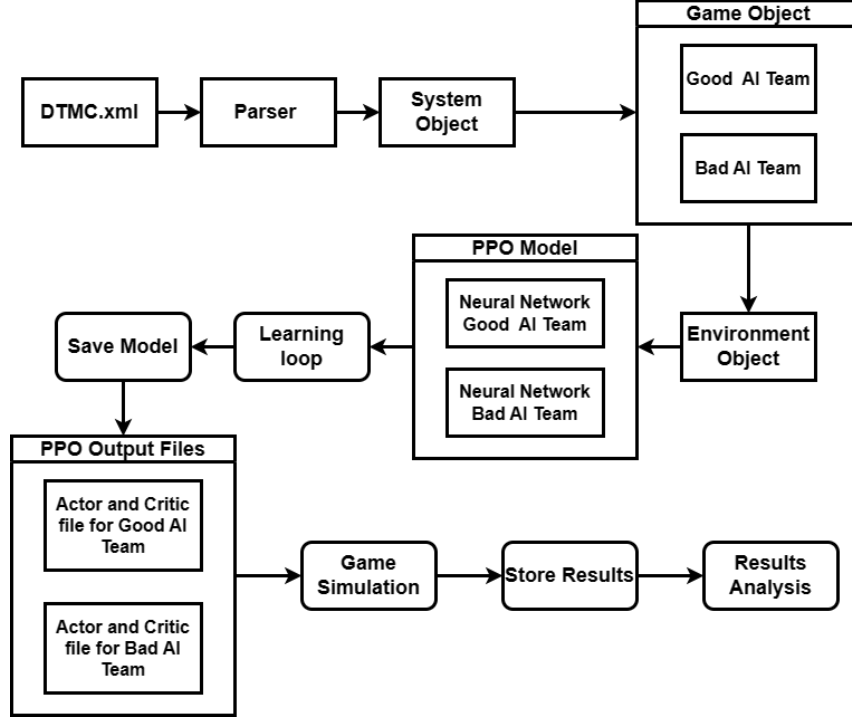


Figure 12: System Software Architecture

3.1.1 DTMC Model Parsing

As it was introduced in the Basis chapter 2.4.2, the DTMC model can be defined as a tuple $= \{S, E, P\}$. Where S is a set of system states, E is a set of edges connecting the states, and P is a set of transition probabilities, one for each edge. This model can be stored in the form of an XML file as follows:

```
<model type="DTMC" name="Model_Name">
  <define-state name="State_Name" number="State_Number" type="State_Type"
    state_Undesirability="State_Unesirability"/>
  <define-edge from="Source_State" to="Target_State" value="Transition_Probability"/>
</model>
```

Where,

- Model_Name is the name describing the model, for example, "CleaningRobot".
- State_Name is the name describing the state, for example, "S0(start)".
- State_Type is the type of the state $\in \{"transient", "absorbing"\}$.

-
- **State_Undesirability** is a number describing the state desirability from $-x$ representing the most desirable state (Safe state) to x representing the least desirable state (Failure or End state), where $x \in \text{integers}$.
 - **Source_State** is the tail of an edge $\in [0, \text{number of states} - 1]$.
 - **Target_State** is the head of an edge $\in [0, \text{number of states} - 1]$.

3.1.1.1 Parsing Object

This python object takes the file model file introduced in section 3.1.1, for example, "cleaningrobot.xml" parses this file based on its extension and returns a system object which will be introduced in the next sub-section.

3.1.2 System Object

This Python object is the output of the parsing object. It abstracts all the system attributes such as system name, for example, "CleaningRobot", system type, "DTMC", and system objects (states and edges). This Python object should also include all the system functions that handle the system objects, and dynamics, for example system reset function. The system object should be passed to the game object introduced in the next subsection.

3.1.3 Game Object

This Python object should be created based on the system object or objects passed to the game object. It abstracts the game attributes such as system objects, players, number of players, initial, current, and ending states, and current steps. It also should include the functions that handle the game dynamics and simulate the game.

3.1.3.1 Player Object

This Python object abstracts the player's attributes such as player name, valid actions, game score, and strategy type. Each player belongs to the good AI team that tries to move the system to its success or safe-ending state or the bad AI team that moves the system to its failure state.

3.1.4 Environment Object

This Python object should be created based on the AECEnv environment from the PettingZoo library introduced in the Basis chapter 2.4.7. It should take the game object as an input and extract all the game information needed to set the environment attributes, such as agents, action spaces, observation spaces, number of states, and number of actions. It also should include the functions that handle the environment dynamics, such as environment reset, environment step, and reward functions.

3.1.5 PPO Model

This thesis work selected PPO from the algorithms introduced in section 2.4.5. Selecting PPO for the thesis task depends on various factors, and it's important to consider the characteristics of the thesis problem and the strengths of different algorithms. Table 2 introduces the reasons for selecting PPO.

Characteristic	Explanation
Stability and Robustness	The goal of PPO is to create a reliable and stable algorithm. It has a clipping mechanism built in to help stop big policy updates, which can cause training instability.
Simplicity of Implementation	When it comes to implementation, PPO is not as complicated as some other sophisticated algorithms for RL. It successfully balances efficacy and simplicity.
Sample Efficiency	PPO is known for having a higher sample efficiency than some other algorithms for policy optimization. During training, it utilizes gathered samples more efficiently.
Applicability to Continuous Action Spaces	PPO is adaptable and useful for a variety of tasks because it can be applied to problems with both discrete and continuous action spaces.
Good Empirical Performance	PPO has shown strong empirical performance across a range of benchmarks and domains. It has been effectively used for tasks in complex environments such as gaming and robotics.
Parallelization	Due to PPO's parallelization capabilities, training can be accelerated by utilizing multiple environments or parallel processing.
Tunable Parameters	Compared to some other algorithms, PPO has a few hyperparameters and is generally less sensitive to their values. This may facilitate tuning.

Table 2: PPO Characteristics

The PPO Python object should be responsible for the training process of the game players within the environment objects. It abstracts the PPO algorithm shown in figure 13, and PPO model attributes such as the neural network, the actor-network, and the critic network for each agent. Both networks are trained simultaneously. The actor-network figures out the move of the agent, so what will be the next state of the system if the agent selects a certain action? However, the critic network criticizes the output of the actor-network, so if the agent takes a certain action what is the value of the reward it gets back from the environment? The PPO model also includes the functions that perform the learn and policy render tasks.

Algorithm 1 PPO, Actor-Critic Style

```

for iteration=1,2,... do
  for actor=1,2,...,N do
    Run policy  $\pi_{\theta_{old}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{old} \leftarrow \theta$ 
end for

```

Figure 13: PPO algorithm pseudocode (Schulman, Wolski et al., 2017)

After introducing each of the system software architectures the following section will introduce the system flow and how these blocks are connected.

3.1.6 System Software Architecture

As shown in figure 12, the system engineering models, which is the DTMC model in this project are passed to the parser object of the system software. The parser object reads the model file and parses a system object based on the file extension either .xml, .json, ...etc, In this project, the .xml format, and one system object will be used. The created system object in the parsing step is passed as attributes to the game object. The game object includes two teams of agents,

one represents the good AI team whose actions lead the system to safe states, and the other one represents the bad AI team whose actions try to lead the system to failure states. The game object is then passed as an attribute to the environment object at which both teams are trained to win the game according to the game rules. The environment object is passed as input to the PPO model, which trains the environment agents to win the game for a certain number of episodes. Then the output of the PPO model will be saved in the form of .pth files as an actor, and critic files for both good and bad AI teams. After the training step is done, a game simulation step at which both teams will take turns to play the game against each other and the game results will be stored in the form of .csv, and .txt files to be analyzed later in the results analysis step.

3.2 Game Description

As was mentioned in the section 3.1.3, the game is created based on the system object parsed from the XML file at which the DTMC model is stored in the form of states, states' undesirability, and edges. Once the game is created, an agent for each team, one Blue (Good AI), and one Red (Bad AI), are created. Each agent is given initial resources and should pay the cost for each action it takes during the game, so the actions of the agents are limited by their resources. The action cost is inversely proportional to the transition probability between the states before and after applying the action, for example, $cost_{s_0, s_1} \propto \frac{1}{p_{s_0, s_1}}$. The game starts with an initial state similar, extracted from the system object, which represents the initial state of the DTMC. The game starts, and each agent takes roles to do actions to the system till one of the game termination conditions is achieved, and one of the agents wins or the game ends, with a draw.

3.2.1 Player Strategy

Each player tries to apply an action to the system from which it obtains points. Once the player selects, one action to apply to the system, an action masking step is applied to check if the agent has enough resources to apply this action, otherwise, the action is masked to a "None" value, and the system remains, at the same state. If the system has a new state, an undesirability difference is calculated between the new, and old states, and points are added to the score of the agent that has the game rule at this step as follows:

- If the system is moved from low desirable states to high desirable states, the Blue agent gains points, while the Red agent loses points.
- If the system is moved from high desirable states to low desirable states, the Red agent gains points, while the Blue agent loses points.
- Number of gained or lost points equals the calculated difference of undesirability between the new, and old states.

3.2.2 Game Termination Conditions

The game is terminated and the winning team is obtained if one of the following conditions are met:

- The game reached a maximum number of game steps.
- The game reached a DTMC absorbing state.
- The undesirability threshold of the system is reached.
- The resources of both agents are consumed, and agents can not do actions anymore.

One, or more termination conditions might be applied based on the experiment description in chapter 5.

3.3 Environment Description

As was mentioned in the section 3.1.4, the environment extracts all the system info from the game object. This section will deeply describe the action, and observation spaces, the step function, and of reward function of the learning environment.

3.3.1 Observation Space

The environment has a discrete observation space containing all the system states. The **agent_selector** of the environment rolls the turns between the environment agents, and the agent with the current role observes the environment state and applies an action according to it to maximize its reward value.

3.3.2 Action Space

The environment has a discrete space containing all possible actions to the environment. Each action leads to moving the system to one of its possible states. For example, if the system has 16 states, the action space is discrete $[0,15]$, and one action corresponds to one state.

3.3.3 Action Masking

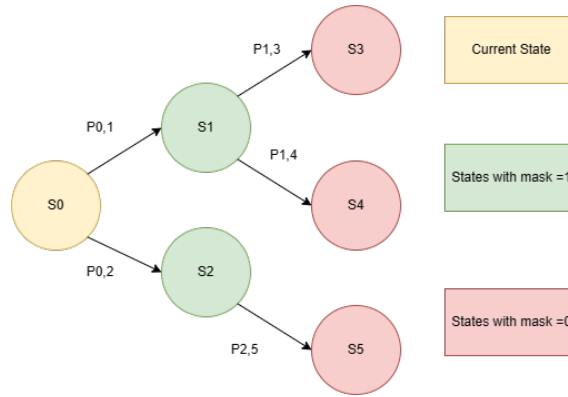


Figure 14: Action Masking Example

The environment applies action masking before the agent selects one possible action from the action space. According to the definition of the DTMC, at each system state, there are only some states connected with transition edges to the current state, the agent selects one of these states to move the system to this selected state. Based on that all possible actions are masked to zero, so the agent can not select one of them, except actions lead to these connected states as shown in figure 14. After that, the agent can only apply this action in case that it has enough resources.

3.3.4 Step Function

The step function is the function in which the interaction between the agents and the environment. The environment agent_selector selects the agent of this environment step. If the agent is dead, the function is immediately terminated. Otherwise, the agent applies its action for the current step, and the environment possibly has a new state. The undesirability difference between the old and new states of the environment is stored, so the reward of the agent is calculated afterward. When all agents applied took rounds to interact with the environment, the reward of all agents is calculated and provided based on the reward function of each agent. The Episode ends once the game ends with one of the conditions mentioned in section 3.2.2.

3.3.5 Reward Function

The reward function gives rewards to each agent based on the strategy of the agent (either Blue or Red), and the undesirability difference between the new and old states as the following equation as introduced in section 3.2.1:

$$R = A(a_k) \cdot \sum_{k=0}^n [U(s_k) - U(s_{k+1})] \cdot \gamma^k \quad (24)$$

where R is the Reward value, $A(a_k)$ the player strategy is -1 for Bad AI and 1 for Good AI, and $U(s_k)$ state undesirability represents how the state is desirable -ve values for desirable states and +ve values for undesirable states, and γ is a discount factor = 0.99.

3.4 Summary

The described system involves converting a DTMC model into a zero-sum game, training AI agents to play the game, and using game theory for safety analysis. The architecture comprises several components:

- **DTMC Model Parsing:** Parse DTMC models from XML files to extract states, edges, and probabilities.
- **System Object:** Abstracts DTMC attributes and dynamics, such as system name, type, states, and edges.
- **Game Object:** Created using the system object, it manages game attributes, players, and dynamics.
- **Player Object:** Abstracts player attributes, teams (good and bad AI), and strategies.
- **Environment Object:** Created from the PettingZoo library's AECEnv, sets up the environment for training.
- **PPO Agent Object:** Handles training of the environment agents, including neural networks and learning.

The system flow involves parsing models, creating system and game objects, training AI and agents using PPO, and simulating games for analysis. Chapter 4 will introduce detailed documentation for the implementation of these software components.

4 Prototype Description

This chapter details the implementation of a prototype that transforms a DTMC model into a zero-sum game where AI agents undergo training to win this game. The subsequent diagram in 15 illustrates the prototype for this project's implementation. The subsequent sections will introduce each component indicated in the diagram.

As shown in figure 15, the project involves utilizing system engineering models, including a DTMC model, which is processed by a parser object within the system software. The parser reads the model file and extracts the system object based on file extensions like .xml or .json. In this case, the .xml format is employed. This system object is then used as an attribute within a game object.

The game object consists of two teams of agents. One represents a good AI team focused on actions leading to safe system states, while the other represents a bad AI team aiming to trigger system failures. This game object is subsequently passed to an environment object. Both teams are trained within this environment, adhering to the game rules. The environment object is then input into a PPO model, which trains the AI agents over a specified number of episodes. The PPO model's output is saved as .pth files, containing actor and critic data for both the good and bad AI teams.

Upon completing the training phase, a game simulation occurs where the two AI teams take turns playing against each other. The outcomes of these game rounds are stored in .csv and .txt files for later analysis during the results analysis phase and testing of the system components.

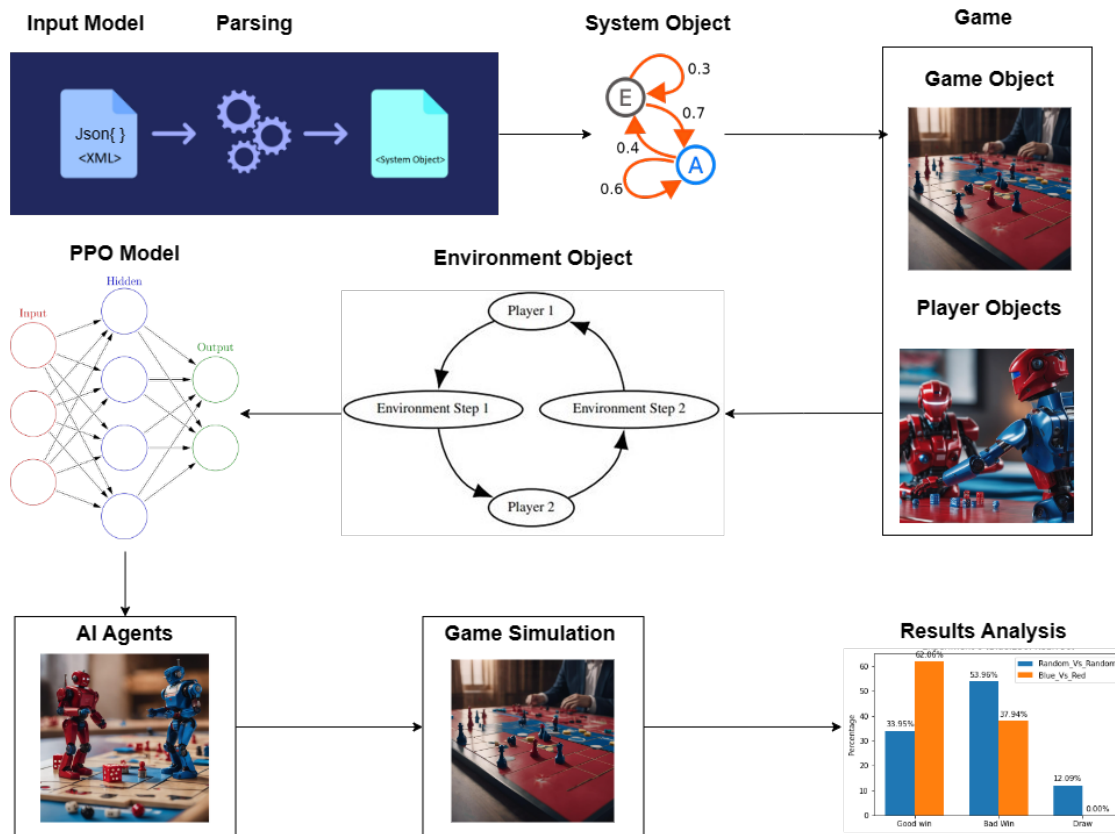


Figure 15: Prototype Implementation

The following subsections will describe the system components shown in figure 15.

4.1 Description of the System Components

4.1.1 Parser Object

As it was introduced in the conception chapter 3.1.1, a parser object is used to parse different input models such as DTMC, and Dynamic Fault Tree DFT in different file formats such as XML and JSON, and returns a system object that contains all the system information. Appendix A.1 includes detailed documentation about the classes and functions included in the parser object.

4.1.2 System Object

As was introduced in the conception chapter 3.1.2, the system object is the output of the parsing process. Appendix A.2 includes detailed documentation about the classes and functions included in the system object.

4.1.3 DTMC Object

It is a Python object that implements the main two elements of the DTMC model which are states and edges. Appendix A.3 includes detailed documentation about the classes and functions included in the DTMC object.

4.1.4 Game Object

As it was introduced in the conception chapter 3.1.3, the game object extracts the game information from the system object and creates and structures it in the form of the game. Appendix A.4 includes detailed documentation about functions included in the DTMC Model Generator.

4.1.5 Environment Object

As it was introduced in the conception chapter 3.1.4, the environment object extracts all the system information from the game object and implements all the methods needed to train the adversarial agents to win the designed game. The following subsections describe the classes implementing the game object with different methods.

4.1.6 PPO Model

As it was introduced in the conception chapter 3.1.5, the PPO Object is responsible for training the environment agent using the PPO algorithm introduced in the basics chapter 2.4.5.

Appendix A.6 includes detailed documentation about the classes and functions included in the PPO agent object.

4.1.7 DTMC Model Generator

This Python module was implemented to generate random DTMC models in the form of XML files. It also visualizes the generated DTMC in the form of directed graphs, and transition matrix heat maps. Appendix A.7 includes detailed documentation about functions included in the DTMC Model Generator.

Chapter 5 will be an experimental chapter introducing different experiments that investigate the results of the game and the behavior of the agents under different game conditions.

5 Experiments and Results

This chapter represents the thesis' results designed to investigate and validate the hypothesis introduced in the preceding basis, conceptual, and prototype implementation chapters. Through several experiments that investigate the behavior of different DTMC models under the conditions of changing one or more model variables, such as the total number of training episodes, the undesirability of system states, the number of state transitions, and the initial resources of each agent, and the order of moves of players. The results of experiments were reported in text files, which appear in the appendix chapter in the form of the win rate of each team.

Figure 16 shows the general structure of the DTMCs used for all upcoming experiments in chapters 5, and 6, where the transition probability between s_0 and s_1 , $p_{0,1} = 1.0$. From s_1 , the system can reach any other state. Once the system reaches s_{14} , there is a possibility of 0.9 to go back to starting state s_0 , and a possibility of 0.1 to go to the ending absorbing state s_{15} with $p_{15,15} = 1$. Each DTMC includes 16 states, and a maximum number of transitions from one state to another state = 3 except for experiment 5 where the number of transitions outgoing from one state to on is a parameter investigated by this experiment. The DTMCs were randomly generated by the DTMC Model Generator introduced in 4.1.7.

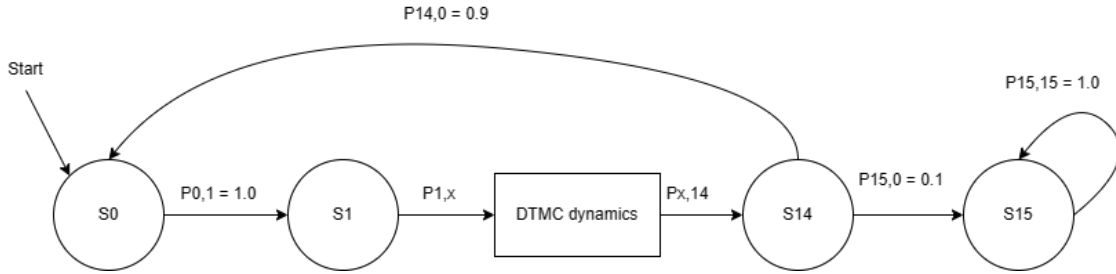


Figure 16: General Structure of the DTMCs used for the experiments

5.1 Experiment 1: Results Difference

Experiment Question

What happens if one experiment is repeated multiple times?

Experiment Expectations

This experiment expects a small difference in results if agents were trained for the same total episodes.

Experiment Input

The DTMC shown in Figure 33 (Experiment 1) was used in this Experiment. It has 50% desirable states, and 50% undesirable states with a maximum of three transitions from one state to another. The experiment was held four times in four different GPUs.

Experiment Results

Figure 17 visualizes (left to right and up to down) the results of Experiment 1.

1. As shown in Appendix C.1 sections 1 to 4, the Blue (Good AI) team won with 34.09% on average, while the Red (Bad AI) won with 65.152% on average, and the game ended with a

draw with 0.758% on average. The results' maximum difference between the four experiments was less than 10% for the Blue team, 8% for the Red team, and 1% for the Draw case which is accepted as the agents have not trained yet.

2. As shown in Appendix C.1 sections 5 to 7, the Blue (Good AI) team won with 3.349% on average, while the Red (Bad AI) won with 96.6506% on average, and the game ended with a draw with 0% on average. The results' maximum difference between the four experiments was less than 5% for the Blue team, 5% for the Red team, and 0% for the Draw case. The Experiment in Appendix C.1 section 8 has a huge difference from the Experiments in Appendix C.1 sections 5 to 7, which means that the agents can be trapped in a local optimal point.
3. As shown in Appendix C.1 sections 9, where trained Blue (Good AI) played against a random Red agent. The Blue (Good AI) team won with 43.073% compared to only 6.41% in the base experiment (blue columns), while the Red (Bad AI) won with 56.433%, and the game ended with a draw with 0.494%. This shows that the trained Blue agent performs better than when it performs against a random Red team.
4. As shown in Appendix C.1 sections 10, where trained Red (Bad AI) played against a random Blue agent. The Blue (Good AI) team won with 36.946%, while the Red (Bad AI) won with 58.703% compared to only 93.59% in the base experiment (blue columns), and the game ended with a draw with 4.351%. This shows that the trained Red agent performs worse than when it performs against a random Blue team.

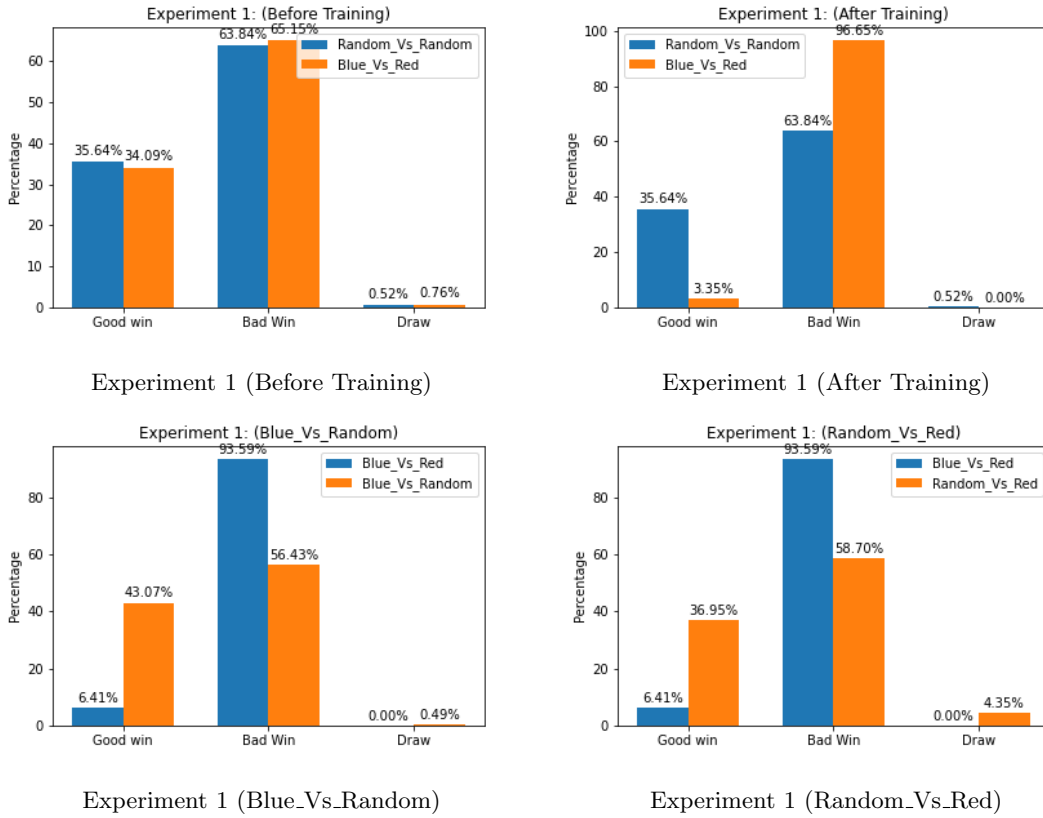


Figure 17: Results of Experiment 1

Experiment Conclusion

From this experiment, one can conclude that

- Experiment repetition leads to the same results in three experiments out of four (75%).

-
- In unsafe systems, the trained Blue team does better against the random Red team.
 - In unsafe systems, the trained Red team does better than the trained Blue team.
 - A Random team does the same as an untrained team.

5.2 Experiment 2: Episodes Increase

Experiment Question

What happens if training Episodes are reduced?

Experiment Expectations

The more the agents are trained, the more the game is biased toward one of them, as it is learning the strategy of the other agent.

Experiment Inputs

The DTMC shown in Figure 33 (Experiment 2) was used in this Experiment. It has 50% desirable states, and 50% undesirable states with a maximum of three transitions from one state to another. The experiment was held four times with different numbers of training episodes:

1. Agents were trained with a total number of episodes = 25000.
2. Agents were trained with a total number of episodes = 50000.
3. Agents were trained with a total number of episodes = 75000.
4. Agents were trained with a total number of episodes = 100000.

Experiment Results

Figure 18 visualizes (left to right and up to down) the results of Experiment 2.

1. As shown in Appendix C.2 section 2, the Blue (Good AI) team won with 45.444%, while the Red (Bad AI) won with 54.556%, and the game ended with a draw with 0%.
2. As shown in Appendix C.2 section 3, the Blue (Good AI) team won with 3.469%, while the Red (Bad AI) won with 96.531%, and the game ended with a draw with 0%.
3. As shown in Appendix C.2 section 4, the Blue (Good AI) team won with 0%, while the Red (Bad AI) won with 100%, and the game ended with a draw with 0%.
4. As shown in Appendix C.2 section 5, the Blue (Good AI) team won with 1.611 %, while the Red (Bad AI) won with 98.305%, and the game ended with a draw with 4.337 %.

Experiment Conclusion

As shown in the results section the Red Agent does better, when the number of training episodes increases. So it can be concluded that, if the system is biased towards one agent, the more the agents are trained, the better the won rate of this agent.

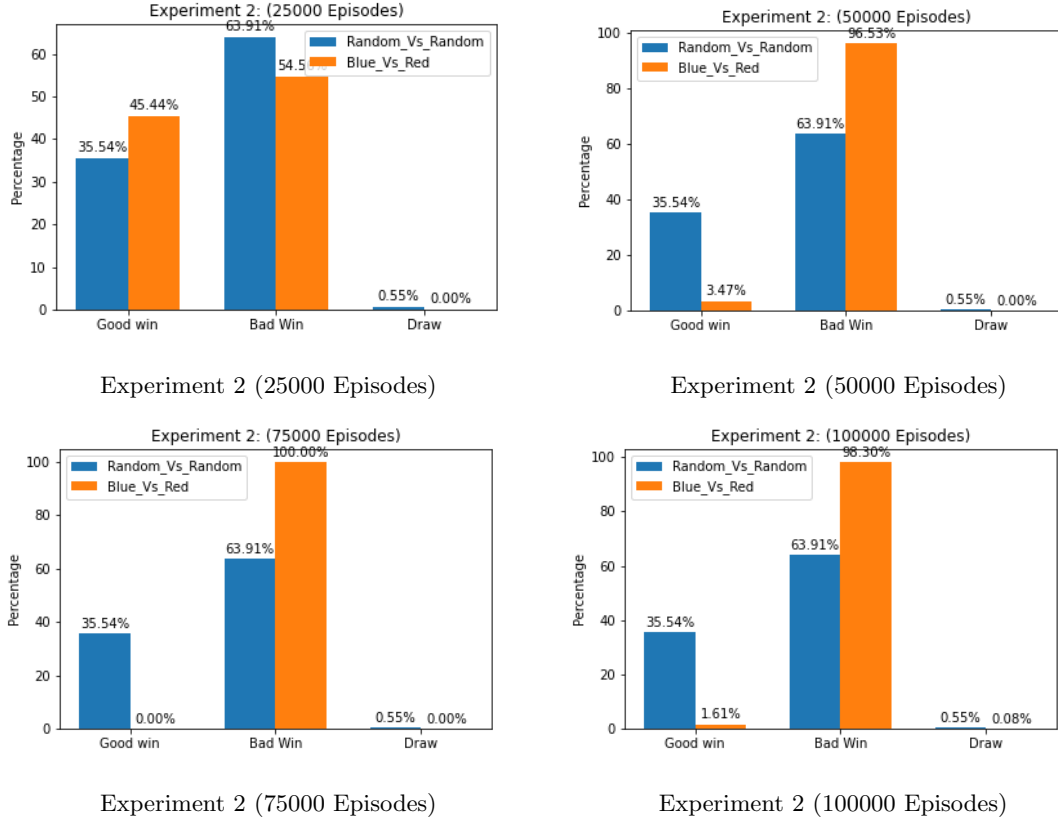


Figure 18: Results of Experiment 2

5.3 Experiment 3: State Desirability

Experiment Question

- What happens if the system has 50% desirable and 50% undesirable states?
- What happens if the system has 70% desirable and 30% undesirable states?
- What happens if the system has 30% desirable and 70% undesirable states?

Experiment Expectations

The win rate of each team will change according to the desirable state's ratio.

Experiment Inputs

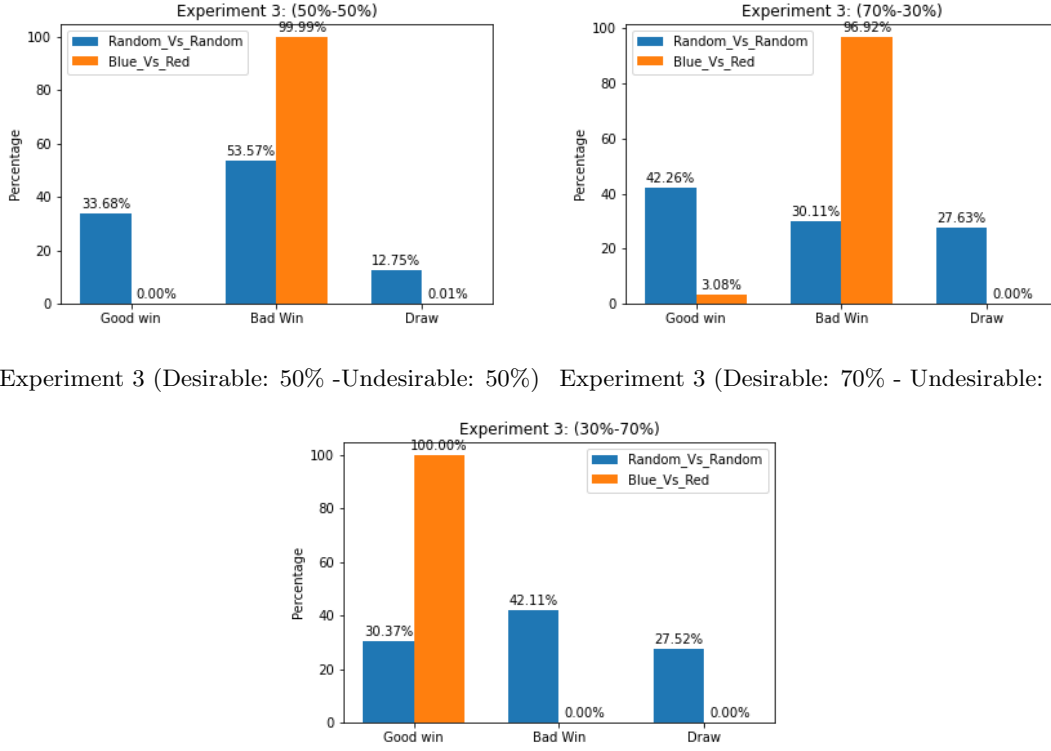
The DTMC shown in Figure 34 (Experiment 3) was used in this Experiment. This Experiment has been held three times:

1. DTMC has 50% desirable states, and 50% undesirable states .
2. DTMC has 70% desirable states, and 30% undesirable states .
3. DTMC has 30% desirable states and 70% undesirable states .

Experiment Results

Figure 19 visualizes (left to right and up to down) the results of Experiment 3.

- As shown in Appendix C.3 section 2, the Blue (Good AI) team won with 0.001%, while the Red (Bad AI) won with 99.986%, and the game ended with a draw with 0.013%. However, the system is 50% desirable and 50% undesirable states, the system was biased to the Red team as the transitions are mostly from desirable to undesirable states.
- As shown in Appendix C.3 section 4, the Blue (Good AI) team won with 3.077%, while the Red (Bad AI) won with 96.923%, and the game ended with a draw with 0.0%. The system was biased toward the Red team due to the same reason as the 50-50% case.
- As shown in Appendix C.3 section 6, the Blue (Good AI) team won with 99.996%, while the Red (Bad AI) won with 0.001%, and the game ended with a draw with 0.003%. The system was biased towards the blue team as the desirability sign of case 70-30% was flipped for all states so it is now 30-70%.



Experiment 3 (Desirable: 50% - Undesirable: 50%) Experiment 3 (Desirable: 70% - Undesirable: 30%)

Experiment 3 (Desirable: 30% - Undesirable: 70%)

Figure 19: Results of Experiment 3

Experiment Conclusion

The ratio of desirable states does not affect the system safety, however, the transition probabilities dominate. This means a system may include more safe states than an unsafe state, however the total safety of the system is low because the unsafe states are easy to reach.

5.4 Experiment 4: DTMC Solution

Experiment Question

What happens if the transition between states is selected based on the transition probabilities of the DTMC model?

Experiment Expectations

This experiment expects a difference in simulation results between the cases in which the actions are selected randomly with each action having a uniform selection probability and the case in which each action has a selection probability equal to the transition probability between the current state and the next state after the action is applied.

Experiment Inputs

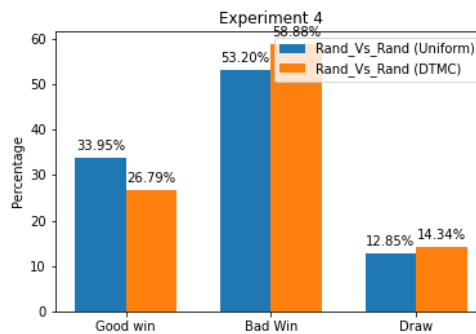
The DTMC shown in Figure 34 (Experiment 4) was used in this Experiment. It has 50% desirable states, and 50% undesirable states with a maximum of three transitions from one state to another. The Experiment was held twice:

1. Random_Vs_Random (Uniform Random).
2. Random_Vs_Random (DTMC Solution).

Experiment Results

Figure 20 visualizes (left to right and up to down) the results of Experiment 4.

- As shown in Appendix C.4 section 1, the Blue (Good AI) team won with 33.95%, while the Red (Bad AI) won with 53.20%, and the game ended with a draw with 14.337%.
- As shown in Appendix C.4 section 2, the Blue (Good AI) team won with 26.787%, while the Red (Bad AI) won with 58.876%, and the game ended with a draw with 12.849%.



Experiment 4

Figure 20: Results of Experiment 4

Experiment Conclusion

From the experiment's result, both cases are close to each other. So no clear conclusion can be provided for this experiment, however, it should be held again. For the next experiments, the DTMC solution model will be used as the experiment base.

5.5 Experiment 5: Number of Transitions

Experiment Question

- What happens if a model has many possible transitions for every state?
- What happens if a model has few possible transitions for every state?

Experiment Expectations

The number of transitions for every state affects the win rate of each team.

Experiment Inputs

The DTMCs shown in Figures 35, and 36 (Experiments 5.1, 5.2, and 5.3) were used in this Experiment. This Experiment was executed three times:

1. DTMC has a maximum of 6 transitions from one state to another.
2. DTMC has a maximum of 9 transitions from one state to another.
3. DTMC has a maximum of 12 transitions from one state to another.

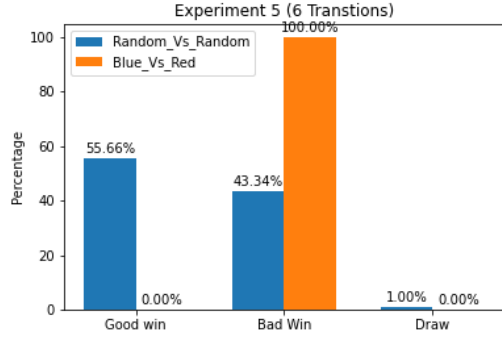
Experiment Results

Figure 21 visualizes (left to right and up to down) the results of Experiment 5.

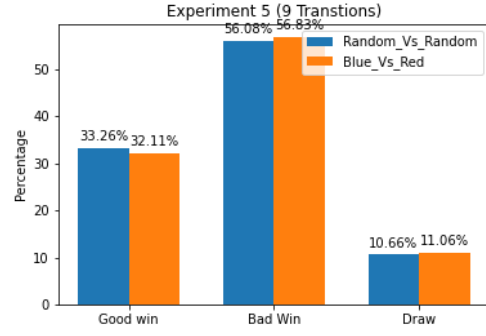
- As shown in Appendix C.5 section 2, the Blue (Good AI) team won with 0 %, while the Red (Bad AI) won with 100%, and the game ended with a draw with 0 %.
- As shown in Appendix C.5 section 4, the Blue (Good AI) team won with 32.11%, while the Red (Bad AI) won with 56.83%, and the game ended with a draw with 11.06%.
- As shown in Appendix C.5 section 6, the Blue (Good AI) team won with 24.9%, while the Red (Bad AI) won with 62.9%, and the game ended with a draw with 12.2%.

Experiment Conclusion

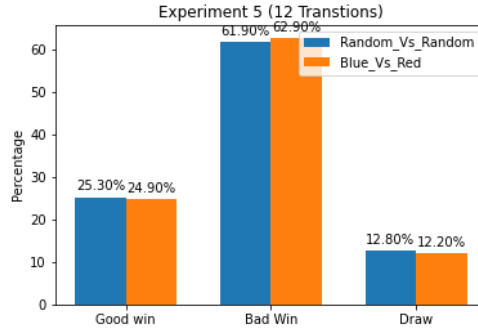
The win rates of both agents were radically reduced when the number of transitions was increased from 6 to 9, or 12 it became close to the win rates of the random agents. It can be concluded that the agents behave as random agents when transitions for every state are increased. Another conclusion could be made that, DTMC with a higher number of transitions for each, agents need more training episodes.



Experiment 5: (6 transitions)



Experiment 5: (9 transitions)



Experiment 5: (12 transitions)

Figure 21: Results of Experiment 5

5.6 Experiment 6: Initial Resources

Experiment Question

what happens if one agent has initial resources more than the other agent?

Experiment Expectations

The agent with more resources has more ability to win the game.

Experiment Inputs

The DTMC shown in Figure 36 (Experiment 6) was used in this Experiment. This Experiment was executed four times :

1. (Blue, Red) were given initial resources equal (250,500) respectively.
2. (Blue, Red) were given initial resources equal (250,750) respectively.
3. (Blue, Red) were given initial resources equal (500,250) respectively.
4. (Blue, Red) were given initial resources equal (750,250) respectively.

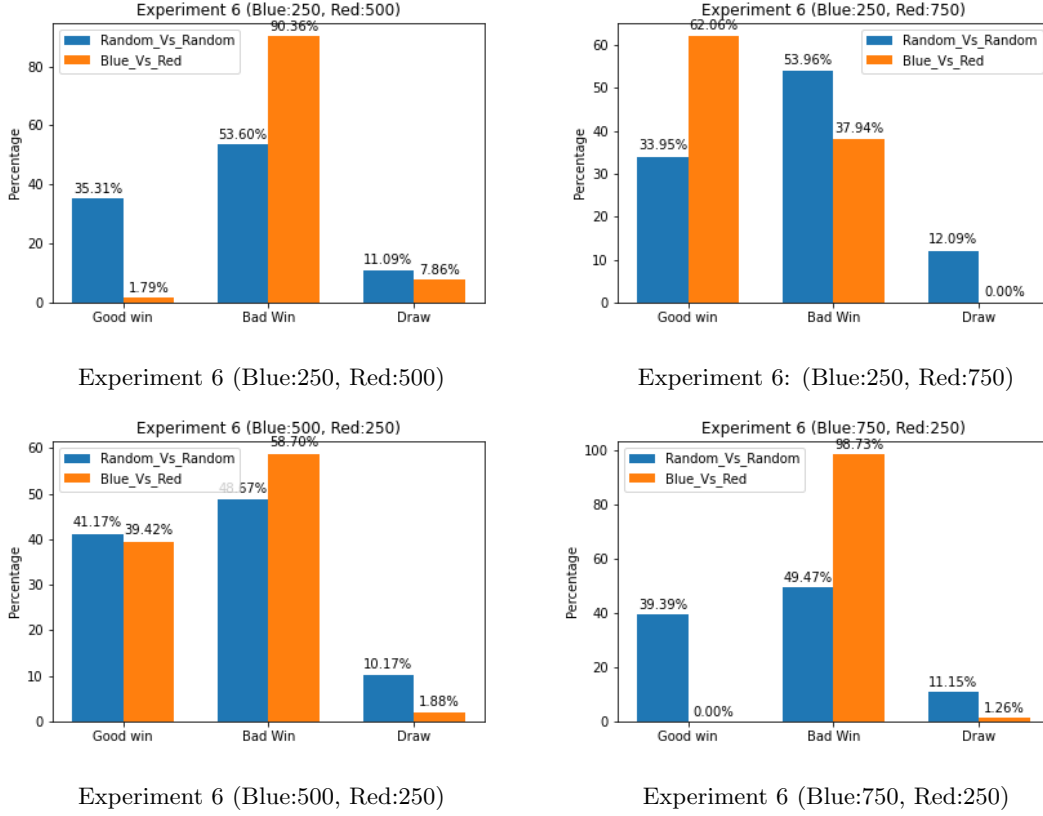


Figure 22: Results of Experiment 6

Experiment Results

Figure 22 visualizes (left to right and up to down) the results of Experiment 6.

- As shown in Appendix C.6 section 2, the Blue (Good AI) team won with 1.787 %, while the Red (Bad AI) won with 90.356%, and the game ended with a draw with 7.857 %.
- As shown in Appendix C.6 section 4, the Blue (Good AI) team won with 62.061%, while the Red (Bad AI) won with 37.939%, and the game ended with a draw with 0.0%.
- As shown in Appendix C.6 section 6, the Blue (Good AI) team won with 39.418%, while the Red (Bad AI) won with 58.697%, and the game ended with a draw with 1.885%.
- As shown in Appendix C.6 section 8, the Blue (Good AI) team won with 0%, while the Red (Bad AI) won with 98.735%, and the game ended with a draw with 1.265%.

Experiment Conclusion

From the results, it can be concluded that the difference in initial resources is a critical parameter in this experiment. When the difference is small, the agent with larger initial resources can do better, but when the difference is increased, the agent with larger resources does worse.

5.7 Experiment 7: Udesirableness threshold

Experiment Question

What happens if there is a threshold to the cumulative undesirableness of the system state sequence?

Experiment Expectations

The win rate of the Red team will decrease when the undesirability threshold is increased.

Experiment Inputs

The DTMC shown in Figure 37 (Experiment 7) was used in this Experiment. It has 50% desirable and 50% undesirable states with a maximum of three transitions from one state to another. The experiment was held three times:

1. The threshold value was 20.
2. The threshold value was 40.
3. The threshold value was 60.
4. The threshold value was ∞ .

Experiment Results

Figure 23 visualizes (left to right and up to down) the results of Experiment 7.

- As shown in Appendix C.7 section 2, the Blue (Good AI) team won with 25.48%, while the Red (Bad AI) won with 72.32%, and the game ended with a draw with 2.20%.
- As shown in Appendix C.7 section 4, the Blue (Good AI) team won with 36.33%, while the Red (Bad AI) won with 56.57%, and the game ended with a draw with 7.10%.
- As shown in Appendix C.7 section 6, the Blue (Good AI) team won with 40.36%, while the Red (Bad AI) won with 51.86%, and the game ended with a draw with 8.59%.
- As shown in Appendix C.7 section 8, the Blue (Good AI) team won with 27.08%, while the Red (Bad AI) won with 69.31%, and the game ended with a draw with 3.61%.

Experiment Conclusion

The rate of the Red team decreased when the threshold of undesirability was increased from 20 to 40 and then to 60, which means that the less the undesirability threshold the more the Red team was able to win the game. However, in the case when there was no undesirability threshold, the win Rate of the Red team increased again and became similar to the case when (threshold = 20). This means that the inversely proportional relation between the threshold and the win rate of the Red team is limited in a range of threshold values for example $[0,60]$, and outside this range, it could be directly proportional again, and this is what happened in this experiment.

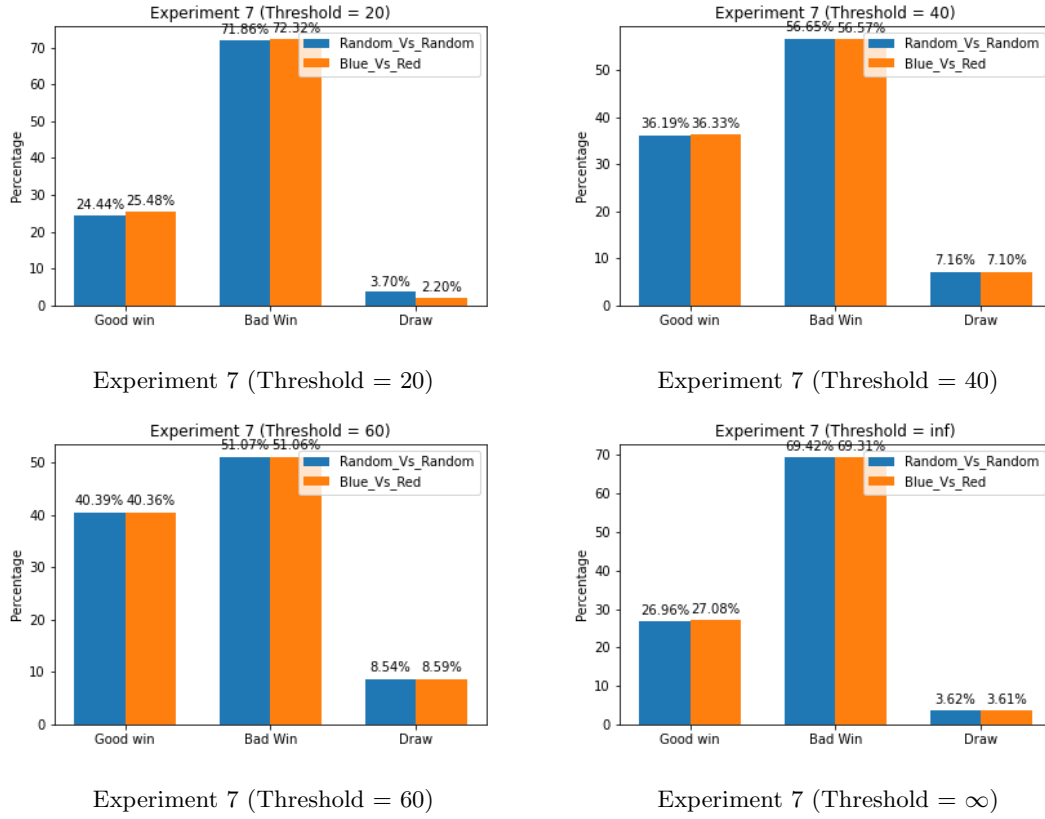


Figure 23: Results of Experiment 7

5.8 Experiment 8: Changing Order of Moves

Experiment Question

What happens if the order of moves between two agents is changed?

Experiment Expectations

In DTMCs used for conducting these experiments, transitions from state s_0 to s_1 have a probability of 1.0 as shown in figure 16, which means that the result of the initial move does not depend on the team strategy, however, the result of the second move depends on the team. This means the team that second move should have more advantage to win the game.

Experiment Input

The DTMC shown in Figure 37 (Experiment 8) was used in this Experiment. It has 50% desirable and 50% undesirable states with a maximum of three transitions from one state to another. The experiment was held twice:

1. The Red team has an initial move.
2. The Blue team has an initial move.

Experiment Results

Figure 24 visualizes (left to right and up to down) the results of Experiment 8.

- As shown in Appendix C.8 section 2, the Blue (Good AI) team won with 34.928%, while the Red (Bad AI) won with 59.59%, and the game ended with a draw with 5.482%.
- As shown in Appendix C.8 section 4, the Blue (Good AI) team won with 2.201%, while the Red (Bad AI) won with 97.797%, and the game ended with a draw with 0.002%.

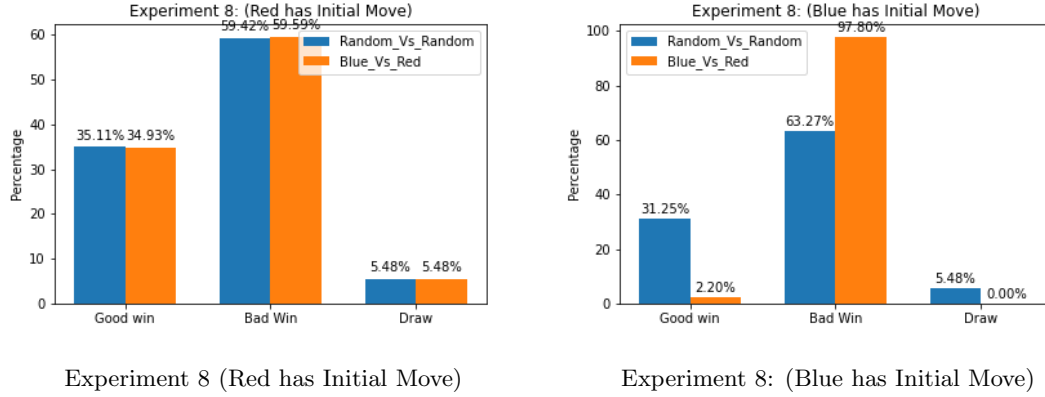


Figure 24: Results of Experiment 8

Experiment Conclusion

As was expected, as shown in figure 24 Experiment 8: (Blue has Initial Move), the Red team had more advantage to win the game, so the system was biased toward it. However, in figure 24 Experiment 8: (Red has Initial Move), the Blue team could maintain its chances to win the game close to the base Experiment.

5.9 Experiment 9: Transitions Add/Remove

Experiment Question

What happens if some transitions between states are added or removed?

Experiment Expectations

Adding or removing transitions between system states would change the agent's strategy.

Experiment Input

The DTMC shown in Figure 38 (Experiment 9) was used in this Experiment. It has 50% desirable and 50% undesirable states with a maximum of three transitions from one state to another. The experiment was held three times:

1. It was held with an initial system.
2. It has two new transitions that were added to the initial system.
3. It has two transitions that were removed from the system.

Experiment Results

Figure 25 visualizes (left to right and up to down) the results of Experiment 9.

- As shown in Appendix C.9 section 2, the Blue (Good AI) team won with 40.706%, while the Red (Bad AI) won with 55.224%, and the game ended with a draw with 4.07%.
- As shown in Appendix C.9 section 4, the Blue (Good AI) team won with 35.614%, while the Red (Bad AI) won with 59.671%, and the game ended with a draw with 4.715%.
- As shown in Appendix C.9 section 6, the Blue (Good AI) team won with 40.622%, while the Red (Bad AI) won with 55.375%, and the game ended with a draw with 4.003%.

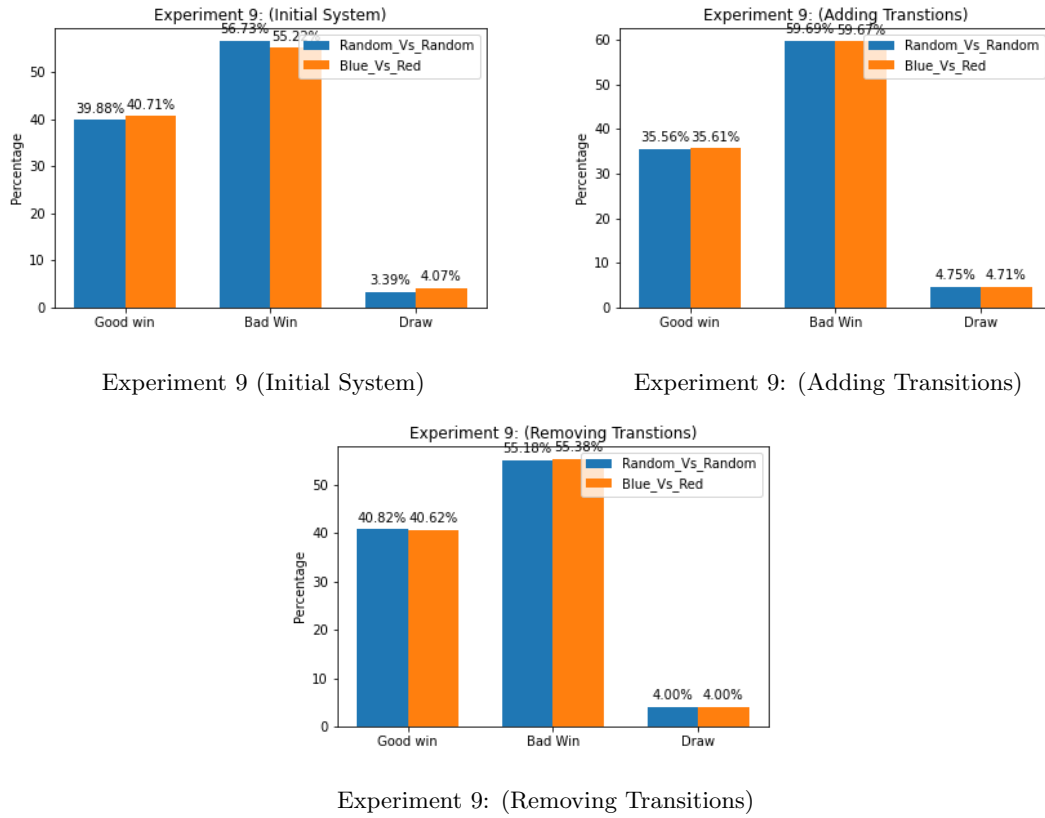


Figure 25: Results of Experiment 9

Experiment Conclusion

The results of the three experiments are similar, meaning that adding or removing transitions does not affect the win rate of each team.

5.10 Experiment 10: Changing Order of Moves without Retrain

Experiment Question

What happens if the order of moves between two agents is changed without agents being trained again?

Experiment Expectations

Changing the order of moves without retraining might affect the win rate of each team.

Experiment Input

This experiment was held based on Experiment 8. The same DTMC model and agents were used again, the Only change was in the order of moves which was opposite to the order of moves of Experiment 8.

Experiment Results

Figure 26 visualizes (left to right and up to down) the results of Experiment 10.

- As shown in Appendix C.10 section 2, the Blue (Good AI) team won with 35.424%, while the Red (Bad AI) won with 59.091%, and the game ended with a draw with 5.485%.
- As shown in Appendix C.10 section 4, the Blue (Good AI) team won with 2.895%, while the Red (Bad AI) won with 97.103%, and the game ended with a draw with 0.002%.

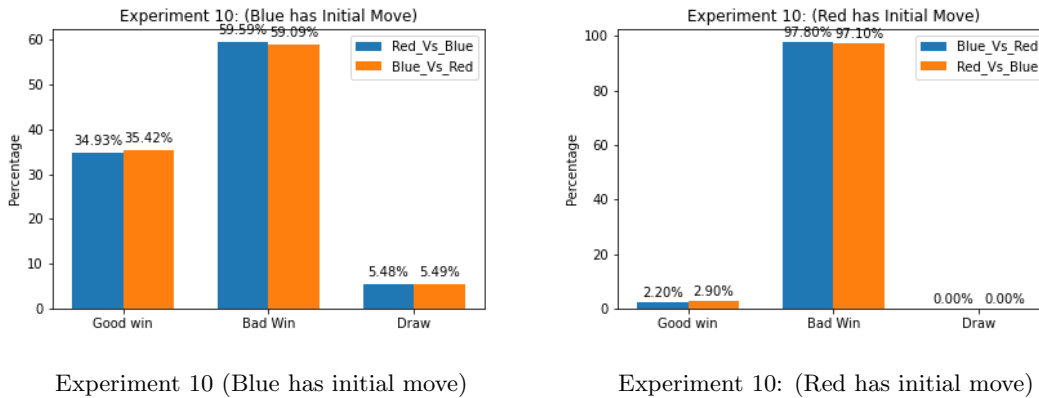


Figure 26: Results of Experiment 10

Experiment Conclusion

The results of the two experiments are similar and each agent's win rate is the same regardless of his order of move, which means that changing the order of moves without retraining does not affect the win rate of each team.

6 Thesis Specialization

This chapter will use the results of the previous chapters to build a path elimination algorithm by optimizing the agent's initial resources.

6.1 Path Elimination Algorithm

As introduced in the game description in chapter 3.2, each agent has an initial value of resources, so it selects actions only if its resources are enough to apply to the system. In this chapter, a path elimination algorithm is to be built by optimizing the initial value of resources that should be given to each agent, so that the selected actions of the agents are restricted by their resources and improbable system paths that rarely happen can be prevented. For Example, as shown in figure 27, if path $\{s_0, s_1, s_2, s_5\}$ has a probability of happening below a certain threshold for example 5%, the algorithm should optimize the initial resources of the agents so they can not afford the cost of the actions leading to move the system from s_1 to s_2 and from s_2 to s_5 .

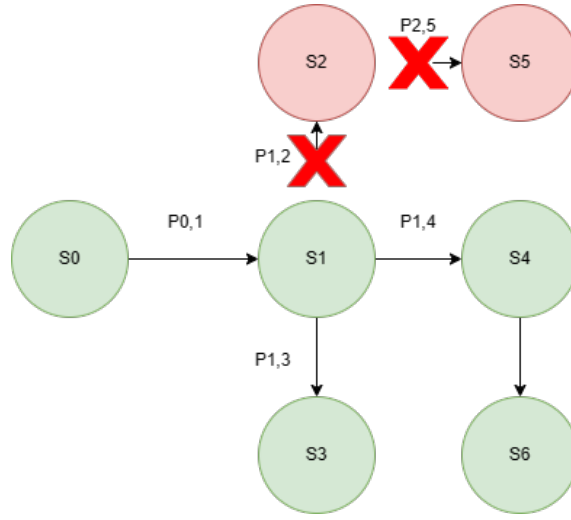


Figure 27: Path Elimination Algorithm

The implemented algorithm works as follows:

1. **Extracts** all possible DTMC paths from s_0 to s_n state where n is the number of states ($n=14$), according to the DTMC general structure introduced in chapter 5.
2. **Calculates** the probability of each path by multiplying all probabilities of transitions along the path.
3. **Simulates** the system with two random agents as (DTMC solution) introduced in Experiment 4 chapter 5.4.
4. **Excludes** all paths with a probability less than a certain threshold.
5. **Trains** the environment with one Red and one Blue agent.
6. **Runs** the optimization algorithm introduced in section 6.2 to obtain initial resources for each agent.
7. **Simulates** the system with the trained agents having optimized values of resources.
8. **Compares** the results of the experiment in steps 3 and 7.

6.2 Optimization Problem

The following mathematical equation represents the resources' optimization problem:

$$\begin{aligned} \max_x \quad & f(x) = \sum_{i=1}^{n=2} x_i \\ \text{subject to:} \quad & g_1(x) = \sum_{i=1}^{n=2} x_i \leq l1 \\ & g_2(x) = x_2 \leq l2 \end{aligned} \tag{25}$$

Where x_1 , and x_2 are initial resources for the Blue and Red agents respectively, $l1$ is the minimum cumulative cost between all excluded paths, $l2$ is the minimum cumulative cost between paths that contribute more to the win rate of the Red agent for the base experiment in step 3. The following section will include the results of the implemented path elimination algorithm for different DTMC models in different Experiments.

6.3 Experiments and Results

In this section, the results of four experiments will be introduced as a proof of concept for the implemented resource optimization algorithm.

6.3.1 Experiment 1

The DTMC shown in figure 39 (Experiment 1) was used for this experiment. The system has 16 states, with 50% states with -ve undesirability, and 50% with +ve undesirability value. The system has a maximum number of transitions from one state equals 3. The Model has 7 different paths from start to end states.

Experiment Results

Figure 28 visualizes (left to right and up to down) the results of Experiment 1.

- As shown in Appendix C.11 section 2, the Blue (Good AI) team won with 23.31%, while the Red (Bad AI) won with 76.01%, and the game ended with a draw with 0.68%.
- As shown in Appendix C.11 section 3, the Blue (Good AI) team won with 39.75%, while the Red (Bad AI) won with 53.49%, and the game ended with a draw with 6.77%.
- As shown in figure 28, paths 2, 3, 4, 5 have probability 3%, 1%, 0%, 0% without optimization.
- As shown in figure 28, paths 2, 3, 4, 5 all have probability 0% with optimization.



Figure 28: Path Elimination Algorithm: Results of Experiment 1

6.3.2 Experiment 2

The DTMC shown in figure 39 (Experiment 2) was used for this experiment. The system has 16 states, with 50% states with -ve undesirability, and 50% with +ve undesirability value. The system has a maximum number of transitions from one state equals 3. The Model has 133 different paths from start to end states.

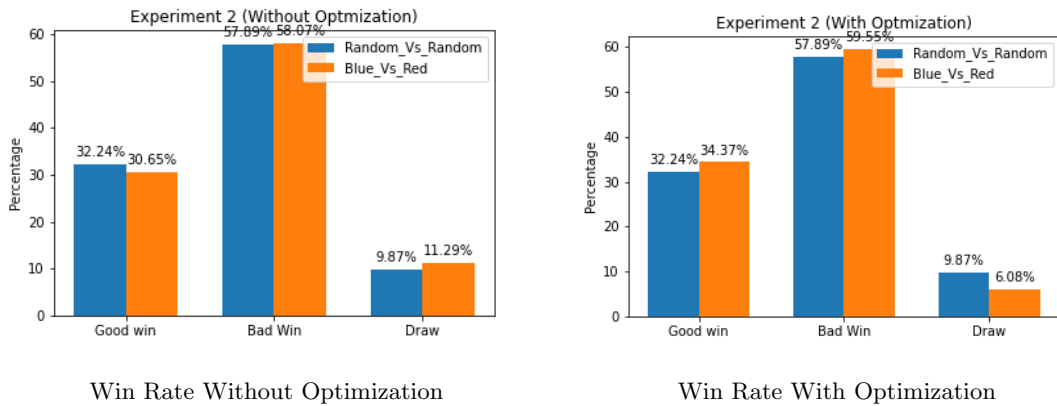


Figure 29: Path Elimination Algorithm: Results of Experiment 2

Experiment Results

Figure 29 visualizes (left to right and up to down) the results of Experiment 2.

- As shown in Appendix C.12 section 2, the Blue (Good AI) team won with 30.646 %, while the Red (Bad AI) won with 58.069 %, and the game ended with a draw with 11.285 %.
- As shown in Appendix C.12 section 3, the Blue (Good AI) team won with 34.37%, while the Red (Bad AI) won with 59.552%, and the game ended with a draw with 6.078%.
- As shown in Appendix C.12 section 2, paths $\{0,1,2,...,129\}$ have probability $\geq 0\%$ without optimization.
- As shown in Appendix C.12 section 3, paths $\{0,1,2,...,129\}$ all have probability 0% with optimization.

6.3.3 Experiment 3

The DTMC shown in figure 40 (Experiment 3) was used for this experiment. The system has 16 states, with 50% states with -ve undesirability, and 50% with +ve undesirability value. The system has a maximum number of transitions from one state equals 3. The Model has 18 different paths from start to end states.

Experiment Results

Figure 30 visualizes (left to right and up to down) the results of Experiment 3.

- As shown in Appendix C.13 section 2, the Blue (Good AI) team won with 16.82%, while the Red (Bad AI) won with 70.28%, and the game ended with a draw with 12.40%.
- As shown in Appendix C.13 section 3, the Blue (Good AI) team won with 41.777%, while the Red (Bad AI) won with 24.419%, and the game ended with a draw with 33.804%.
- As shown in Appendix C.13 section 2, paths $\{9, 13\}$ have probability $\geq 0\%$ without optimization.
- As shown in Appendix C.13 section 3, paths $\{9, 13\}$ all have probability 0% with optimization.

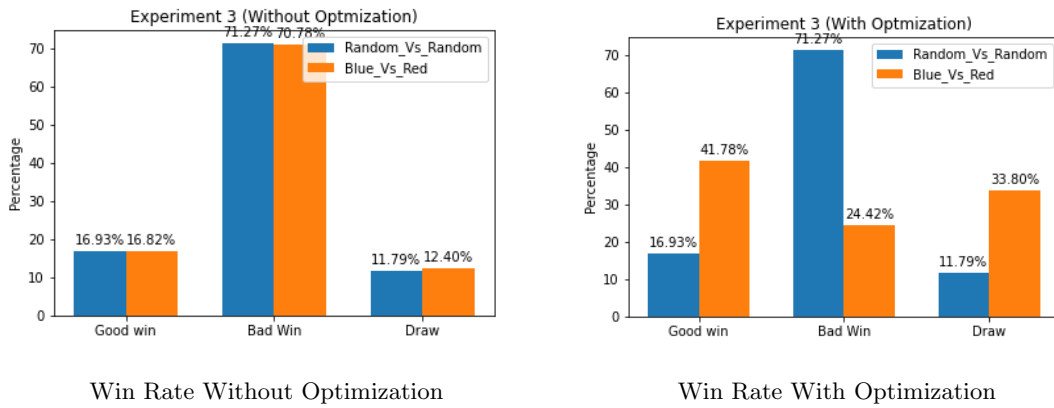


Figure 30: Path Elimination Algorithm: Results of Experiment 3

6.3.4 Experiment 4

The DTMC shown in figure 40 (Experiment 4) was used for this experiment. The system has 16 states, with 50% states with -ve undesirability, and 50% with +ve undesirability value. The system has a maximum number of transitions from one state equals 3. The Model has 35 different paths from start to end states.

Experiment Results

Figure 31 visualizes (left to right and up to down) the results of Experiment 4.

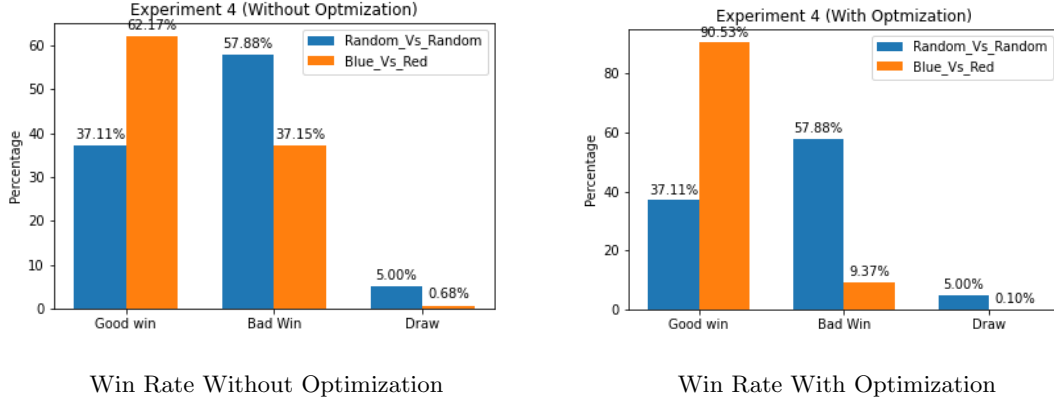


Figure 31: Path Elimination Algorithm: Results of Experiment 4

- As shown in Appendix C.14 section 2, the Blue (Good AI) team won with 62.173 %, while the Red (Bad AI) won with 37.145 %, and the game ended with a draw with 0.682 %.
- As shown in Appendix C.14 section 3, the Blue (Good AI) team won with 90.532%, while the Red (Bad AI) won with 9.367%, and the game ended with a draw with 0.101%.
- As shown in Appendix C.14 section 2, paths $\{0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 13, 15, \dots, 20, 21, 22, \dots, 25, 28, 30, 31, 33\}$ have probability $\geq 0\%$ without optimization.
- As shown in Appendix C.14 section 3, paths $\{0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 13, 15, \dots, 20, 21, 22, \dots, 25, 28, 30, 31, 33\}$ all have probability 0% with optimization.

6.4 Experiments Conclusion

From the results of the four previous experiments, and by comparing the results of an experiment held without resource optimization and experiments that were applied with resource optimization, one can conclude that the implemented algorithm for path elimination succeeded in restricting the RL agents from taking the most improbable paths. At the same time, the win rate of the Blue team was increased as shown in Appendix C.4 or remained closed to the base experiment as in experiments 1, 2, and 3.

6.5 Other Optimization Techniques

The following techniques can be used to optimize the behavior of the Red and Blue RL agents, with initial resources, various algorithms and techniques can be used to restrict and guide their actions. The following sections will introduce some of them followed by a paper from literature.

6.5.1 Reinforcement Learning with Constraints

This general strategy enables the addition of limitations to RL problems. The agents will optimize their rules while observing any resource use limits that you provide.

Cautious Reinforcement Learning with Logical Constraints

This paper introduces the idea of an adaptive safe padding that encourages Reinforcement Learning (RL) to create the best control strategies while maintaining learners' safety. To achieve a goal with the greatest probability expressed as a temporal logic formula policies are combined. Although it demonstrates that the suggested architecture is capable of automatically handling the trade-off between efficient progress in exploration (towards goal satisfaction) and maintaining safety, constraining the RL agent to remain safe throughout learning may limit the exploration. On the synthesized policies' optimality and the learning algorithm's convergence, theoretical assurances are available. The effectiveness of the suggested method is demonstrated by the experimental findings. (Hasanbeig, Abate and Kroening, 2020)

6.5.2 Soft Actor-Critic with Soft Constraints

SAC is an off-policy RL algorithm. SAC can be modified to include soft constraints on resource usage, making the agents penalized for exceeding resource limits.

Worst-Case Soft Actor-Critic for Safety-Constrained Reinforcement Learning

RL research focuses on safe exploration with distinct rewards and security. A new RL algorithm, Worst-Case, is proposed to achieve risk control. The algorithm uses the Soft Actor-Critic algorithm with a safety critic, assessing constraint satisfaction and modifying policies based on worst-case performance. Empirical analysis shows the algorithm achieves better risk control than expectation-based approaches. (Yang et al., 2021)

6.5.3 Proximal Policy Optimization with Penalty Terms

It extends PPO to include penalty terms in the reward function for violating resource constraints. This encourages agents to act within resource limits.

Penalized Proximal Policy Optimization for Safe Reinforcement Learning

RL aims to learn the best course of action while satisfying safety requirements. Current algorithms struggle with strict constraints and updating policies effectively. The Penalized Proximal Policy Optimization (P3O) proposes a method that minimizes the cost of a single unconstrained problem, removes cost constraints, and outperforms state-of-the-art algorithms in reward improvement and constraint satisfaction. (Zhang et al., 2022)

6.5.4 Constrained Optimization with Lagrange Multipliers

It formulates the RL problem as a constrained optimization problem with Lagrange multipliers. This allows for balanced resource optimization and constraint satisfaction.

Constrained Optimization Using Lagrange Multipliers

In optimal design problems, it is necessary to find values for a set of n design variables, (x_1, x_2, x_n) , that minimize a scalar-valued objective function of the design variables while satisfying a set of m inequality constraints. Typically, constrained optimization issues are expressed as:

$$\begin{aligned}
& \text{minimize} && J = f(x_1, x_2, \dots, x_n) \\
& \text{subject to} && g_1(x_1, x_2, \dots, x_n) \leq 0 \\
& && g_2(x_1, x_2, \dots, x_n) \leq 0 \\
& && \dots \\
& && g_m(x_1, x_2, \dots, x_n) \leq 0
\end{aligned} \tag{26}$$

The optimization problem has a singular solution if the objective function is quadratic in the design variables and the constraint equations are linearly independent. Take a look at the easiest constrained minimization problem

$$\min_{x \geq b} \frac{1}{2} kx^2 \text{ where } k \geq 0 \text{ such that } x \geq b \tag{27}$$

The problem involves a quadratic objective function with a single constraint inequality. The optimum solution is determined by the constraint equation, which is linear in x . If $g \geq 0$, the constraint constrains the optimum, while if $g \leq 0$, it doesn't. Lagrange multiplier methods are applied to modify the objective function by adding constraint terms.

$$J_A(x_1, x_2, \dots, x_n, \lambda_1, \lambda_2, \dots, \lambda_m) = f(x_1, x_2, \dots, x_n) + \sum_{j=1}^m \lambda_j g_j(x_1, x_2, \dots, x_n) \tag{28}$$

For the problem of equation 27, $n = 1$ and $m = 1$, so

$$J_A(x, \lambda) = \frac{1}{2} kx^2 + \lambda(b - x) = \frac{1}{2} kx^2 - \lambda x + \lambda b. \tag{29}$$

The objective function is modified (enhanced) by the Lagrange multiplier, λ , from one quadratic ($\frac{1}{2} kx^2$) to another ($\frac{1}{2} kx^2 - \lambda x + \lambda b$), where the minimum of the modified quadratic satisfies the constraint ($x \geq b$). (Gavin and Scruggs, 2012)

6.5.5 Trust Region Policy Optimization with KL Divergence Constraints

It modifies TRPO to include constraints on the KL divergence between policies, ensuring that the agents' policies do not deviate too far from the initial resources.

Trust Region Policy Optimization

This paper outlines a technique for control policy optimization with a guarantee of monotonic improvement. A useful algorithm called Trust Region Policy Optimization (TRPO) was created by approximating the theoretically justified scheme several times. This algorithm works well for maximizing complex nonlinear structures like neural networks. The tests show that it performs well on a variety of tasks, including playing Atari games with screen images as input and learning simulated robotic swimming, hopping, and walking gaits. Despite its approximations departing from the theory, TRPO typically results in monotonic improvement with minimal hyperparameter tuning. (Schulman, Levine et al., 2015)

6.5.6 Action Space Constraints

It limits the RL agents' action space to a subset of actions that are resource-efficient. This approach restricts the agents to making choices within predefined resource boundaries.

6.5.7 Resource-Adaptive State Space

It modifies the state representation to include information about available resources. This enables RL agents to adapt their policies based on resource availability.

6.5.8 Resource Allocation Networks

It implements a separate network or algorithm to manage resource allocation. The RL agents interact with this network to request and manage resources within predefined limits.

6.5.9 Dynamic Resource Allocation Strategies

It develops dynamic strategies for allocating resources based on RL agent performance and constraints. This can involve resource allocation algorithms like the multi-armed bandit or queuing theory.

A Survey and Comparative Study of Hard and Soft Real-Time Dynamic Resource Allocation Strategies for Multi-/Many-Core Systems

Multi-Many-core systems are designed to meet increasing performance needs in various industries, including embedded and high-performance computing. These systems require dynamic resource allocation strategies to meet real-time constraints. This article surveys various dynamic resource allocation strategies since the mid-1990s, comparing hard and soft real-time optimization goals. It also discusses trends, research problems, and potential future directions. (Singh et al., 2017)

6.5.10 Hybrid Control and Reinforcement Learning

It combines classical control methods with RL to ensure that resource allocation and agent behavior align with predefined constraints.

6.5.11 Reward Shaping with Resource Costs

It uses reward-shaping techniques to encourage resource-efficient behavior. Design the reward function to include costs associated with resource consumption.

Reward Shaping-Based Actor-Critic Deep Reinforcement Learning for Residential Energy Management

The article presents a reward shaping (RS)-based actor-critic deep RL (ACDRL) algorithm for managing residential energy consumption. The algorithm uses a Markov decision process to represent the interaction between energy management centers and residential loads. It learns optimal energy consumption schedules using an RS mechanism. The algorithm outperforms state-of-the-art RL methods in learning speed, solution optimality, and cost reduction, addressing the challenges of managing uncertain factors in residential energy consumption. (Lu et al., 2023)

6.5.12 Multi-Agent Reinforcement Learning with Communication

If the two agents can communicate, MARL can be used to coordinate their actions and jointly optimize resource usage while respecting constraints.

Multi-Agent Reinforcement Learning for Network Selection and Resource Allocation in Heterogeneous Multi-RAT Networks

The rapid growth of mobile devices and wireless applications necessitates innovative network selection techniques to improve the Quality of Service. A distributed framework for dynamic network selection and resource allocation is proposed, using a deep MARL algorithm to maximize edge node experience, extend battery life, and leverage adaptive compression schemes. The solution outperforms current techniques in terms of energy consumption, latency, and cost. (Allahham et al., 2022)

6.5.13 Model Predictive Control

MPC is used to generate optimal control sequences considering resource constraints. The RL agents can follow the recommended control sequences.

Review on model predictive control: An engineering perspective

Model-based predictive control (MPC) is an advanced control method that uses a process model to predict future system behavior. It solves optimization problems implicitly, shifting the focus from designing a controller to modeling the to-be-controlled process. MPC can even control systems that conventional feedback controllers cannot control. This review article provides a concise summary of MPC's current state, including theory, historic evolution, and practical considerations. It also discusses implantation details and strategies to cope with computational burdens. (Schwenzer et al., 2021)

6.5.14 Hierarchical Reinforcement Learning

It employs hierarchical RL to separate high-level resource allocation decisions from low-level task-specific actions, ensuring that resource constraints are met at all levels.

Hierarchical Reinforcement Learning: A Comprehensive Survey

Hierarchical RL (HRL) is a method that simplifies complex decision-making tasks into smaller ones. With the growing research landscape, a comprehensive overview is needed. This survey explores various HRL approaches, including learning hierarchical policies, subtask discovery, transfer learning, and multi-agent learning. It proposes open problems for future research and outlines suitable task domains. (Pateria et al., 2021)

6.5.15 Supervised Learning for Resource Constraint Enforcement

It trains a supervised model to predict when resource constraints are about to be violated and intervene to adjust agent behavior accordingly.

Hardware-assisted machine learning in resource-constrained IoT environments for security: review and future prospective

The Internet of Things (IoT) presents challenges in ensuring trusted computing, user privacy, and protection against attacks. Machine learning (ML)-based intrusion detection has gained traction due to its ability to handle encrypted threats. This work explores ML and deep learning methodologies for IoT device security, examining their benefits, drawbacks, and potential. It also addresses trade-offs in integrating accelerators and customizing embedded device architectures for effective ML-based methods. (Kornaros, 2022)

7 Conclusion and Future Work

As a summary of this thesis work, chapter 2 introduced some fundamentals about MBSE, DTMC, Adv-MARL, and game theory. The thesis designed a concept for converting a (DTMC) into a game where two agents compete to control the system states and influence its safety characteristics.

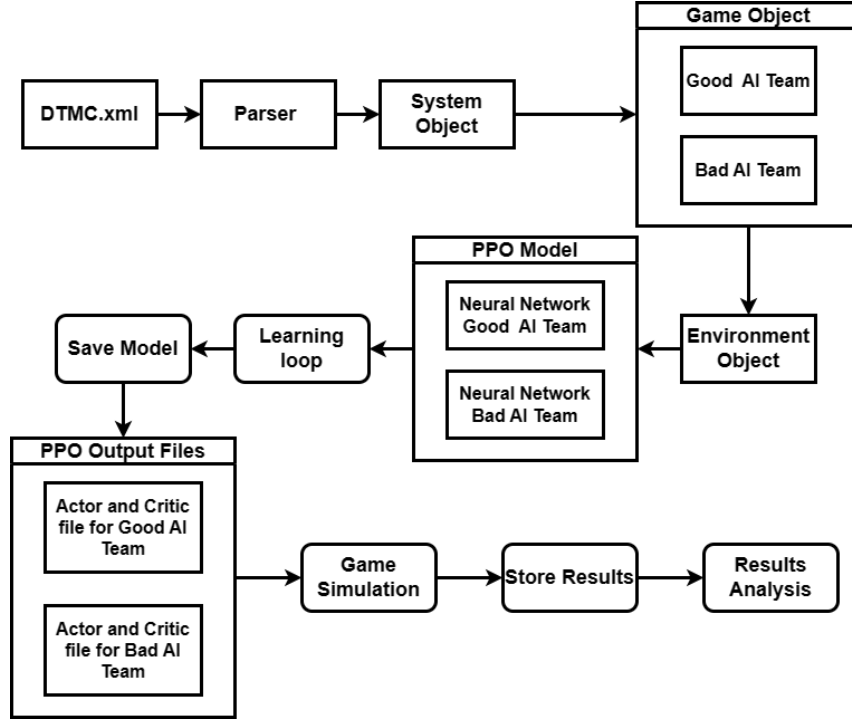


Figure 32: System Software Architecture

As shown in figure 32, and introduced in chapters 3, and 4 the DTMC file was parsed into a system object that abstracts all the system attributes, such as the states, the edges, the transition matrix, and system dynamics.

The system object is then passed as an attribute of a game object. The game object uses the attributes and the dynamic functions of the system object, in addition to having game attributes, such as agents, strategies, scores, and the dynamic functions of the game.

The game object is passed as an attribute to the environment object. The environment object uses the attributes of the game object in addition to having environment attributes such as observation space, action space, reset, step, and reward functions.

The environment object is passed as an attribute to a PPO model that uses the concept of neural networks to train the agents. The output files of the training process are then used to simulate the game and analyze the results.

As introduced in chapter 5, ten different experiments were conducted to verify the concept designed and the prototype developed, each experiment has its expectation, input model, results, and conclusion.

As introduced in chapter 6, a path elimination algorithm was implemented by calculating the optimized resources that should be provided to the agents to restrict the action selection of the agents and exclude the most improbable systems' paths. The chapter introduced four different experiments to verify the algorithm. The chapter also introduced some other optimization techniques that could be investigated and implemented in future work.

For future work, the following points are recommended:

- The current implementation only considers one DTMC as a system component. Future work may consider multiple DTMCs by combining their states into one DTMC, so after parsing the system has one system object.
- The current implementation implemented only PPO to train agents due to the thesis time limit. However, future work may try other RL algorithms, such as SAC, and DDPG which were introduced in the basis chapter 2.4.5.
- The current implementation considers only one DTMC model for each experiment due to the thesis time limit. However, future work may consider at least five DTMC models to have a better conclusion for each experiment.
- The future work may investigate different techniques for path elimination algorithm as introduced in section Other Optimization Techniques 6.5.
- The current implementation only considers the design of the game due to the thesis limit time. However, the future may consider more concepts of game theory such as Nash equilibrium and best response which were introduced in the basis chapter 2.4.3.
- Future work may consider designing a more realistic game that has a real abstraction of the system and the model.

Bibliography

- Acaroglu, Leyla (2017). *ITools for Systems Thinkers: The 6 Fundamental Concepts of Systems Thinking*. URL: <https://medium.com/disruptive-design/tools-for-systems-thinkers-the-6-fundamental-concepts-of-systems-thinking-379cdac3dc6a>. (accessed: 05.06.2023).
- Allahham, Mhd Saria et al. (2022). ‘Multi-agent reinforcement learning for network selection and resource allocation in heterogeneous multi-RAT networks’. In: *IEEE Transactions on Cognitive Communications and Networking* 8.2, pp. 1287–1300.
- Askari, Gholamreza, Madjid Eshaghi Gordji and Choonkil Park (2019). ‘The behavioral model and game theory’. In: *Palgrave Communications* 5.1.
- Bernstein, Daniel S et al. (2002). ‘The complexity of decentralized control of Markov decision processes’. In: *Mathematics of operations research* 27.4, pp. 819–840.
- Boutilier, Craig (1996). ‘Planning, learning and coordination in multiagent decision processes’. In: *TARK*. Vol. 96. Citeseer, pp. 195–210.
- Desmar (2022). *Model-Based System Engineering: Pros and Cons*. URL: <https://www.eclipsesuite.com/model-based-system-engineering-pros-and-cons/>. (accessed: 10.06.2023).
- Gavin, Henri P and Jeffrey T Scruggs (2012). ‘Constrained optimization using lagrange multipliers’. In: *CEE 201L. Duke University*.
- Grinstead, Charles and Laurie J Snell (2006). *Introduction to probability*.
- Han, Songyang et al. (2022). ‘What is the solution for state adversarial multi-agent reinforcement learning?’ In: *arXiv preprint arXiv:2212.02705*.
- Hart, Laura E. (2015). *Introduction To Model-Based System Engineering (MBSE) and SysML*. URL: <https://www.incose.org/docs/default-source/delaware-valley/mbse-overview-incose-30-july-2015.pdf>. (accessed: 05.06.2023).
- Hasanbeig, Mohammadhosein, Alessandro Abate and Daniel Kroening (2020). ‘Cautious reinforcement learning with logical constraints’. In: *arXiv preprint arXiv:2002.12156*.
- Hotz, Heiko (2006). *A Short Introduction to Game Theory*. URL: https://www.theorie.physik.uni-muenchen.de/lfsfrey/teaching/archiv/sose_06/softmatter/talks/Heiko.Hotz-Spieltheorie-Handout.pdf. (accessed: 16.06.2023).
- INCSE (2023). *Systems Engineering Definition*. URL: https://www.incose.org/about-systems-engineering/system-and-se-definition/systems-engineering-definition#ENGINEERED_SYSTEM. (accessed: 05.06.2023).
- Kornaros, Georgios (2022). ‘Hardware-assisted machine learning in resource-constrained IoT environments for security: review and future prospective’. In: *IEEE Access* 10, pp. 58603–58622.
- Long, David and Zane Scott (2012). *A primer for model-based systems engineering*. Lulu. com.
- Lourentzou, Ismini (2016). ‘Markov Games and Reinforcement Learning’. In.
- Lu, Renzhi et al. (2023). ‘Reward Shaping-Based Actor–Critic Deep Reinforcement Learning for Residential Energy Management’. In: *IEEE Transactions on Industrial Informatics* 19.3, pp. 2662–2673. DOI: 10.1109/TII.2022.3183802.
- Morozov, Andrey (2022). *Modeling Analysis for Automation Systems: Probabilistic Models*.
- OpenAI (2018). *Part 1: Key Concepts in RL*; *Spinning up Documentation*. URL: https://spinningup.openai.com/en/latest/spinningup/rl_intro.html. (accessed: 10.06.2023).
- Pateria, Shubham et al. (2021). ‘Hierarchical reinforcement learning: A comprehensive survey’. In: *ACM Computing Surveys (CSUR)* 54.5, pp. 1–35.
- Picardo, Elvis (2022). *How Game Theory Strategy Improves Decision Making*. URL: <https://www.investopedia.com/articles/investing/111113/advanced-game-theory-strategies-decisionmaking.asp>. (accessed: 17.06.2023).
- Salamat, S et al. (2019). ‘Workload-aware opportunistic energy efficiency in multi-FPGA platforms, in 2019 IEEE’. In: *ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE.
- Schulman, John, Sergey Levine et al. (2015). ‘Trust region policy optimization’. In: *International conference on machine learning*. PMLR, pp. 1889–1897.
- Schulman, John, Filip Wolski et al. (2017). ‘Proximal policy optimization algorithms’. In: *arXiv preprint arXiv:1707.06347*.
- Schvchenko, Nataliya (2020). *An Introduction to Model-Based Systems Engineering (MBSE)*. URL: <https://insights.sei.cmu.edu/blog/introduction-model-based-systems-engineering-mbse/>. (accessed: 05.06.2023).

-
- Schwenzer, Max et al. (2021). ‘Review on model predictive control: An engineering perspective’. In: *The International Journal of Advanced Manufacturing Technology* 117.5-6, pp. 1327–1349.
- Shapley, Lloyd S (1953). ‘Stochastic games’. In: *Proceedings of the national academy of sciences* 39.10, pp. 1095–1100.
- Singh, Amit Kumar et al. (2017). ‘A survey and comparative study of hard and soft real-time dynamic resource allocation strategies for multi-/many-core systems’. In: *ACM Computing Surveys (CSUR)* 50.2, pp. 1–40.
- Terry, J et al. (2021). ‘Pettingzoo: Gym for multi-agent reinforcement learning’. In: *Advances in Neural Information Processing Systems* 34, pp. 15032–15043.
- Walliser, Bernard et al. (2008). *Information and beliefs in game theory*. Tech. rep.
- Yang, Qisong et al. (2021). ‘WCSAC: Worst-case soft actor-critic for safety-constrained reinforcement learning’. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 12, pp. 10639–10646.
- Zhang, Linrui et al. (2022). ‘Penalized proximal policy optimization for safe reinforcement learning’. In: *arXiv preprint arXiv:2205.11814*.

Appendix

A Prototype Implementation

A.1 Parser Object

A.1.1 Class Parse

Method	Inputs	Outputs	Description
<code>--init--</code>	Name	None	It initializes the object's attributes. In this case, the method takes a parameter called name , which is used to set the name attribute of the object being created.
<code>from_file</code>	<code>file_name</code>	system	It processes a given file containing system data, utilizing an XML parser if the file has an XML extension. It extracts and returns the parsed system information.

A.1.2 Class Parse_XML

Method	Inputs	Outputs	Description
<code>--init--</code>	<code>self, name</code>	None	It initializes the object's attributes. In this case, the method takes a parameter called name , which is used to set the name attribute of the object being created.
<code>from_file</code>	<code>self, file_name</code>	system	It reads an XML file, extracts information about system components and their relationships, uses a factory pattern to create corresponding objects, and then returns the system object that embodies the structure and components of the system described in the XML file.

A.1.3 Class Factory

Method	Inputs	Outputs	Description
<code>get_system_and_builder</code>	<code>self, data</code>	factory, system	It determines the correct factory and system object based on input data. It checks for a 'tag' key in the data and, if its value is 'model', extracts the system type using the 'type' key. Depending on the system type (e.g., 'DFT' or 'DTMC'), it assigns a corresponding factory (e.g., <code>Factory_DFT</code> or <code>Factory_DTMC</code>). The method then utilizes this factory to create a system object using the input data and returns both the chosen factory and the generated system object.

A.1.4 Class Factory_DTMC

Method	Inputs	Outputs	Description
create_system	self, data	system	It generates a system using input data. It initializes an instance of the System_DTMC class with a name obtained from the input data and returns this newly created system instance. The input data is expected to contain attributes necessary for system creation, including the system name.
create_object	self, data, system	None	It takes input data and system information, processes the data based on the object type, creates appropriate objects (either State or Edge), and adds these objects to the system's list of objects. The extracted inputs include information about the type of object and its associated data (like state name, number, type, etc.), while the outputs are the created object instances that are appended to the system's list of objects.

A.2 System Object

A.2.1 Class System

Method	Inputs	Outputs	Description
__init__	self, name, type	None	It takes 'name' and 'type' as arguments to set attributes. The 'name' attribute stores the given name, while 'type' holds the provided type.
get_object	self, data, system	None	It retrieves an object from a collection based on its name. It scans through the stored objects, looking for a match with the provided name, and returns the corresponding object.
get_name	self	name	It retrieves and provides the value stored in the name attribute.

A.2.4 Class System_DTMC

Method	Inputs	Outputs	Description
__init__	self, name	None	It initializes an object within a class that likely inherits from the class System. It takes a parameter name, representing the object's name. The method initializes the object by calling the superclass's constructor with the provided name and the string 'DTMC' as arguments.
reset_system	self	None	It resets the system by performing a series of actions, such as resetting the clock, resetting objects associated with the system, updating the transition matrix, and setting the current state to the initial state.
update_transition_matrix_from_objects	self, objects	None	It updates the transition matrix of the system using the objects provided.

set_num_states	self, num_states	None	It sets the number of states attribute of the class instance.
set_num_edges	self, num_edges	None	It sets the number of edges attribute of the class instance.
set_current_state	self, current_state	None	It sets the current state attribute of the class instance.
set_initial_state	state	None	It sets the initial state and current state attributes of the class instance, it also performs validation on the given state value.
set_transition_matrix	self, transition_matrix	None	It assigns a given matrix as the transition matrix in the class. It converts the input matrix into a NumPy array and stores it as the class attribute transition_matrix.
transition_to_state	self, state	None	It takes a target state as input and checks if an initial state is set. If the target state is within a valid range, it calculates the transition probability using a matrix. A random number determines whether the transition occurs, updating the current state if the generated number is below the transition probability. This method ensures controlled state transitions based on probabilities and handles errors related to state validity and initialization.
transition	self	next_state	It simulates state transitions in a system. It uses transition probabilities from a matrix associated with the current state. By selecting the next state randomly according to these probabilities, it updates the current state and returns the new state. The method does not directly handle input and output extraction but focuses on managing state transitions based on probabilities.
_is_valid_transition_matrix	self, transition_matrix	None	It checks the validity of a square transition matrix by evaluating two conditions: matrix dimensions must be equal, and each row's probabilities must meet specific criteria. It assesses if any row contains negative values or if the sum of probabilities in a row differs significantly from 1. The method outputs True for a valid matrix and False otherwise, effectively extracting matrix inputs while determining their validity.
get_current_state	self	current_state	It gets the system's current state.
get_next_state	self	None	It gets one of the systems possible next states.
get_transition_matrix	self	transition_matrix	It gets the system's transition matrix.
get_num_states	self	num_states	It gets the system's number of states.
get_state_undesirability	self, state	state_undesirability	It gets the state desirability of the input state stored in the state object.
get_state_name	self, state	state_name	It gets the name of the input state stored in the state object.
get_state_type	self, state	state_type	It gets the type of the input state stored in the state object.

get_state_number	self, name	state_number	It gets the number of the state with the input name.
simulate	self, num_steps	None	It simulates the Markovian game for num_steps steps, starting from the initial states. It returns a list of lists, where each inner list represents the sequence of states visited by the players during the simulation.

A.3 DTMC Object

A.3.1 Class Element

Method	Inputs	Outputs	Description
<code>__init__</code>	self, element_type	None	It initializes the class attributes, which are the element type (either State or object).
<code>reset</code>	self	None	It resets the element class attributes.
<code>conv_str_to_number</code>	self, string	number	It converts a number value from string format to integer format. The Parser Object extracts data such as state number, states between edges, and state desirability from an XML file as a string.

A.3.2 Class State

Method	Inputs	Outputs	Description
<code>__init__</code>	self, element_type, state_name, state_number, state_type, state_desirability	None	It inherits the element class. It initializes the state class attributes which are element type, state name, state number, state type, and state desirability.

A.3.3 Class Edge

Method	Inputs	Outputs	Description
<code>__init__</code>	self, element_type, source_state, target_state, prob_value	None	It inherits the element class. It initializes the state class attributes which are element type, state name, state number, state type, and state desirability.

A.4 Game Object

A.4.1 Class Game

Method	Inputs	Outputs	Description
<code>__init__</code>	<code>self</code> , <code>System_Objects</code> , <code>max_num_players</code> , <code>max_steps</code>	None	It initializes the class attributes which are an array of system objects, maximum number of players, array of good AI players, array of bad AI, teams, initial state, current state, maximum number of steps, game over flag, number of game states, array of failure states, dictionary of players' initial resources, and dictionary of players' current resources.
<code>set_initial_state</code>	<code>self</code>	<code>initial_states</code>	It sets the game initial state.
<code>set_current_state</code>	<code>self</code>	None	It sets the game current state.
<code>set_failure_states</code>	<code>self</code>	None	It sets the game failure states.
<code>update_resources</code>	<code>self</code>	None	It updates the players' resources after each taken action.
<code>create_player</code>	<code>self</code> , <code>name</code> , <code>strategy_type</code> , <code>resources</code>	<code>player</code>	It creates a new player with a certain name, strategy type, and initial resources and adds it to the game array of players if the number of players will not exceed the maximum number of players.
<code>get_valid_actions</code>	<code>self</code> , <code>player</code>	<code>valid_actions</code>	It returns the array of valid actions for the input player.
<code>is_game_over</code>	<code>self</code>	None	It sets the game over flag if the current steps reach the maximum number of steps.
<code>get_winner</code>	<code>self</code>	<code>winners</code>	It gets the game array of winners.
<code>get_player_by_name</code>	<code>self</code> , <code>name</code>	<code>player</code>	It gets the player object with a certain name.
<code>get_players</code>	<code>self</code>	<code>players</code>	It gets the game array of players.
<code>get_players_strategy_type</code>	<code>self</code> , <code>strategy_type</code>	<code>players</code>	It gets an array of either good AI or bad AI players based on the input strategy type.
<code>get_num_players</code>	<code>self</code>	<code>num_players</code>	It gets the number of game players.
<code>get_num_players_strategy_type</code>	<code>self</code> , <code>strategy_type</code>	<code>num_players</code>	It gets the number of either good AI or bad AI players based on the input strategy type.
<code>get_current_state</code>	<code>self</code>	<code>current_states</code>	It gets an array of system objects' current state.
<code>get_current_state_desirabilities</code>	<code>self</code>	<code>state_undesirabilities</code>	It gets the desirability of each current state for each system object.
<code>get_num_states</code>	<code>self</code>	<code>num_states</code>	It gets the number of game states.
<code>get_initial_state</code>	<code>self</code>	None	It gets the game initial state.
<code>get_game_over_flag</code>	<code>self</code>	<code>game_over_flag</code>	It gets the game's over flag.
<code>get_current_step</code>	<code>self</code>	<code>current_step</code>	It gets the game's current step.
<code>get_max_steps</code>	<code>self</code>	<code>max_steps</code>	It gets the game's maximum number of steps.
<code>get_num_system_objects</code>	<code>self</code>	<code>num_system_objects</code>	It gets the game number of system objects.
<code>get_current_state_names</code>	<code>self</code>	<code>current_state_names</code>	It gets the names of the game current states of each system object.
<code>get_current_state_types</code>	<code>self</code>	<code>current_state_types</code>	It gets an array of current state types for each of the system objects.
<code>get_resources_players</code>	<code>self</code>	<code>resources_players</code> , <code>resources_players</code>	It gets the current resources of the game's players.
<code>increase_steps</code>	<code>self</code>	None	It increases the steps of the game by one step.
<code>perform_action</code>	<code>self</code>	<code>system_action</code>	It applies an action on a system.

<code>play</code>	<code>self, Path, Good.AI, Bad.AI</code>	<code>winners</code>	It simulates one game. It takes a path variable to store the game result, a good AI, and a bad AI-trained model for obtaining players' actions at each game step.
<code>action_masking</code>	<code>player, action</code>	<code>action, cost</code>	It masks the player's action before being applied to the game and calculates its cost. If the action is not valid, it returns a None action and zero cost.
<code>reset_game</code>	-	-	It resets all the system attributes.
<code>record_player_behavior</code>	<code>self, path, player, observation, action</code>	None	It records the observation and action taken by a player at each step.
<code>check_system_failure</code>	<code>self</code>	<code>bool</code>	It returns whether the system is currently in a failure state.

A.4.2 Class Player

Method	Inputs	Outputs	Description
<code>__init__</code>	<code>self, name, strategy_type, resources, systems</code>	None	It initializes the attributes of the player's class which are the name, array of valid action, score, strategy type, initial resources, and resources.
<code>choose_action</code>	<code>self, valid_actions</code>	None	It returns a random action of the system valid actions.
<code>create_valid_actions</code>	<code>self, System_Objects</code>	<code>valid.action, valid.action_mask</code>	It creates the system valid actions and valid actions masks.
<code>increase_score</code>	<code>self</code>	None	It increases the player's score by one.
<code>decrease_score</code>	<code>self</code>	-	It decreases the player's score by one.
<code>set_score</code>	<code>self, points</code>	None	It adds points to the player's score.
<code>get_valid_actions</code>	<code>self</code>	<code>valid.actions</code>	It gets the player's valid action array.
<code>get_valid_actions_mask</code>	<code>self</code>	<code>valid.actions_mask</code>	It gets the player's valid action mask array.
<code>get_action_num</code>	<code>self, action</code>	<code>action.num</code>	It gets the index of the action in the valid actions array.
<code>get_score</code>	<code>self</code>	<code>score</code>	It gets the player's score.
<code>get_name</code>	<code>self</code>	<code>name</code>	It gets the player's name.
<code>get_player_strategy</code>	<code>self</code>	<code>strategy_type</code>	It gets the player's strategy type.
<code>get_resources</code>	<code>self</code>	<code>resources</code>	It gets the player's current resources.
<code>set_resources</code>	<code>self, resources</code>	<code>resources</code>	It sets the player's current resources.
<code>update_valid_actions_mask</code>	<code>self, system</code>	None	It updates the player's valid action mask based on the system's current state.
<code>reset_player</code>	<code>self</code>	None	It resets all the player's attributes.

A.5 Environment Object

A.5.1 Class `Raw_Env`

Method	Inputs	Outputs	Description
<code>__init__</code>	<code>self</code> , <code>game</code> , <code>render_mode</code>	None	It inherits the <code>AECEnv</code> class. It sets up the initial state and parameters for the environment or game being modeled, using information from the game object to define action and observation spaces and agent-related attributes.
<code>seed</code>	<code>self</code> , <code>seeds</code>	None	It sets a random seed, which can control the randomness of certain operations, making them reproducible. If a basis is provided, it sets the seed for NumPy's random number generator. It may use a default source for randomness if no seed is provided.
<code>reset</code>	<code>self</code> , <code>seed</code> , <code>return_info</code> , <code>options</code>	None	It is responsible for initializing various attributes and setting up the environment for subsequent operations like <code>render()</code> , <code>step()</code> , and <code>observe()</code> . It prepares the environment and various tracking attributes for a new game or episode, ensuring that subsequent actions can be carried out smoothly.
<code>observe</code>	<code>agent</code>	<code>obs</code>	It returns the observation of the specified agent. This function should return a sane observation (though not necessarily the most up-to-date possible) at any time after <code>reset()</code> is called.
<code>step</code>	<code>self</code> , <code>action</code>	None	It is used to advance the game environment by taking a specified action for the current agent and updating various aspects of the game state. It is a fundamental part of the game simulation process, where agents take actions, the game state evolves, rewards are calculated, and the environment is updated accordingly.
<code>render</code>	<code>self</code> , <code>mode</code>	None	It is used to visualize the current state of the environment, especially in a human-readable form. Depending on the number of agents and the render mode, it constructs and prints information about the agents' actions or the game's status.
<code>close</code>	<code>self</code>	None	It cleans up any environmental resources.
<code>reward_function</code>	<code>self</code> , <code>player</code> , <code>old_state_undesirabilities</code> , <code>new_state_undesirabilities</code>	<code>reward</code>	It is designed to calculate a reward value. This reward is determined based on the changes in state desirabilities between the old and new states, and it takes several steps into account to compute this reward.

observation_space	self, agent	obs	When it is called with an agent argument, retrieves and returns the observation space specific to that agent. The use of the <code>@functools.lru_cache</code> decorator suggests that the method's results are cached to improve efficiency when called multiple times with the same agent.
action_space	self, agent	action	It uses memorization via the <code>functools.lru_cache</code> decorator. It takes an agent as a parameter and returns the associated action_space from the <code>self.action_spaces</code> dictionary. The method doesn't directly gather inputs but relies on the provided agent to determine the action space it should retrieve.

A.6 PPO Model

A.6.1 Class Neural Network

Method	Inputs	Outputs	Description
<code>__init__</code>	self, in_dim, out_dim, strategy_type	None	It initializes the class with input and output dimensions, and a strategy type. The neural network architecture consists of three hidden layers, each with 1024 units and ReLU activation functions. It has two output layers: one for actions (<code>self.actor</code>) and one for value estimation (<code>self.critic</code>).

A.6.2 Class Categorical Masked

Method	Inputs	Outputs	Description
<code>__init__</code>	self, probs, logits, validate_args, masks	None	It allows you to create categorical distributions with the capability to apply masks to the distribution, effectively zeroing out certain categories based on the provided masks. This can be useful in various scenarios, especially in cases where you want to restrict or condition the distribution based on certain criteria defined by masks.
<code>entropy</code>	self	entropy	It calculates the entropy of a distribution, but when masks are present, it considers only the elements of the distribution where the masks are True, effectively masking out specific elements before performing the entropy calculation.

A.6.3 Class Agent

Method	Inputs	Outputs	Description
<code>__init__</code>	<code>self, env,</code> <code>in_dim,</code> <code>out_out</code>	None	It initializes instances of a class by setting up attributes to store an environment object and two neural networks labeled as "GOOD" and "BAD." These neural networks are presumably used for some kind of AI training or decision-making within the context of the larger program. However, some variables, like <code>device</code> , are not defined within this method and need to be defined elsewhere in the code for this class to work correctly.
<code>get_value</code>	<code>self, x,</code> <code>strategy_type</code>	value	It takes an input <code>x</code> , processes it using a neural network associated with a specific strategy type, and then returns a value, possibly representing some evaluation or prediction based on the input data and the chosen neural network strategy. The exact interpretation and functionality of this method would depend on the context and the specific neural network models involved.
<code>get_action_and_value</code>	<code>self, x,</code> <code>strategy_type,</code> <code>action,</code> <code>invalid_action_masks</code>	action, value	It takes input data <code>x</code> , a strategy type, optional action, and invalid action masks. It processes the input through neural networks to compute action probabilities. If invalid action masks are provided, it creates masked probability distributions. It then samples actions and calculates log probabilities and entropies. The method returns the sampled actions, the sum of log probabilities, the sum of entropies, and a value estimate from a critical network. Essentially, it's responsible for action selection and value estimation for AI agents in a reinforcement learning context.
<code>get_random_action</code>	<code>invalid_action_masks</code>	action	It is designed to choose a random valid action from a set of actions based on the provided invalid action masks. It ensures that only valid actions are considered for selection.
<code>learn</code>	<code>self, Path,</code> <code>env,</code> <code>total_episode</code>	None	It is responsible for training AI agents using the Proximal Policy Optimization (PPO) algorithm. It begins by setting up algorithm parameters, initializing the environment, and preparing data structures for episode storage. It then enters a training loop where it collects observations, calculates rewards and losses, and optimizes the policy and value networks. Training statistics such as episodic returns and loss values are recorded and printed. Periodically, trained models are saved, and training curves are plotted. Overall, this method manages the entire training process for AI agents within a complex gaming environment.

<code>render_policy</code>	<code>self, path, env</code>	None	It is used to visualize the policy of a reinforcement learning agent over five episodes. It first sets the model to evaluation mode, prepares a directory for storing results, and then iterates through episodes. In each episode, it resets the environment, collects rewards and actions for each agent, and plots the rewards' progress, saving them as images. The method also prints agent information and the states where episodes end. Inputs to this method are the saving path and the environment, and its primary outputs are reward plots and diagnostic information.
----------------------------	------------------------------	------	---

A.6.4 Helper Methods

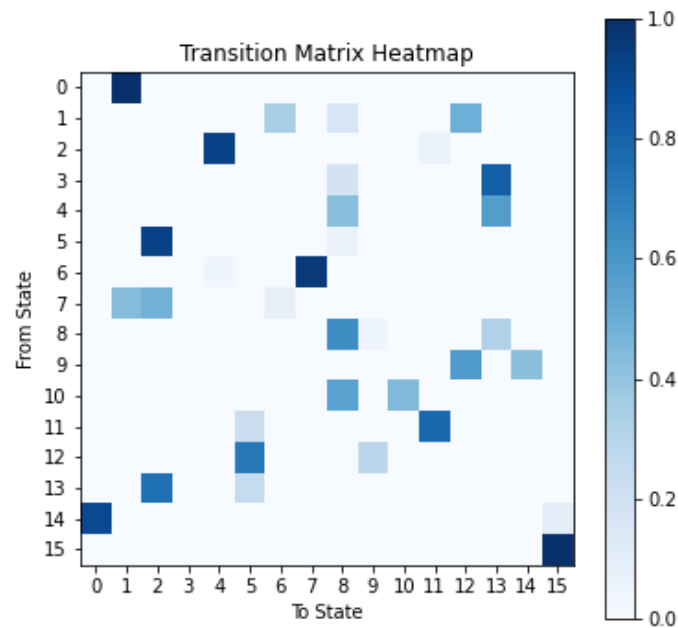
Method	Inputs	Outputs	Description
<code>train</code>	<code>path, env, actor_model, critic_model, total_episodes</code>	None	It sets up and trains a PPO reinforcement learning agent on the provided environment for a specified number of episodes, and it may save training-related data in the specified PATH. The optional rendering functionality is currently disabled but can be uncommented if needed for visualization. Additionally, the <code>actor_model</code> and <code>critic_model</code> are expected to be pre-trained neural network models used in the PPO agent.
<code>create_ppo_agent</code>	<code>env, actor_model, critic_model</code>	<code>model</code>	It initializes a PPO agent for reinforcement learning, allowing for the loading of pre-trained models if available, and ensures that both actor and critic models are specified correctly or omitted to prevent errors.
<code>threshold</code>	<code>val, minval, maxval</code>	<code>val</code>	It is used to ensure that the value remains within the boundaries defined by the min value and the max value. It acts as a simple clamp or boundary-checking mechanism.
<code>batchify_obs</code>	<code>- obs, agent, device</code>	<code>- obs</code>	It is used to convert observations in a specific format (referred to as "PZ style") into a batch of Torch tensors. It takes the observations, the agent for which the observations are intended, and the target computing device as inputs. The function first organizes the observations into a dictionary and then stacks them into a NumPy array. Next, it converts this array into Torch tensors and ensures they are placed on the specified computing device. Overall, the function is designed to prepare the observations for further processing, likely within a machine learning or deep learning context.
<code>batchify</code>	<code>x, device</code>	<code>x</code>	It takes a dictionary-like structure (<code>x</code>) containing data, stack the data into a batched tensor, and ensures that the resulting tensor is placed on the specified computing device. This is often used when preparing data for neural network training in PyTorch.

unbatchify	x	x	It converts a NumPy array into a more usable format, but the specific format it returns depends on whether the commented-out code is active or not. If active, it would return a dictionary; otherwise, it returns the first element of the NumPy array.
------------	---	---	--

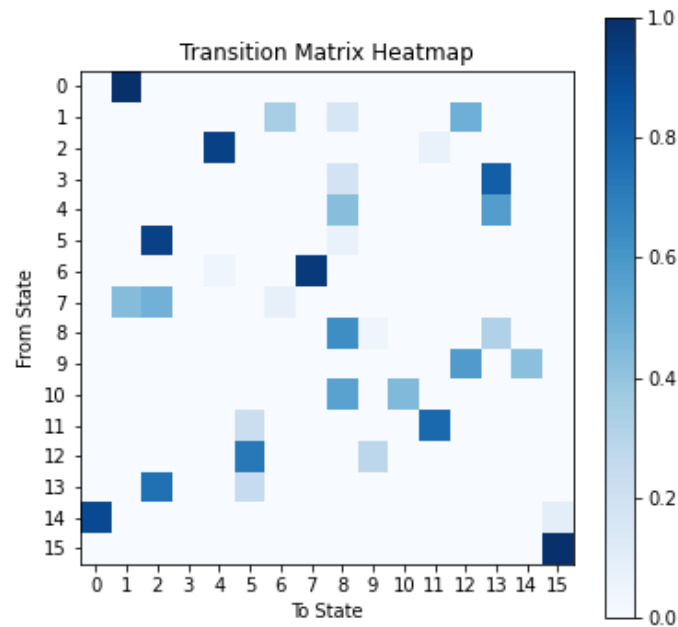
A.7 DTMC Model Generator

Method	Inputs	Outputs	Description
generate_model	num_states, max_edges, model_name	None	It generates a DTMC model, creates a visual representation of it, constructs an XML representation of the model, and saves it to a file. It also provides print statements for tracking progress and confirming the save operation.
generate_random_transition_matrix	num_states, max_edges, transition_matrix	None	It generates a random transition matrix for a Markov chain with a specified number of states, where each state can have a random number of transitions to other states, and the transition probabilities are also random but normalized to ensure the matrix is a valid transition matrix.
draw_transition_matrix	transition_matrix, model_name	None	It takes a transition matrix and a model name as inputs, and it generates a visual representation of the transition probabilities as a directed graph, which can be saved as an image file.
draw_heat_map	transition_matrix, model_name	None	It takes a transition matrix and a model name as input creates a heatmap visualization of the transition matrix, adds labels and a color bar for reference, saves the heatmap as an image file, and then displays the heatmap. This can be useful for visualizing and analyzing the transitions between states in a system or model.
quotes	string	string	It adds double quotes to the input string and can be used to format strings for various purposes like creating XML strings or formatting text for specific output requirements.
indentation	None	None	It returns a string consisting of four space characters, effectively creating an indentation of four spaces.

B DTMCs of Different Experiments

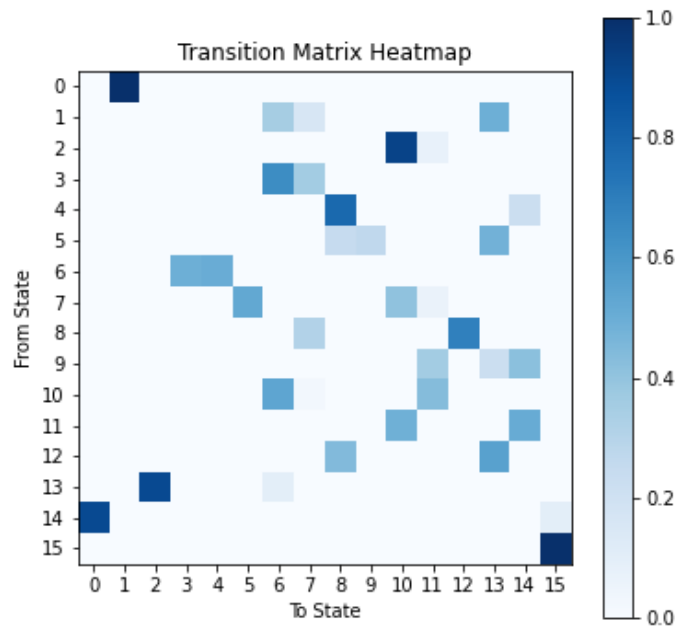


Experiment 1: Transition matrix

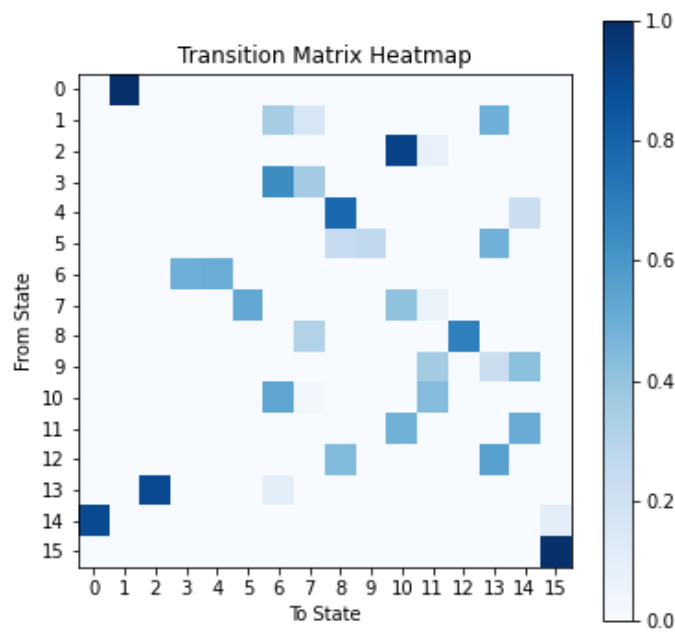


Experiment 2: Transition matrix

Figure 33: Transition matrices for experiments 1 and 2.

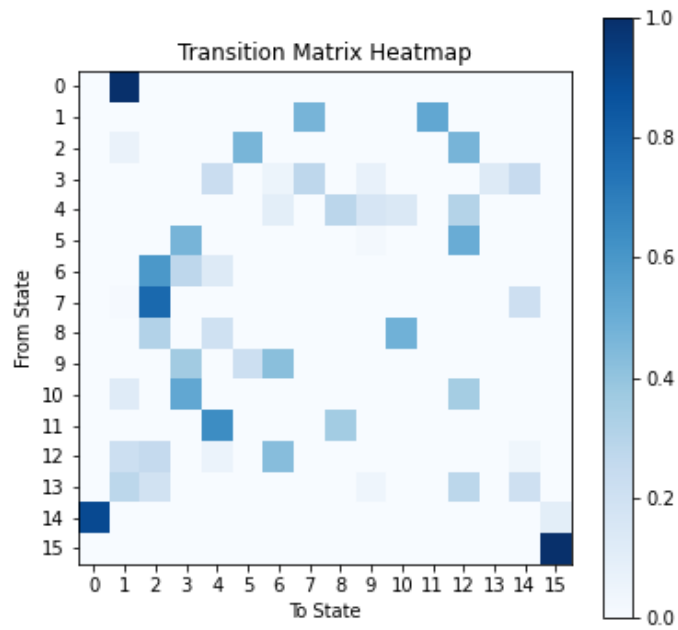


Experiment 3: Transition matrix

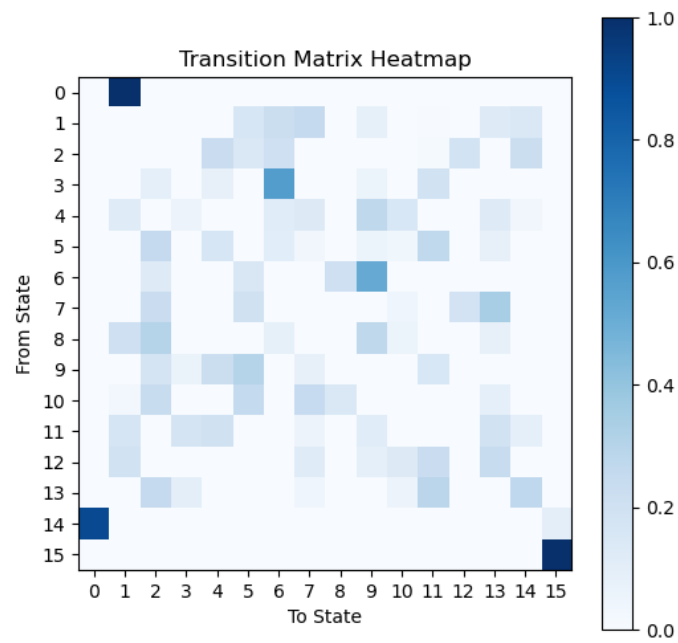


Experiment 4: Transition matrix

Figure 34: Transition matrices for experiments 3 and 4.

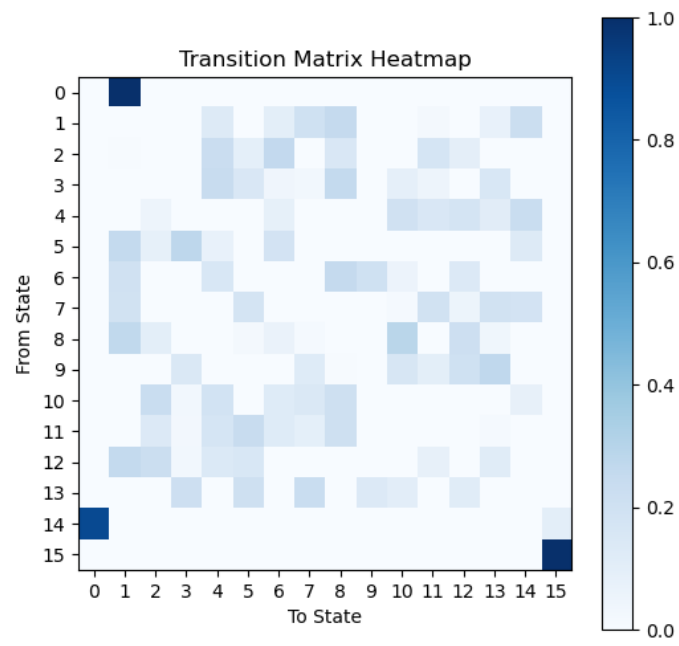


Experiment 5.1: Transition matrix

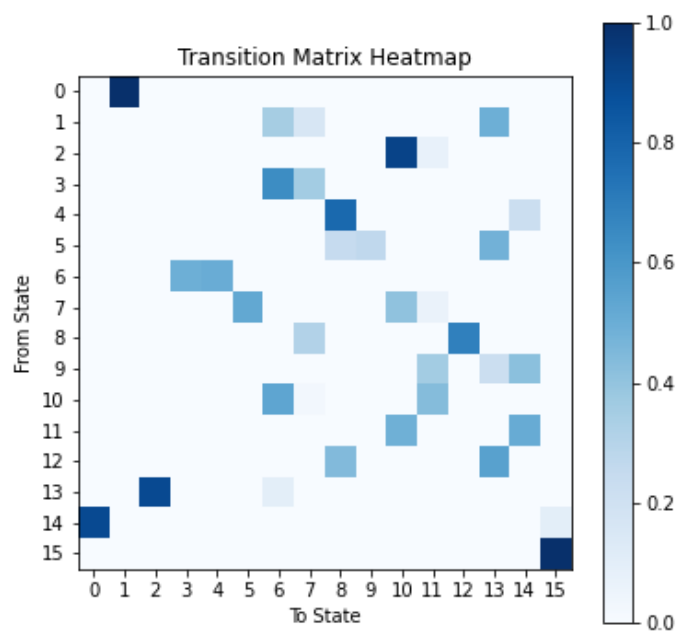


Experiment 5.2: Transition matrix

Figure 35: Transition matrices for experiments 5.1 and 5.2.

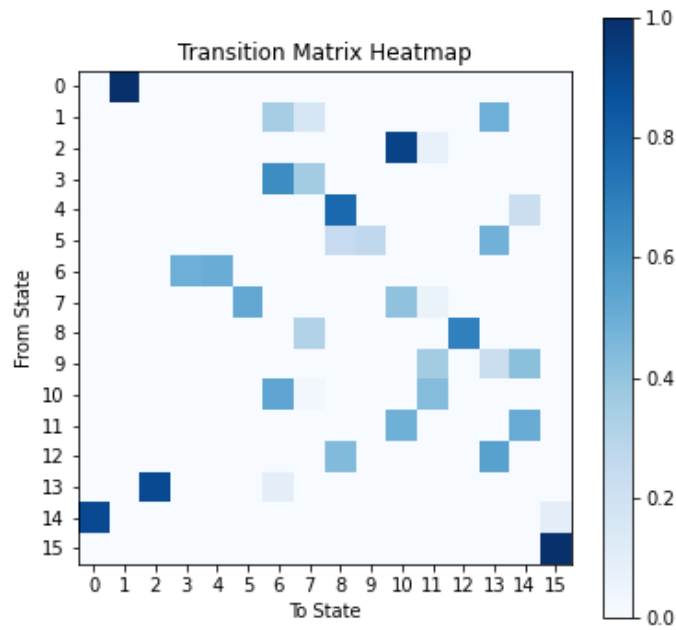


Experiment 5.3: Transition matrix

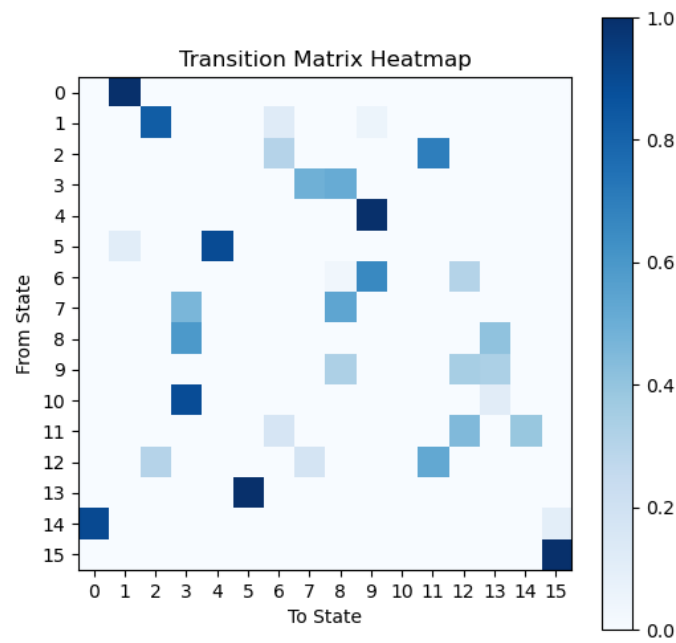


Experiment 6: Transition matrix

Figure 36: Transition matrices for experiments 5.3 and 6.

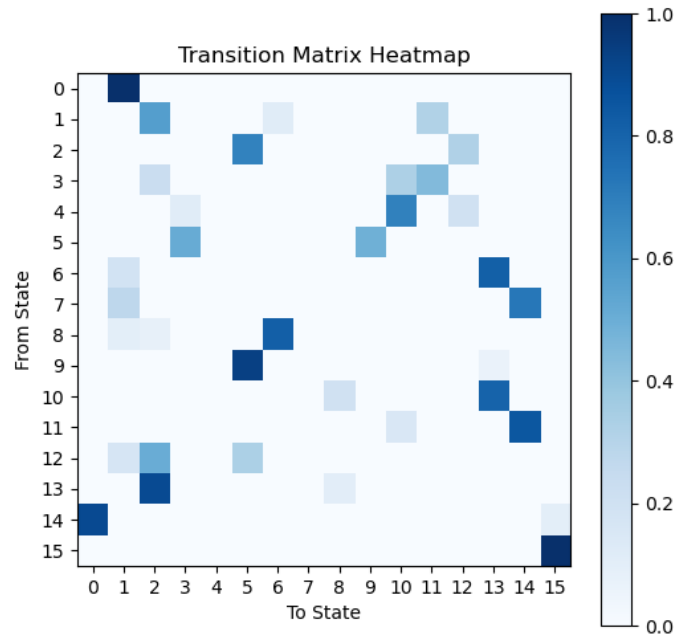


Experiment 7: Transition matrix

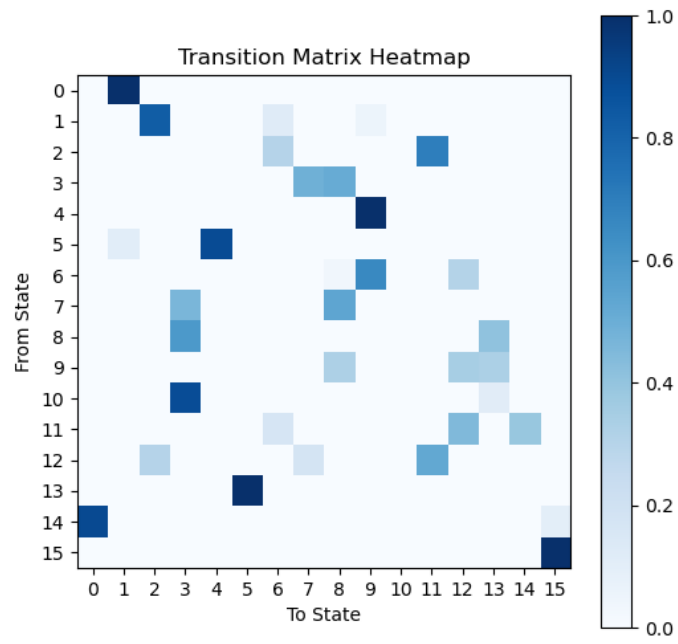


Experiment 8: Transition matrix

Figure 37: Transition matrices for experiments 7 and 8.

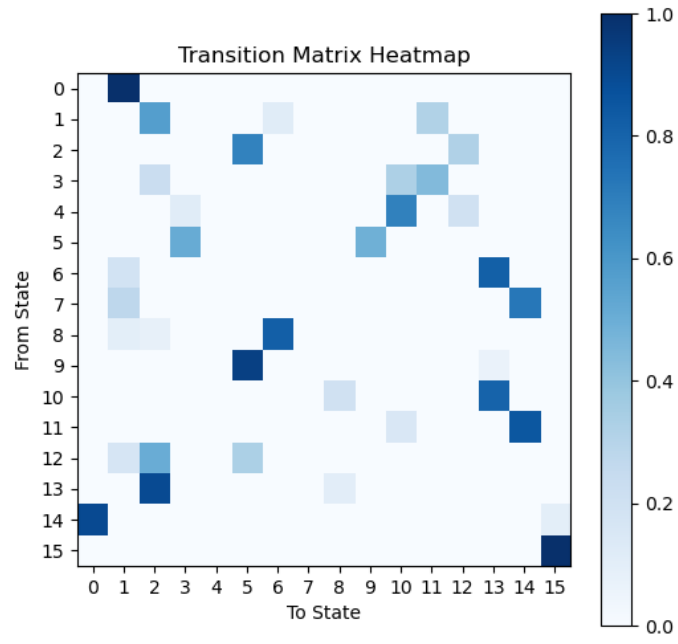


Experiment 9: Transition matrix

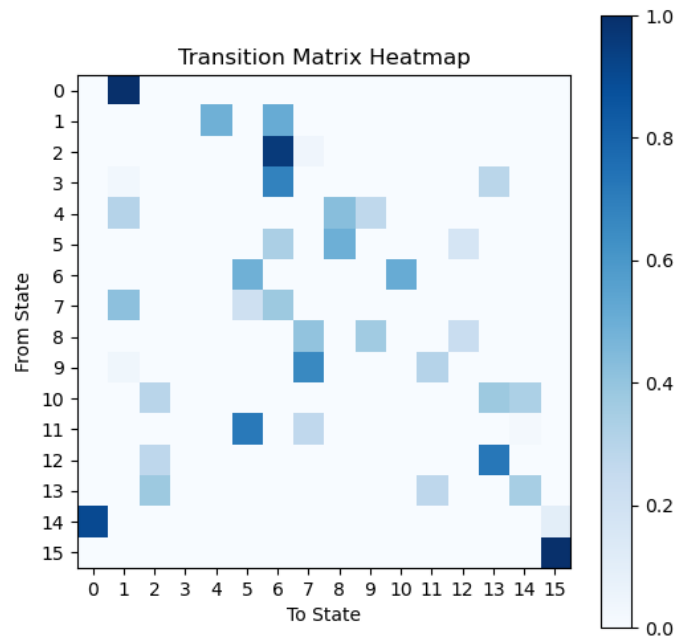


Experiment 10: Transition matrix

Figure 38: Transition matrices for experiments 8 and 9.

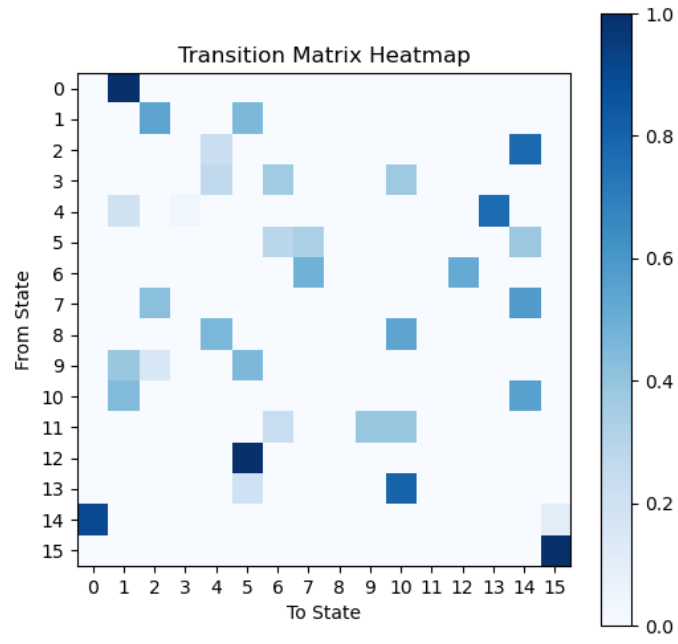


Resource Optimization Experiment 1: Transition matrix

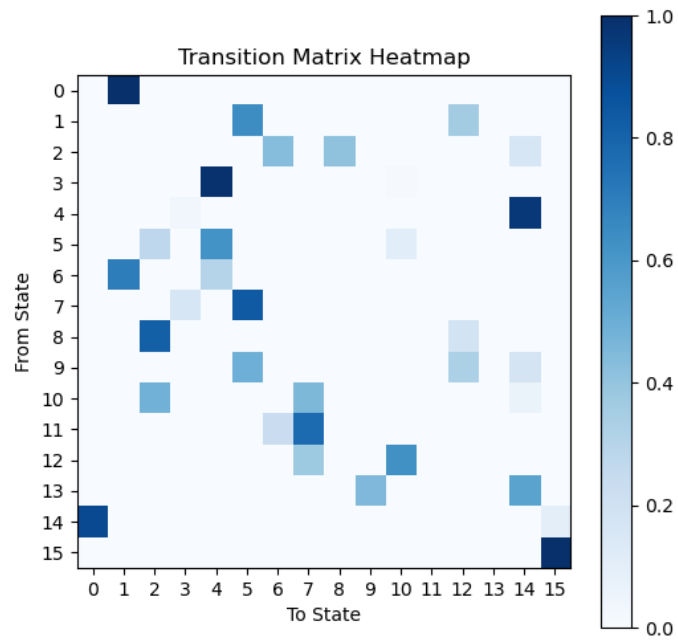


Resource Optimization Experiment 2: Transition matrix

Figure 39: Transition matrices for resource optimization experiments 1 and 2.



Resource Optimization Experiment 3: Transition matrix



Resource Optimization Experiment 4: Transition matrix

Figure 40: Transition matrices for resource optimization experiments 3 and 4.

C Experiment Results

C.1 Experiment 1

1. Blue_Vs_Red before training (1_{st})

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with several Epochs = 0

Total number of game simulations = 100000

Good Team wins 29926 times with 29.926 %

Bad Team wins 68612 times with 68.612 %

End with Draw 1462 times with 1.462 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 4, 8, 9, 14] was taken 112 times.

Path.1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 5 times.

Path.2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 5 times.

Path.3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 864 times.

Path.4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 402 times.

Path.5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 10 times.

Path.6: [0, 1, 8, 9, 14] was taken 1902 times.

Path.7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 1504 times.

Path.8: [0, 1, 12, 5, 8, 9, 14] was taken 281 times.

Path.9: [0, 1, 12, 9, 14] was taken 14664 times.

2. Blue_Vs_Red before training (2_{nd})

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with several Epochs = 0

Total number of game simulations = 100000

Good Team wins 38763 times with 38.763 %

Bad Team wins 60596 times with 60.596 %

End with Draw 641 times with 0.641 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 4, 8, 9, 14] was taken 51 times.

Path.1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 13 times.

Path.2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 1 times.

Path.3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 443 times.

Path.4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 244 times.

Path.5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 4 times.

Path.6: [0, 1, 8, 9, 14] was taken 867 times.

Path.7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 636 times.

Path.8: [0, 1, 12, 5, 8, 9, 14] was taken 121 times.

Path.9: [0, 1, 12, 9, 14] was taken 6378 times.

3 Blue_Vs_Red before training (3_{rd})

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with several Epochs = 0

Total number of game simulations = 100000

Good Team wins 33248 times with 33.248 %

Bad Team wins 66243 times with 66.243 %

End with Draw 509 times with 0.509 %
Safety result: The system is unsafe
Paths Records:
Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 38 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 3 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 1 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 405 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 266 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 4 times.
Path_6: [0, 1, 8, 9, 14] was taken 830 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 805 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 152 times.
Path_9: [0, 1, 12, 9, 14] was taken 5134 times.

4. Blue_Vs_Red before training (4_{th})

Experiment Summary: Blue_Vs_Red
Description: Agents were trained with several Epochs = 0
Total number of game simulations = 100000
Good Team wins 34423 times with 34.423 %
Bad Team wins 65157 times with 65.157 %
End with Draw 420 times with 0.42 %
Safety result: The system is unsafe
Paths Records:
Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 52 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 1 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 2 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 329 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 291 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 5 times.
Path_6: [0, 1, 8, 9, 14] was taken 703 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 691 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 147 times.
Path_9: [0, 1, 12, 9, 14] was taken 3162 times.

5. Blue_Vs_Red after training (1_{st})

Experiment Summary: Blue_Vs_Red
Description: Agents were trained with several Epochs = 100000
Total number of game simulations = 100000
Good Team wins 6407 times with 6.407 %
Bad Team wins 93592 times with 93.592 %
End with Draw 1 time with 0.001 %
Safety result: The system is unsafe
Paths Records:
Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 0 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 0 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_6: [0, 1, 8, 9, 14] was taken 13 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 0 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.

Path_9: [0, 1, 12, 9, 14] was taken 13 times.

6. Blue_Vs_Red after training (2_{nd})

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with several Epochs = 100000

Total number of game simulations = 100000

Good Team wins 1243 times with 1.243 %

Bad Team wins 98757 times with 98.757 %

End with Draw 0 times with 0.0 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 20 times.

Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.

Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.

Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 330 times.

Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.

Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.

Path_6: [0, 1, 8, 9, 14] was taken 8 times.

Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 0 times.

Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.

Path_9: [0, 1, 12, 9, 14] was taken 4658 times.

7. Blue_Vs_Red after training (3_{rd})

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with several Epochs = 100000

Total number of game simulations = 100000

Good Team wins 2397 times with 2.397 %

Bad Team wins 97603 times with 97.603 %

End with Draw 0 times with 0.0 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 0 times.

Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.

Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.

Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 0 times.

Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.

Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.

Path_6: [0, 1, 8, 9, 14] was taken 15 times.

Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 0 times.

Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.

Path_9: [0, 1, 12, 9, 14] was taken 3383 times.

8. Blue_Vs_Red after training (4_{th})

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with a number of Epochs = 100000

Total number of game simulations = 100000

Good Team wins 27536 times with 27.536 %

Bad Team wins 72464 times with 72.464 %

End with Draw 0 times with 0.0 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 6 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 84 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_6: [0, 1, 8, 9, 14] was taken 29 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 53 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.
Path_9: [0, 1, 12, 9, 14] was taken 187 times.

9. Blue_Vs_Random after training (1_{st})

Experiment Summary: Blue_Vs_Random

Description: Agents were trained with a number of Epochs = 100000

Total number of game simulations = 100000

Good Team wins 43073 times with 43.073 %

Bad Team wins 56433 times with 56.433 %

End Team wins 494 times with 0.494 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 43 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 10 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 1235 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 478 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 3 times.
Path_6: [0, 1, 8, 9, 14] was taken 1426 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 186 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 5 times.
Path_9: [0, 1, 12, 9, 14] was taken 9163 times.

10. Random_Vs_Red after training (1_{st})

Experiment Summary: Random_Vs_Red

Description: Agents were trained with a number of Epochs = 100000

Total number of game simulations = 100000

Good Team wins 36946 times with 36.946 %

Bad Team wins 58703 times with 58.703 %

End with Draw 4351 times with 4.351 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 245 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 38 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 74 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 551 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 635 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 133 times.
Path_6: [0, 1, 8, 9, 14] was taken 2028 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 211 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 589 times.
Path_9: [0, 1, 12, 9, 14] was taken 13388 times.

11. Random_Vs_Random (1_st)

Experiment Summary: Random_Vs_Random

Description: Agents were trained with a number of Epochs = 100000

Total number of game simulations = 100000

Good Team wins 35639 times with 35.639 %

Bad Team wins 63838 times with 63.838 %

End Team wins 523 times with 0.523 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 4, 8, 9, 14] was taken 35 times.

Path.1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 4 times.

Path.2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 2 times.

Path.3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 404 times.

Path.4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 184 times.

Path.5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 4 times.

Path.6: [0, 1, 8, 9, 14] was taken 864 times.

Path.7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 806 times.

Path.8: [0, 1, 12, 5, 8, 9, 14] was taken 157 times.

Path.9: [0, 1, 12, 9, 14] was taken 7114 times.

C.2 Experiment 2

1. Random_Vs_Random

Experiment Summary: Random_Vs_Random

Description: Agents were trained with a number of Epochs = 100000

Total number of game simulations = 100000

Good Team wins 35541 times with 35.541 %

Bad Team wins 63908 times with 63.908 %

End with Draw 551 times with 0.551 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 4, 8, 9, 14] was taken 24 times.

Path.1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 2 times.

Path.2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 1 times.

Path.3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 407 times.

Path.4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 179 times.

Path.5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 6 times.

Path.6: [0, 1, 8, 9, 14] was taken 820 times.

Path.7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 818 times.

Path.8: [0, 1, 12, 5, 8, 9, 14] was taken 149 times.

Path.9: [0, 1, 12, 9, 14] was taken 6975 times.

2. Blue_Vs_Red after training (25000 episodes)

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with a number of Epochs = 25000

Total number of game simulations = 100000

Good Team wins 45444 times with 45.444 %

Bad Team wins 54556 times with 54.556 %

End with Draw 0 times with 0.0 %
Safety result: The system is unsafe
Paths Records:
Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 0 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 0 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_6: [0, 1, 8, 9, 14] was taken 0 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 0 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.
Path_9: [0, 1, 12, 9, 14] was taken 0 times.

3. Blue_Vs_Red after training (50000 episodes)

Experiment Summary: Blue_Vs_Red
Description: Agents were trained with a number of Epochs = 50000
Total number of game simulations = 100000
Good Team wins 3469 times with 3.469 %
Bad Team wins 96531 times with 96.531 %
End with Draw 0 times with 0.0 %
Safety result: The system is unsafe
Paths Records:
Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 0 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 0 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_6: [0, 1, 8, 9, 14] was taken 1333 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 0 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.
Path_9: [0, 1, 12, 9, 14] was taken 334 times.

4. Blue_Vs_Red after training (75000 episodes)

Experiment Summary: Blue_Vs_Red
Description: Agents were trained with a number of Epochs = 75000
Total number of game simulations = 100000
Good Team wins 0 times with 0.0 %
Bad Team wins 100000 times with 100.0 %
End with Draw 0 times with 0.0 %
Safety result: The system is unsafe
Paths Records:
Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 216 times.
Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 12445 times.
Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.
Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.
Path_6: [0, 1, 8, 9, 14] was taken 49 times.
Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 0 times.
Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.

Path_9: [0, 1, 12, 9, 14] was taken 75 times.

5. Blue_Vs_Red after training (100000 episodes)

Experiment Summary: Blue_Vs_Red

Description: Agents were trained with a number of Epochs = 100000

Total number of game simulations = 100000

Good Team wins 1611 times with 1.611 %

Bad Team wins 98305 times with 98.305 %

End with Draw 84 times with 0.084 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 4, 8, 9, 14] was taken 916 times.

Path_1: [0, 1, 6, 4, 13, 2, 11, 5, 8, 9, 14] was taken 0 times.

Path_2: [0, 1, 6, 4, 13, 5, 8, 9, 14] was taken 0 times.

Path_3: [0, 1, 6, 7, 2, 4, 8, 9, 14] was taken 468 times.

Path_4: [0, 1, 6, 7, 2, 4, 13, 5, 8, 9, 14] was taken 0 times.

Path_5: [0, 1, 6, 7, 2, 11, 5, 8, 9, 14] was taken 0 times.

Path_6: [0, 1, 8, 9, 14] was taken 35 times.

Path_7: [0, 1, 12, 5, 2, 4, 8, 9, 14] was taken 0 times.

Path_8: [0, 1, 12, 5, 8, 9, 14] was taken 0 times.

Path_9: [0, 1, 12, 9, 14] was taken 4609 times.

C.3 Experiment 3

1. Random_Vs_Random (50%-50%)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with a number of Epochs = 100000

2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Good Team wins 33678 times with 33.678 %

Bad Team wins 53572 times with 53.572 %

End with Ddraw 12750 times with 12.75 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 233 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 33 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 327 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 92 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 8 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 663 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 793 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 93 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 1554 times.

... (and so on)

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 50 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 103 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 19 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 884 times.

2. Blue_Vs_Red after training (50%-50%)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Good Team wins 1 time with 0.001 %

Bad Team wins 99986 times with 99.986 %

End with Draw 13 times with 0.013 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 0 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 0 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 0 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 0 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 0 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 0 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 0 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 0 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 0 times.

... (and so on)

Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 0 times.

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 0 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 0 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 0 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 0 times.

3. Random_Vs_Random (70%-30%)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with a number of Epochs = 100000
2. System has 68.75% desirable states and 31.25% undesirable states

Total number of game simulations = 100000

Good Team wins 42256 times with 42.256 %

Bad Team wins 30109 times with 30.109 %

End with Draw 27635 times with 27.635 %

Safety result: System is safe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 184 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 32 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 309 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 104 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 19 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 731 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 762 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 105 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 1610 times.

... (and so on)

Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 25 times.

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 59 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 100 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 30 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 819 times.

4. Blue_Vs_Red after training (70%-30%)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
2. System has 68.75% desirable states and 31.25% undesirable states

Total number of game simulations = 100000

Good Team wins 3077 times with 3.077 %

Bad Team wins 96923 times with 96.923 %

End with Draw 0 times with 0.0 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 0 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 0 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 0 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 0 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 0 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 0 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 0 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 0 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 0 times.

... (and so on)

Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 0 times.

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 0 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 0 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 0 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 0 times.

5. Random_Vs_Random (30%-70%)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with the number of Epochs = 100000
2. System has 31.25% desirable states and 68.75% undesirable states

Total number of game simulations = 100000

Good Team wins 30370 times with 30.37 %

Bad Team wins 42110 times with 42.11 %

End with Draw 27520 times with 27.52 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 237 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 25 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 320 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 93 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 17 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 702 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 773 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 111 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 1526 times.

... (and so on)

Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 20 times.

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 48 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 111 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 22 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 812 times.

6. Blue_Vs_Red after training (30%-70%)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
 2. System has 31.25% desirable states and 68.75% undesirable states
- Total number of game simulations = 100000
Good Team wins 99996 times with 99.996 %
Bad Team wins 1 time with 0.001 %
End with Draw 3 times with 0.003 %

Safety result: System is safe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 0 times.
Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 0 times.
Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 0 times.
Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 0 times.
Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 0 times.
Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 0 times.
Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 0 times.
Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 0 times.
Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 0 times.
... (and so on)
Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 0 times.
Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 0 times.
Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 0 times.
Path_72: [0, 1, 13, 6, 4, 14] was taken 0 times.

C.4 Experiment 4

1. Random_Vs_Random (Uniform Random)

Experiment Summary: Random_Vs_Random

Description: The system has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 24.01018

Good Team wins 33950 times with 33.95 %

Bad Team wins 53201 times with 53.201 %

End with Draw 12849 times with 12.849 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 205 times.
Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 30 times.
Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 319 times.
Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 91 times.
Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 12 times.
Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 750 times.
Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 747 times.
Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 104 times.
Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 1589 times.
... (and so on)
Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 21 times.
Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 45 times.
Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 99 times.
Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 32 times.

Path.72: [0, 1, 13, 6, 4, 14] was taken 868 times.

2. Random_Vs.Random (DTMC Solution)

Experiment Summary: Random_Vs.Random

Description:

1. System has 50% desirable states and 50% undesirable states
2. Transition happens based on transition probabilities (DTMC was solved)

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 22.48713

Good Team wins 26787 times with 26.787 %

Bad Team wins 58876 times with 58.876 %

End with Draw 14337 times with 14.337 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 444 times.

Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 59 times.

Path.2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 538 times.

Path.3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 168 times.

Path.4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 21 times.

Path.5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1337 times.

Path.6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1478 times.

Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 178 times.

Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2926 times.

... (and so on)

Path.68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 50 times.

Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 89 times.

Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 192 times.

Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 53 times.

Path.72: [0, 1, 13, 6, 4, 14] was taken 1446 times.

C.5 Experiment 5

1. Random_Vs.Random (6 transitions)

Experiment Summary: Random_Vs.Random

Description:

1. Agents were trained with a number of Epochs = 100000.
2. The system has 50% desirable states and 50% undesirable states.
3. The system has up to 6 transitions for each state.

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 24.42137

Good Team wins 55659 times with 55.659 %

Bad Team wins 43339 times with 43.339 %

End with Draw 1002 times with 1.002 %

Safety result: System is safe

Paths Records:

Path.0: [0, 1, 7, 2, 5, 3, 4, 8, 10, 12, 14] was taken 11 times.

Path.1: [0, 1, 7, 2, 5, 3, 4, 10, 12, 14] was taken 17 times.

Path.2: [0, 1, 7, 2, 5, 3, 4, 12, 14] was taken 53 times.

Path.3: [0, 1, 7, 2, 5, 3, 6, 4, 8, 10, 12, 14] was taken 0 times.

Path.4: [0, 1, 7, 2, 5, 3, 6, 4, 10, 12, 14] was taken 1 times.
 Path.5: [0, 1, 7, 2, 5, 3, 6, 4, 12, 14] was taken 4 times.
 Path.6: [0, 1, 7, 2, 5, 3, 9, 6, 4, 8, 10, 12, 14] was taken 0 times.
 Path.7: [0, 1, 7, 2, 5, 3, 9, 6, 4, 10, 12, 14] was taken 0 times.
 Path.8: [0, 1, 7, 2, 5, 3, 9, 6, 4, 12, 14] was taken 2 times.
 ... (and so on)
 Path.658: [0, 1, 11, 8, 10, 12, 6, 4, 9, 5, 3, 13, 14] was taken 0 times.
 Path.659: [0, 1, 11, 8, 10, 12, 6, 4, 9, 5, 3, 14] was taken 2 times.
 Path.660: [0, 1, 11, 8, 10, 12, 6, 4, 9, 5, 7, 14] was taken 0 times.
 Path.661: [0, 1, 11, 8, 10, 12, 14] was taken 152 times.

2. Blue_Vs_Red (6 transitions)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000.
2. The system has 50% desirable states and 50% undesirable states.
3. The system has up to 6 transitions for each state.

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 9.1293

Good Team wins 0 times with 0.0 %

Bad Team wins 100000 times with 100.0 %

End with Draw 0 times with 0.0 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 7, 2, 5, 3, 4, 8, 10, 12, 14] was taken 0 times.
 Path.1: [0, 1, 7, 2, 5, 3, 4, 10, 12, 14] was taken 0 times.
 Path.2: [0, 1, 7, 2, 5, 3, 4, 12, 14] was taken 0 times.
 Path.3: [0, 1, 7, 2, 5, 3, 6, 4, 8, 10, 12, 14] was taken 0 times.
 Path.4: [0, 1, 7, 2, 5, 3, 6, 4, 10, 12, 14] was taken 0 times.
 Path.5: [0, 1, 7, 2, 5, 3, 6, 4, 12, 14] was taken 0 times.
 Path.6: [0, 1, 7, 2, 5, 3, 9, 6, 4, 8, 10, 12, 14] was taken 0 times.
 Path.7: [0, 1, 7, 2, 5, 3, 9, 6, 4, 10, 12, 14] was taken 0 times.
 Path.8: [0, 1, 7, 2, 5, 3, 9, 6, 4, 12, 14] was taken 0 times.
 ... (and so on)
 Path.657: [0, 1, 11, 8, 10, 12, 6, 4, 9, 5, 3, 7, 14] was taken 0 times.
 Path.658: [0, 1, 11, 8, 10, 12, 6, 4, 9, 5, 3, 13, 14] was taken 0 times.
 Path.659: [0, 1, 11, 8, 10, 12, 6, 4, 9, 5, 3, 14] was taken 0 times.
 Path.660: [0, 1, 11, 8, 10, 12, 6, 4, 9, 5, 7, 14] was taken 0 times.
 Path.661: [0, 1, 11, 8, 10, 12, 14] was taken 4 times.

3. Random_Vs_Random (9 transitions)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with the number of Epochs = 0.
 2. The system has 50% desirable states and 50% undesirable states.
 3. The system has up to 9 transitions for each state.
- **Transition happens based on transition probabilities (DTMC was solved)**

Total number of game simulations = 10000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 21.4514
Good Team wins 3326 times with 33.26 %
Bad Team wins 5608 times with 56.08 %
End with Draw 1066 times with 10.66 %
Safety result: The system is unsafe
Path Records: Was not recorded.

4. Blue_Vs_Red (9 transitions)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000.
 2. The system has 50% desirable states and 50% undesirable states.
 3. The system has up to 9 transitions for each state.
- **Transition happens based on transfer probabilities (DTMC was solved)**
Total number of game simulations = 10000
Total number of system states = 16
Maximum Episodic length: 25
Average Episodic length: 21.4338
Good Team wins 3211 times with 32.11 %
Bad Team wins 5683 times with 56.83 %
End with Draw 1106 times with 11.06 %
Safety result: The system is unsafe
Path Records: Was not recorded.

5. Random_Vs_Random (12 transitions)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with a number of Epochs = 100000.
 2. The system has 50% desirable states and 50% undesirable states.
 3. The system has up to 12 transitions for each state.
- **Transition happens based on transition probabilities (DTMC was solved)**
Total number of game simulations = 1000
Total number of system states = 16
Maximum Episodic length: 25
Average Episodic length: 21.239
Good Team wins 253 times with 25.3 %
Bad Team wins 619 times with 61.9 %
End with Draw 128 times with 12.8 %
Safety result: The system is unsafe
Path Records: Was not recorded.

6. Blue_Vs_Red (12 transitions)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000.
 2. The system has 50% desirable states and 50% undesirable states.
 3. The system has up to 12 transitions for each state.
- **Transition happens based on transition probabilities (DTMC was solved)**
Total number of game simulations = 100000
Total number of system states = 16

Maximum Episodic length: 25
Average Episodic length: 21.413
Good Team wins 249 times with 24.9 %
Bad Team wins 629 times with 62.9 %
End with Draw 122 times with 12.2 %
Safety result: The system is unsafe
Path Records: Was not recorded.

C.6 Experiment 6

1. Random:250_Vs_Random:500

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
 2. System has 50% desirable states and 50% undesirable states
- **Transition happens based on transition probabilities (DTMC was solved)**

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 23.35036

agent_red was given 500 as initial resources

agent_blue was given 250 as initial resources

Good Team wins 35307 times with 35.307 % Bad Team wins 53605 times with 53.605 %

End with Draw 11088 times with 11.088 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 264 times.

Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 29 times.

Path.2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 334 times.

Path.3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 94 times.

Path.4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 14 times.

Path.5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 751 times.

Path.6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 754 times.

Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 111 times.

Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 1606 times.

...(and so on)

Path.68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 24 times.

Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 55 times.

Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 109 times.

Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 27 times.

Path.72: [0, 1, 13, 6, 4, 14] was taken 829 times.

2. Blue:250_Vs_Red:500

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 20.60449

agent_red was given 500 as initial resources

agent_blue was given 250 as initial resources

Good Team wins 1787 times with 1.787 %

Bad Team wins 90356 times with 90.356 %
 End with Draw 7857 times with 7.857 %
 Safety result: The system is unsafe
 Paths Records: Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 0 times.
 Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 0 times.
 Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 0 times.
 Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 0 times.
 Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 0 times.
 Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 0 times.
 Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 0 times.
 Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 0 times.
 Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 0 times.
 ...(and so on)
 Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 0 times.
 Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 0 times.
 Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 0 times.
 Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 0 times.
 Path_72: [0, 1, 13, 6, 4, 14] was taken 0 times.

3. Random:250_Vs_Random:750

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
 2. System has 50% desirable states and 50% undesirable states
- **Transition happens based on transition probabilities (DTMC was solved)**

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 23.28785

agent_red was given 750 as initial resources

agent_blue was given 250 as initial resources

Good Team wins 33951 times with 33.951 %

Bad Team wins 53956 times with 53.956 %

End with Draw 12093 times with 12.093 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 268 times.
 Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 29 times.
 Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 340 times.
 Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 108 times.
 Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 11 times.
 Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 815 times.
 Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 855 times.
 Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 110 times.
 Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 1776 times.
 ...(and so on)
 Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 22 times.
 Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 59 times.
 Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 124 times.
 Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 26 times.
 Path_72: [0, 1, 13, 6, 4, 14] was taken 958 times.

4. Blue:250_Vs_Red:750

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 23.05749

agent_red was given 750 as initial resources

agent_blue was given 250 as initial resources

Good Team wins 62061 times with 62.061 %

Bad Team wins 37939 times with 37.939 %

End with Draw 0 times with 0.0 %

Safety result: System is safe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 0 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 0 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 0 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 0 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 0 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 0 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 0 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 0 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 0 times.

...(and so on)

Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 0 times.

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 0 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 0 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 0 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 0 times.

5. Random:500_Vs_Random:250

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Transition happens based on transition probabilities (DTMC was solved)

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 24.63711

agent_red was given 250 as initial resources

agent_blue was given 500 as initial resources

Good Team wins 41166 times with 41.166 %

Bad Team wins 48665 times with 48.665 %

End with Draw 10169 times with 10.169 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 86 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 3 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 173 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 28 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 2 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 460 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 377 times.

Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 28 times.
Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 999 times.
...(and so on)
Path.68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 6 times.
Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 15 times.
Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 45 times.
Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 6 times.
Path.72: [0, 1, 13, 6, 4, 14] was taken 427 times.

6. Blue:500_Vs_Red:250

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 21.76811

agent_red was given 250 as initial resources

agent_blue was given 500 as initial resources

Good Team wins 39418 times with 39.418 %

Bad Team wins 58697 times with 58.697 %

End with Draw 1885 times with 1.885 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 0 times.
Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 0 times.
Path.2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 0 times.
Path.3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 0 times.
Path.4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 0 times.
Path.5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 0 times.
Path.6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 0 times.
Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 0 times.
Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 0 times.
...(and so on)
Path.68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 0 times.
Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 0 times.
Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 0 times.
Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 0 times.
Path.72: [0, 1, 13, 6, 4, 14] was taken 0 times.

7. Random:750_Vs_Random:250

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
 2. System has 50% desirable states and 50% undesirable states
- **Transition happens based on transition probabilities (DTMC was solved)**
- Total number of game simulations = 100000
- Maximum Episodic length: 25
- Average Episodic length: 24.56483
- Good Team wins 39386 times with 39.386 %
- Bad Team wins 49466 times with 49.466 %
- End with Draw 11148 times with 11.148 %
- Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 120 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 11 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 211 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 43 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 3 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 556 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 468 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 59 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 1076 times.

...(and so on)

Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 11 times.

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 30 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 56 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 13 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 486 times.

8. Blue:750_Vs_Red:250

Experiment Summary: Blue:750_Vs_Red:250

Description:

1. Agents were trained for a number of Episodes = 100000

2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 17.15057

agent_red was given 250 as initial resources

agent_blue was given 750 as initial resources

Good Team wins 0 times with 0.0 %

Bad Team wins 98735 times with 98.735 %

End with Draw 1265 times with 1.265 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 0 times.

Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 0 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 0 times.

Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 0 times.

Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 0 times.

Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 0 times.

Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 0 times.

Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 0 times.

Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 0 times.

...(and so on)

Path_68: [0, 1, 13, 6, 4, 8, 7, 5, 9, 11, 14] was taken 0 times.

Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 0 times.

Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 0 times.

Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 0 times.

Path_72: [0, 1, 13, 6, 4, 14] was taken 0 times.

C.7 Experiment 7

1. Random:_Vs_Random (Threshold = 20)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with the number of Epochs = 0
2. System has 50% desirable states and 50% undesirable states
3. Episode is terminated when undesirableness threshold is reached

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 17.17751

Undesirableness threshold = 20

Average Undesirableness = 7.53452

Good Team wins 24314 times with 24.314 %

Bad Team wins 71845 times with 71.845 %

End with Draw 3841 times with 3.841 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 384 times.

Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 45 times.

Path.2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 477 times.

Path.3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 153 times.

Path.4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 20 times.

Path.5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1098 times.

Path.6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1280 times.

Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 158 times.

Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2518 times.

...(and so on)

Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 88 times.

Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 148 times.

Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 38 times.

Path.72: [0, 1, 13, 6, 4, 14] was taken 1228 times.

2. Blue:_Vs_Red (Threshold = 20)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
2. System has 50% desirable states and 50% undesirable states
3. Episode is terminated when undesirableness threshold is reached

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 16.96143

Undesirableness threshold = 20

Average Undesirableness = 4.74937

Good Team wins 25481 times with 25.481 %

Bad Team wins 72315 times with 72.315 %

End with Draw 2204 times with 2.204 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 304 times.

Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 41 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 432 times.
 Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 136 times.
 Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 15 times.
 Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 962 times.
 Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1150 times.
 Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 164 times.
 Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2134 times.
 ...(and so on)
 Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 70 times.
 Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 141 times.
 Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 27 times.
 Path_72: [0, 1, 13, 6, 4, 14] was taken 1082 times.

3. Random:_Vs_Random (Threshold = 40)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with the number of Epochs = 0
2. System has 50% desirable states and 50% undesirable states
3. Episode is terminated when undesirableness threshold is reached

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 21.48729

Undesirableness threshold = 40

Average Undesirableness = 10.7254

Good Team wins 36330 times with 36.33 %

Bad Team wins 56572 times with 56.572 %

End with Draw 7098 times with 7.098 %

Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 497 times.
 Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 48 times.
 Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 540 times.
 Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 171 times.
 Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 32 times.
 Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1292 times.
 Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1460 times.
 Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 176 times.
 Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2666 times.
 ...(and so on)
 Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 103 times.
 Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 188 times.
 Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 44 times.
 Path_72: [0, 1, 13, 6, 4, 14] was taken 1332 times.

4. Blue:_Vs_Red (Threshold = 40)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with the number of Epochs = 100000
2. System has 50% desirable states and 50% undesirable states
3. Episode is terminated when undesirableness threshold is reached

Total number of game simulations = 100000

Total number of system states = 16
 Maximum Episodic length: 25
 Average Episodic length: 21.50506
 Undesirableness threshold = 40
 Average Undesirableness = 10.71736
 Good Team wins 36232 times with 36.232 %
 Bad Team wins 56495 times with 56.495 %
 End with Draw 7273 times with 7.273 %
 Safety result: The system is unsafe
 Paths Records:
 Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 465 times.
 Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 50 times.
 Path.2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 566 times.
 Path.3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 188 times.
 Path.4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 20 times.
 Path.5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1258 times.
 Path.6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1455 times.
 Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 204 times.
 Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2822 times.
 ...(and so on)
 Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 92 times.
 Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 186 times.
 Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 41 times.
 Path.72: [0, 1, 13, 6, 4, 14] was taken 1315 times.

5. Random:_Vs_Random (Threshold = 60)

Experiment Summary: Random_Vs_Random
 Description:
 1. Agents were trained with the number of Epochs = 0
 2. System has 50% desirable states and 50% undesirable states
 3. Episode is terminated when undesirableness threshold is reached
 Total number of game simulations = 100000
 Total number of system states = 16
 Maximum Episodic length: 25
 Average Episodic length: 22.41932
 Undesirableness threshold = 60
 Average Undesirableness = 11.44842
 Good Team wins 40392 times with 40.392 %
 Bad Team wins 51066 times with 51.066 %
 End with Draw 8542 times with 8.542 %
 Safety result: The system is unsafe
 Paths Records:
 Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 484 times.
 Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 58 times.
 Path.2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 556 times.
 Path.3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 193 times.
 Path.4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 24 times.
 Path.5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1280 times.
 Path.6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1491 times.
 Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 182 times.
 Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2809 times.
 ...(and so on)
 Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 84 times.
 Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 187 times.
 Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 52 times.

Path.72: [0, 1, 13, 6, 4, 14] was taken 1407 times.

6. Blue:_Vs_Red (Threshold = 60)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
 2. System has 50% desirable states and 50% undesirable states
 3. Episode is terminated when undesirableness threshold is reached
- Total number of game simulations = 100000
Total number of system states = 16
Maximum Episodic length: 25
Average Episodic length: 22.38612
Undesirableness threshold = 60
Average Undesirableness = 11.48853 Good Team wins 40357 times with 40.357 %
Bad Team wins 51056 times with 51.056 %
End with Draw 8587 times with 8.587 %
Safety result: The system is unsafe

Paths Records:

- Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 445 times.
Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 47 times.
Path.2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 536 times.
Path.3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 203 times.
Path.4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 23 times.
Path.5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1326 times.
Path.6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1545 times.
Path.7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 190 times.
Path.8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2832 times.
...(and so on)
Path.69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 102 times.
Path.70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 190 times.
Path.71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 50 times.
Path.72: [0, 1, 13, 6, 4, 14] was taken 1366 times.

7. Random:_Vs_Random (No Threshold Condition)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with the number of Epochs = 0
 2. System has 50% desirable states and 50% undesirable states
 3. Episode is not terminated when undesirableness threshold is reached
- Total number of game simulations = 100000
Total number of system states = 16
Maximum Episodic length: 25
Average Episodic length: 22.51163
Undesirableness threshold = ∞
Average Undesirableness = 11.54126
Good Team wins 26961 times with 26.961 %
Bad Team wins 69417 times with 69.417 %
End with Draw 3622 times with 3.622 %
Safety result: The system is unsafe
- Paths Records:
- Path.0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 444 times.
Path.1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 60 times.

Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 556 times.
 Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 202 times.
 Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 24 times.
 Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1250 times.
 Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1493 times.
 Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 193 times.
 Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2723 times.
 ...(and so on)
 Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 84 times.
 Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 195 times.
 Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 43 times.
 Path_72: [0, 1, 13, 6, 4, 14] was taken 1332 times.

8. Blue:_Vs_Red (No Threshold Condition)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
 2. System has 50% desirable states and 50% undesirable states
 3. Episode is not terminated when undesirableness threshold is reached
- Total number of game simulations = 100000
 Total number of system states = 16
 Maximum Episodic length: 25
 Average Episodic length: 22.45893
 Undesirableness threshold = ∞
 Average Undesirableness = 11.50504
 Good Team wins 27161 times with 27.161 %
 Bad Team wins 69149 times with 69.149 %
 End with Draw 3690 times with 3.69 %
 Safety result: The system is unsafe

Paths Records:

Path_0: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 10, 11, 14] was taken 443 times.
 Path_1: [0, 1, 6, 3, 7, 5, 8, 12, 13, 2, 11, 14] was taken 58 times.
 Path_2: [0, 1, 6, 3, 7, 5, 9, 11, 14] was taken 587 times.
 Path_3: [0, 1, 6, 3, 7, 5, 9, 13, 2, 10, 11, 14] was taken 179 times.
 Path_4: [0, 1, 6, 3, 7, 5, 9, 13, 2, 11, 14] was taken 23 times.
 Path_5: [0, 1, 6, 3, 7, 5, 9, 14] was taken 1278 times.
 Path_6: [0, 1, 6, 3, 7, 5, 13, 2, 10, 11, 14] was taken 1462 times.
 Path_7: [0, 1, 6, 3, 7, 5, 13, 2, 11, 14] was taken 201 times.
 Path_8: [0, 1, 6, 3, 7, 10, 11, 14] was taken 2802 times.
 ...(and so on)
 Path_69: [0, 1, 13, 6, 4, 8, 7, 5, 9, 14] was taken 101 times.
 Path_70: [0, 1, 13, 6, 4, 8, 7, 10, 11, 14] was taken 181 times.
 Path_71: [0, 1, 13, 6, 4, 8, 7, 11, 14] was taken 49 times.
 Path_72: [0, 1, 13, 6, 4, 14] was taken 1369 times.

C.8 Experiment 8

1. Random:_Vs_Random (Red as the first Move)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with the number of Epochs = 0
2. System has 50% desirable states and 50% undesirable states

****Transition happens based on transition probabilities (DTMC was solved)****

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 22.7559

Undesirableness threshold = 10

Average Undesirableness = -44.61332

Good Team wins 35108 times with 35.108 %

End with Draw 5476 times with 5.476 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 194 times.

Path.1: [0, 1, 2, 6, 9, 12, 11, 14] was taken 6009 times.

Path.2: [0, 1, 2, 6, 12, 11, 14] was taken 7655 times.

Path.3: [0, 1, 2, 11, 14] was taken 49139 times.

Path.4: [0, 1, 6, 8, 13, 5, 4, 9, 12, 2, 11, 14] was taken 39 times.

Path.5: [0, 1, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 47 times.

Path.6: [0, 1, 6, 9, 12, 2, 11, 14] was taken 943 times.

Path.7: [0, 1, 6, 9, 12, 11, 14] was taken 1628 times.

Path.8: [0, 1, 6, 12, 2, 11, 14] was taken 1199 times.

Path.9: [0, 1, 6, 12, 11, 14] was taken 2136 times.

Path.10: [0, 1, 9, 12, 2, 11, 14] was taken 487 times.

Path.11: [0, 1, 9, 12, 11, 14] was taken 1031 times.

2. Blue:_Vs_Red (Red as the first Move)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000

2. System has 50% desirable states and 50% undesirable states

****Transition happens based on transition probabilities (DTMC was solved)****

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 22.74751

Undesirableness threshold = 10

Average Undesirableness = -44.55197

Good Team wins 34928 times with 34.928 %

Bad Team wins 59590 times with 59.59 %

End with Draw 5482 times with 5.482 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 186 times.

Path.1: [0, 1, 2, 6, 9, 12, 11, 14] was taken 6064 times.

Path.2: [0, 1, 2, 6, 12, 11, 14] was taken 7513 times.

Path.3: [0, 1, 2, 11, 14] was taken 49087 times.

Path.4: [0, 1, 6, 8, 13, 5, 4, 9, 12, 2, 11, 14] was taken 39 times.

Path.5: [0, 1, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 43 times.

Path.6: [0, 1, 6, 9, 12, 2, 11, 14] was taken 965 times.

Path.7: [0, 1, 6, 9, 12, 11, 14] was taken 1633 times.

Path.8: [0, 1, 6, 12, 2, 11, 14] was taken 1101 times.

Path.9: [0, 1, 6, 12, 11, 14] was taken 2094 times.

Path.10: [0, 1, 9, 12, 2, 11, 14] was taken 497 times.

Path.11: [0, 1, 9, 12, 11, 14] was taken 1064 times.

3. Random:_Vs_Random (Blue as the first Move)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained with the number of Epochs = 0
2. System has 50% desirable states and 50% undesirable states
3. Episode is not terminated when undesirableness threshold is reached

Total number of game simulations = 100000

Total number of system states = 16

Maximum Episodic length: 25

Average Episodic length: 23.96904

Undesirableness threshold = 10

Average Undesirableness = -42.83024

Good Team wins 31248 times with 31.248 %

Bad Team wins 63274 times with 63.274 %

End with Draw 5478 times with 5.478 %

Safety result: The system is unsafe Paths Records:

Path.0: [0, 1, 2, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 92 times.

Path.1: [0, 1, 2, 6, 9, 12, 11, 14] was taken 3653 times.

Path.2: [0, 1, 2, 6, 12, 11, 14] was taken 4847 times.

Path.3: [0, 1, 2, 11, 14] was taken 33256 times.

Path.4: [0, 1, 6, 8, 13, 5, 4, 9, 12, 2, 11, 14] was taken 16 times.

Path.5: [0, 1, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 36 times.

Path.6: [0, 1, 6, 9, 12, 2, 11, 14] was taken 537 times.

Path.7: [0, 1, 6, 9, 12, 11, 14] was taken 1074 times.

Path.8: [0, 1, 6, 12, 2, 11, 14] was taken 698 times.

Path.9: [0, 1, 6, 12, 11, 14] was taken 1411 times.

Path.10: [0, 1, 9, 12, 2, 11, 14] was taken 302 times.

Path.11: [0, 1, 9, 12, 11, 14] was taken 666 times.

4. Blue:_Vs_Red (Blue as the first Move)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
2. System has 50% desirable states and 50% undesirable states
3. Episode is not terminated when undesirableness threshold is reached

Total number of game simulations = 100000

Total number of system states = 16 Maximum Episodic length: 25

Average Episodic length: 10.41316

Undesirableness threshold = 10

Average Undesirableness = 1.03212

Good Team wins 2201 times with 2.201 %

Bad Team wins 97797 times with 97.797 %

End with Draw 2 times with 0.002 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 0 times.

Path.1: [0, 1, 2, 6, 9, 12, 11, 14] was taken 0 times.

Path.2: [0, 1, 2, 6, 12, 11, 14] was taken 0 times.

Path.3: [0, 1, 2, 11, 14] was taken 1141 times.

Path.4: [0, 1, 6, 8, 13, 5, 4, 9, 12, 2, 11, 14] was taken 0 times.

Path.5: [0, 1, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 0 times.

Path.6: [0, 1, 6, 9, 12, 2, 11, 14] was taken 0 times.

Path.7: [0, 1, 6, 9, 12, 11, 14] was taken 0 times.

Path.8: [0, 1, 6, 12, 2, 11, 14] was taken 0 times.

Path.9: [0, 1, 6, 12, 11, 14] was taken 7 times.
Path.10: [0, 1, 9, 12, 2, 11, 14] was taken 0 times.
Path.11: [0, 1, 9, 12, 11, 14] was taken 0 times.

C.9 Experiment 9

1. Random:_Vs_Random (Initial System)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 0
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 21.86931

agent_red was given 1000 as initial resources

agent_blue was given 1000 as initial resources

Good Team wins 39882 times with 39.882 %

Bad Team wins 56726 times with 56.726 %

End with Draw 3392 times with 3.392 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 21982 times.

Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 9472 times.

Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 3137 times.

Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1349 times.

Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 331 times.

Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 147 times.

Path.6: [0, 1, 11, 14] was taken 74302 times.

2. Blue:_Vs_Red (Initial System)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 21.82374

agent_red was given 1000 as initial resources

agent_blue was given 1000 as initial resources

Good Team wins 40706 times with 40.706 %

Bad Team wins 55224 times with 55.224 %

End with Draw 4070 times with 4.07 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 22099 times.

Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 9658 times.

Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 3278 times.

Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1442 times.

Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 350 times.

Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 158 times.

Path.6: [0, 1, 11, 14] was taken 76168 times.

3. Random:_Vs_Random (Adding Transitions)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 0
2. System has 50% desirable states and 50% undesirable states
3. Transitions 1 to 7, and 7 to 11 were added

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 20.69485

agent_red was given 1000 as initial resources

agent_blue was given 1000 as initial resources

Good Team wins 35557 times with 35.557 %

Bad Team wins 59695 times with 59.695 %

End with Draw 4748 times with 4.748 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 13776 times.

Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 5512 times.

Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 4007 times.

Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1643 times.

Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 394 times.

Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 150 times.

Path.6: [0, 1, 7, 11, 14] was taken 40055 times.

Path.7: [0, 1, 7, 14] was taken 38813 times.

Path.8: [0, 1, 11, 14] was taken 92812 times.

4. Blue:_Vs_Red (Adding Transitions)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states
3. Transitions 1 to 7, and 7 to 11 were added

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 20.70762

agent_red was given 1000 as initial resources

agent_blue was given 1000 as initial resources

Good Team wins 35614 times with 35.614 %

Bad Team wins 59671 times with 59.671 %

End with Draw 4715 times with 4.715 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 13546 times.

Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 5475 times.

Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 3938 times.

Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1630 times.

Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 356 times.

Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 149 times.

Path.6: [0, 1, 7, 11, 14] was taken 40206 times.

Path.7: [0, 1, 7, 14] was taken 38908 times.

Path.8: [0, 1, 11, 14] was taken 92831 times.

5. Random:_Vs_Random (Removing Transitions)

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states
3. Transitions 1 to 12 , and 13 to 12 were removed

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 21.85024

agent_red was given 1000 as initial resources

agent_blue was given 1000 as initial resources

Good Team wins 40818 times with 40.818 %

Bad Team wins 55181 times with 55.181 %

End with Draw 4001 times with 4.001 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 22704 times.

Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 9709 times.

Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 3164 times.

Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1446 times.

Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 313 times.

Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 147 times.

Path.6: [0, 1, 11, 14] was taken 76620 times.

6. Blue:_Vs_Red (Removing Transitions)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states
3. Transitions 3 to 11, and 4 to 10 were removed

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 21.82065

agent_red was given 1000 as initial resources

agent_blue was given 1000 as initial resources

Good Team wins 40622 times with 40.622 %

Bad Team wins 55375 times with 55.375 %

End with Draw 4003 times with 4.003 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 22174 times.

Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 9684 times.

Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 3218 times.

Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1359 times.

Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 338 times.

Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 156 times.

Path.6: [0, 1, 11, 14] was taken 76673 times.

C.10 Experiment 10

1. Blue:_Vs_Red (Blue as the first Move)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100,000
 2. System has 50% desirable states and 50% undesirable states
- Total number of game simulations = 100000
Total number of system states = 16
Maximum Episodic length: 25
Average Episodic length: 22.80184
Undesirableness threshold = 10
Average Undesirableness = -44.68522
Good Team wins 35424 times with 35.424 %
Bad Team wins 59091 times with 59.091 %
End with Draw 5485 times with 5.485 %
Safety result: The system is unsafe

Paths Records:

- Path.0: [0, 1, 2, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 190 times.
Path.1: [0, 1, 2, 6, 9, 12, 11, 14] was taken 6071 times.
Path.2: [0, 1, 2, 6, 12, 11, 14] was taken 7683 times.
Path.3: [0, 1, 2, 11, 14] was taken 49106 times.
Path.4: [0, 1, 6, 8, 13, 5, 4, 9, 12, 2, 11, 14] was taken 25 times.
Path.5: [0, 1, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 46 times.
Path.6: [0, 1, 6, 9, 12, 2, 11, 14] was taken 974 times.
Path.7: [0, 1, 6, 9, 12, 11, 14] was taken 1660 times.
Path.8: [0, 1, 6, 12, 2, 11, 14] was taken 1139 times.
Path.9: [0, 1, 6, 12, 11, 14] was taken 2137 times.
Path.10: [0, 1, 9, 12, 2, 11, 14] was taken 484 times.
Path.11: [0, 1, 9, 12, 11, 14] was taken 1038 times.

2. Blue:_Vs_Red (Red has Initial Move)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained with a number of Epochs = 100000
 2. System has 50% desirable states and 50% undesirable states
- Total number of game simulations = 100000
Total number of system states = 16
Maximum Episodic length: 25
Average Episodic length: 11.02889
Undesirableness threshold = 10
Average Undesirableness = -4.40373
Good Team wins 2895 times with 2.895 %
Bad Team wins 97103 times with 97.103 %
End with Draw 2 times with 0.002 %
Safety result: The system is unsafe

Paths Records:

- Path.0: [0, 1, 2, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 0 times.
Path.1: [0, 1, 2, 6, 9, 12, 11, 14] was taken 0 times.
Path.2: [0, 1, 2, 6, 12, 11, 14] was taken 0 times.
Path.3: [0, 1, 2, 11, 14] was taken 1223 times.
Path.4: [0, 1, 6, 8, 13, 5, 4, 9, 12, 2, 11, 14] was taken 0 times.
Path.5: [0, 1, 6, 8, 13, 5, 4, 9, 12, 11, 14] was taken 0 times.

Path.6: [0, 1, 6, 9, 12, 2, 11, 14] was taken 0 times.
Path.7: [0, 1, 6, 9, 12, 11, 14] was taken 0 times.
Path.8: [0, 1, 6, 12, 2, 11, 14] was taken 0 times.
Path.9: [0, 1, 6, 12, 11, 14] was taken 0 times.
Path.10: [0, 1, 9, 12, 2, 11, 14] was taken 0 times.
Path.11: [0, 1, 9, 12, 11, 14] was taken 0 times.

C.11 Path Elimination: Experiment 1

1. Random:_Vs_Random

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
 2. System has 50% desirable states and 50% undesirable states
- Total number of game simulations = 100000
Maximum Episodic length: 50
Average Episodic length: 28.40645
agent_blue was given 1000 as initial resources
agent_red was given 1000 as initial resources
Good Team wins 46780 times with 46.78 %
Bad Team wins 50760 times with 50.76 %
End with Draw 2460 times with 2.46 %
Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 24355 times.
Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 10769 times.
Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 3531 times.
Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1647 times.
Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 393 times.
Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 166 times.
Path.6: [0, 1, 11, 14] was taken 83195 times.

2. Blue:_Vs_Red (Without Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
 2. System has 50% desirable states and 50% undesirable states
- Total number of game simulations = 100000
Maximum Episodic length: 50
Average Episodic length: 36.54259
agent_blue was given 1000 as initial resources
agent_red was given 1000 as initial resources
Good Team wins 23312 times with 23.312 %
Bad Team wins 76007 times with 76.007 %
End with Draw 681 times with 0.681 % Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 20858 times.
Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 9161 times.
Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 3012 times.
Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 1406 times.
Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 346 times.
Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 138 times.

Path.6: [0, 1, 11, 14] was taken 70893 times.

3. Blue:_Vs_Red (With Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for several Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000
Maximum Episodic length: 50
Average Episodic length: 25.23054
agent_blue was given 199.9036233044353 as the initial resources
agent_red was given 109.36283143000391 as initial resources
Good Team wins 39746 times with 39.746 %
Bad Team wins 53489 times with 53.489 %
End with Draw 6765 times with 6.765 %
Safety result: The system is unsafe
Paths Records: Path.0: [0, 1, 2, 5, 3, 11, 14] was taken 9177 times.
Path.1: [0, 1, 2, 12, 5, 3, 11, 14] was taken 2683 times.
Path.2: [0, 1, 6, 13, 2, 5, 3, 11, 14] was taken 1 times.
Path.3: [0, 1, 6, 13, 2, 12, 5, 3, 11, 14] was taken 0 times.
Path.4: [0, 1, 6, 13, 8, 2, 5, 3, 11, 14] was taken 0 times.
Path.5: [0, 1, 6, 13, 8, 2, 12, 5, 3, 11, 14] was taken 0 times.
Path.6: [0, 1, 11, 14] was taken 31720 times.

C.12 Path Elimination: Experiment 2

1. Random:_Vs_Random

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000
Maximum Episodic length: 25
Average Episodic length: 22.2833
agent_blue was given 1000 as initial resources
agent_red was given 1000 as initial resources
Good Team wins 32236 times with 32.236 %
Bad Team wins 57894 times with 57.894 %
End with Draw 9870 times with 9.87 %
Safety result: System is unsafe
Paths Records:
Path.0: [0, 1, 4, 8, 7, 5, 6, 10, 13, 11, 14] was taken 31 times.
Path.1: [0, 1, 4, 8, 7, 5, 6, 10, 13, 14] was taken 173 times.
Path.2: [0, 1, 4, 8, 7, 5, 6, 10, 14] was taken 478 times.
Path.3: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 13, 11, 14] was taken 26 times.
Path.4: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 13, 14] was taken 75 times.
Path.5: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 14] was taken 116 times.
Path.6: [0, 1, 4, 8, 7, 5, 12, 13, 2, 6, 10, 14] was taken 160 times.
...(and so on)
Path.127: [0, 1, 6, 10, 2, 7, 5, 12, 13, 11, 14] was taken 8 times.
Path.128: [0, 1, 6, 10, 2, 7, 5, 12, 13, 14] was taken 36 times.
Path.129: [0, 1, 6, 10, 13, 2, 7, 5, 8, 9, 11, 14] was taken 10 times.

Path.130: [0, 1, 6, 10, 13, 11, 14] was taken 103 times.
Path.131: [0, 1, 6, 10, 13, 14] was taken 7802 times.
Path.132: [0, 1, 6, 10, 14] was taken 20051 times.

2. Blue:_Vs_Red (Without Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 22.59781

agent_blue was given 1000 as initial resources

agent_red was given 1000 as initial resources

Good Team wins 30646 times with 30.646 %

Bad Team wins 58069 times with 58.069 %

End with Draw 11285 times with 11.285 %

Safety result: System is unsafe

Paths Records:

Path.0: [0, 1, 4, 8, 7, 5, 6, 10, 13, 11, 14] was taken 26 times.

Path.1: [0, 1, 4, 8, 7, 5, 6, 10, 13, 14] was taken 205 times.

Path.2: [0, 1, 4, 8, 7, 5, 6, 10, 14] was taken 481 times.

Path.3: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 13, 11, 14] was taken 22 times.

Path.4: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 13, 14] was taken 57 times.

Path.5: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 14] was taken 102 times.

Path.6: [0, 1, 4, 8, 7, 5, 12, 13, 2, 6, 10, 14] was taken 166 times.

Path.7: [0, 1, 4, 8, 7, 5, 12, 13, 11, 14] was taken 16 times.

Path.8: [0, 1, 4, 8, 7, 5, 12, 13, 14] was taken 380 times.

...(and so on)

Path.127: [0, 1, 6, 10, 2, 7, 5, 12, 13, 11, 14] was taken 6 times.

Path.128: [0, 1, 6, 10, 2, 7, 5, 12, 13, 14] was taken 37 times.

Path.129: [0, 1, 6, 10, 13, 2, 7, 5, 8, 9, 11, 14] was taken 6 times.

Path.130: [0, 1, 6, 10, 13, 11, 14] was taken 110 times.

Path.131: [0, 1, 6, 10, 13, 14] was taken 8111 times.

Path.132: [0, 1, 6, 10, 14] was taken 20408 times.

3. Blue:_Vs_Red (With Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 12.10331

agent_blue was given 108.45693956914083 as initial resources

agent_red was given 93.06613031266453 as initial resources

Good Team wins 34370 times with 34.37 %

Bad Team wins 59552 times with 59.552 %

End with Draw 6078 times with 6.078 %

Safety result: System is unsafe

Paths Records:

Path.0: [0, 1, 4, 8, 7, 5, 6, 10, 13, 11, 14] was taken 0 times.

Path_1: [0, 1, 4, 8, 7, 5, 6, 10, 13, 14] was taken 0 times.
 Path_2: [0, 1, 4, 8, 7, 5, 6, 10, 14] was taken 0 times.
 Path_3: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 13, 11, 14] was taken 0 times.
 Path_4: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 13, 14] was taken 0 times.
 Path_5: [0, 1, 4, 8, 7, 5, 12, 2, 6, 10, 14] was taken 0 times.
 Path_6: [0, 1, 4, 8, 7, 5, 12, 13, 2, 6, 10, 14] was taken 0 times.
 ...(and so on)
 Path_127: [0, 1, 6, 10, 2, 7, 5, 12, 13, 11, 14] was taken 0 times.
 Path_128: [0, 1, 6, 10, 2, 7, 5, 12, 13, 14] was taken 0 times.
 Path_129: [0, 1, 6, 10, 13, 2, 7, 5, 8, 9, 11, 14] was taken 0 times.
 Path_130: [0, 1, 6, 10, 13, 11, 14] was taken 42 times.
 Path_131: [0, 1, 6, 10, 13, 14] was taken 3184 times.
 Path_132: [0, 1, 6, 10, 14] was taken 8380 times.

C.13 Path Elimination: Experiment 3

1. Random:_Vs_Random

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 16.73857

agent_blue was given 1000 as initial resources

agent_red was given 1000 as initial resources

Good Team wins 16932 times with 16.932 %

Bad Team wins 71273 times with 71.273 %

End with Draw 11795 times with 11.795 %

Safety result: System is unsafe

Paths Records:

Path_0: [0, 1, 2, 4, 3, 6, 7, 14] was taken 371 times.
 Path_1: [0, 1, 2, 4, 3, 6, 12, 5, 7, 14] was taken 260 times.
 Path_2: [0, 1, 2, 4, 3, 6, 12, 5, 14] was taken 208 times.
 Path_3: [0, 1, 2, 4, 3, 10, 14] was taken 751 times.
 Path_4: [0, 1, 2, 4, 13, 5, 6, 7, 14] was taken 1303 times.
 Path_5: [0, 1, 2, 4, 13, 5, 7, 14] was taken 3275 times.
 Path_6: [0, 1, 2, 4, 13, 5, 14] was taken 4039 times.
 Path_7: [0, 1, 2, 4, 13, 10, 14] was taken 31822 times.
 Path_8: [0, 1, 2, 14] was taken 237477 times.
 Path_9: [0, 1, 5, 6, 7, 2, 4, 3, 10, 14] was taken 33 times.
 Path_10: [0, 1, 5, 6, 7, 2, 4, 13, 10, 14] was taken 1835 times.
 Path_11: [0, 1, 5, 6, 7, 2, 14] was taken 11070 times.
 Path_12: [0, 1, 5, 6, 7, 14] was taken 20385 times.
 Path_13: [0, 1, 5, 7, 2, 4, 3, 10, 14] was taken 110 times.
 Path_14: [0, 1, 5, 7, 2, 4, 13, 10, 14] was taken 4399 times.
 Path_15: [0, 1, 5, 7, 2, 14] was taken 26839 times.
 Path_16: [0, 1, 5, 7, 14] was taken 48931 times.
 Path_17: [0, 1, 5, 14] was taken 96926 times.

2. Blue:_Vs_Red (Without Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 16.81103

agent_blue was given 1000 as initial resources

agent_red was given 1000 as initial resources

Good Team wins 16821 times with 16.821 %

Bad Team wins 70778 times with 70.778 %

End with Draw 12401 times with 12.401 %

Safety result: The system is unsafe

Paths Records:

Path.0: [0, 1, 2, 4, 3, 6, 7, 14] was taken 444 times.

Path.1: [0, 1, 2, 4, 3, 6, 12, 5, 7, 14] was taken 226 times.

Path.2: [0, 1, 2, 4, 3, 6, 12, 5, 14] was taken 221 times.

Path.3: [0, 1, 2, 4, 3, 10, 14] was taken 748 times.

Path.4: [0, 1, 2, 4, 13, 5, 6, 7, 14] was taken 1307 times.

Path.5: [0, 1, 2, 4, 13, 5, 7, 14] was taken 3265 times.

Path.6: [0, 1, 2, 4, 13, 5, 14] was taken 4132 times.

Path.7: [0, 1, 2, 4, 13, 10, 14] was taken 32634 times.

Path.8: [0, 1, 2, 14] was taken 239430 times.

Path.9: [0, 1, 5, 6, 7, 2, 4, 3, 10, 14] was taken 45 times.

Path.10: [0, 1, 5, 6, 7, 2, 4, 13, 10, 14] was taken 1844 times.

Path.11: [0, 1, 5, 6, 7, 2, 14] was taken 11270 times.

Path.12: [0, 1, 5, 6, 7, 14] was taken 20764 times.

Path.13: [0, 1, 5, 7, 2, 4, 3, 10, 14] was taken 111 times.

Path.14: [0, 1, 5, 7, 2, 4, 13, 10, 14] was taken 4464 times.

Path.15: [0, 1, 5, 7, 2, 14] was taken 27268 times.

Path.16: [0, 1, 5, 7, 14] was taken 49344 times.

Path.17: [0, 1, 5, 14] was taken 98330 times.

3. Blue:_Vs_Red (With Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 9.82275

agent_blue was given 134.34484561965897 as initial resources

agent_red was given 80.75149990743631 as initial resources

Good Team wins 41777 times with 41.777 %

Bad Team wins 24419 times with 24.419 %

End with Draw 33804 times with 33.804 %

Safety result: System is safe

Paths Records:

Path.0: [0, 1, 2, 4, 3, 6, 7, 14] was taken 78 times.

Path.1: [0, 1, 2, 4, 3, 6, 12, 5, 7, 14] was taken 16 times.

Path.2: [0, 1, 2, 4, 3, 6, 12, 5, 14] was taken 26 times.

Path.3: [0, 1, 2, 4, 3, 10, 14] was taken 149 times.

Path.4: [0, 1, 2, 4, 13, 5, 6, 7, 14] was taken 200 times.

Path_5: [0, 1, 2, 4, 13, 5, 7, 14] was taken 539 times.
Path_6: [0, 1, 2, 4, 13, 5, 14] was taken 719 times.
Path_7: [0, 1, 2, 4, 13, 10, 14] was taken 7480 times.
Path_8: [0, 1, 2, 14] was taken 69211 times.
Path_9: [0, 1, 5, 6, 7, 2, 4, 3, 10, 14] was taken 3 times.
Path_10: [0, 1, 5, 6, 7, 2, 4, 13, 10, 14] was taken 252 times.
Path_11: [0, 1, 5, 6, 7, 2, 14] was taken 2864 times.
Path_12: [0, 1, 5, 6, 7, 14] was taken 5316 times.
Path_13: [0, 1, 5, 7, 2, 4, 3, 10, 14] was taken 7 times.
Path_14: [0, 1, 5, 7, 2, 4, 13, 10, 14] was taken 519 times.
Path_15: [0, 1, 5, 7, 2, 14] was taken 6944 times.
Path_16: [0, 1, 5, 7, 14] was taken 13777 times.
Path_17: [0, 1, 5, 14] was taken 30498 times.

C.14 Path Elimination: Experiment 4

1. Random:_Vs_Random

Experiment Summary: Random_Vs_Random

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 19.14341

agent_blue was given 1000 as initial resources

agent_red was given 1000 as initial resources

Good Team wins 37112 times with 37.112 %

Bad Team wins 57884 times with 57.884 %

End with Draw 5004 times with 5.004 %

Safety result: System is unsafe

Paths Records:

Path_0: [0, 1, 5, 2, 6, 4, 3, 10, 14] was taken 2 times.
Path_1: [0, 1, 5, 2, 6, 4, 14] was taken 17059 times.
Path_2: [0, 1, 5, 2, 8, 12, 7, 3, 4, 14] was taken 423 times.
Path_3: [0, 1, 5, 2, 8, 12, 7, 3, 10, 14] was taken 1 times.
Path_4: [0, 1, 5, 2, 8, 12, 10, 7, 3, 4, 14] was taken 357 times.
Path_5: [0, 1, 5, 2, 8, 12, 10, 14] was taken 825 times.
Path_6: [0, 1, 5, 2, 14] was taken 11235 times.
Path_7: [0, 1, 5, 4, 3, 10, 2, 14] was taken 4 times.
Path_8: [0, 1, 5, 4, 3, 10, 14] was taken 1 times.
Path_9: [0, 1, 5, 4, 14] was taken 153091 times.
Path_10: [0, 1, 5, 10, 2, 6, 4, 14] was taken 3121 times.
Path_11: [0, 1, 5, 10, 2, 8, 12, 7, 3, 4, 14] was taken 138 times.
Path_12: [0, 1, 5, 10, 2, 14] was taken 2151 times.
Path_13: [0, 1, 5, 10, 7, 3, 4, 14] was taken 2012 times.
Path_14: [0, 1, 5, 10, 14] was taken 1798 times.
Path_15: [0, 1, 12, 7, 3, 4, 14] was taken 8275 times.
Path_16: [0, 1, 12, 7, 3, 10, 2, 6, 4, 14] was taken 5 times.
Path_17: [0, 1, 12, 7, 3, 10, 2, 14] was taken 2 times.
Path_18: [0, 1, 12, 7, 3, 10, 14] was taken 5 times.
Path_19: [0, 1, 12, 7, 5, 2, 6, 4, 3, 10, 14] was taken 1 times.
Path_20: [0, 1, 12, 7, 5, 2, 6, 4, 14] was taken 3176 times.
Path_21: [0, 1, 12, 7, 5, 2, 14] was taken 1891 times.
Path_22: [0, 1, 12, 7, 5, 4, 3, 10, 2, 14] was taken 1 times.
Path_23: [0, 1, 12, 7, 5, 4, 3, 10, 14] was taken 3 times.

Path_24: [0, 1, 12, 7, 5, 4, 14] was taken 25731 times.
Path_25: [0, 1, 12, 7, 5, 10, 2, 6, 4, 14] was taken 768 times.
Path_26: [0, 1, 12, 7, 5, 10, 2, 14] was taken 399 times.
Path_27: [0, 1, 12, 7, 5, 10, 14] was taken 269 times.
Path_28: [0, 1, 12, 10, 2, 6, 4, 14] was taken 5146 times.
Path_29: [0, 1, 12, 10, 2, 14] was taken 6741 times.
Path_30: [0, 1, 12, 10, 7, 3, 4, 14] was taken 6283 times.
Path_31: [0, 1, 12, 10, 7, 5, 2, 6, 4, 14] was taken 3117 times.
Path_32: [0, 1, 12, 10, 7, 5, 2, 14] was taken 1716 times.
Path_33: [0, 1, 12, 10, 7, 5, 4, 14] was taken 20147 times.
Path_34: [0, 1, 12, 10, 14] was taken 5647 times.

2. Blue:_Vs_Red (Without Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states

Total number of game simulations = 100000

Maximum Episodic length: 25

Average Episodic length: 19.57889

agent_blue was given 1000 as initial resources

agent_red was given 1000 as initial resources

Good Team wins 62173 times with 62.173 %

Bad Team wins 37145 times with 37.145 %

End with Draw 682 times with 0.682 %

Safety result: System is safe

Paths Records:

Path_0: [0, 1, 5, 2, 6, 4, 3, 10, 14] was taken 0 times.
Path_1: [0, 1, 5, 2, 6, 4, 14] was taken 12761 times.
Path_2: [0, 1, 5, 2, 8, 12, 7, 3, 4, 14] was taken 320 times.
Path_3: [0, 1, 5, 2, 8, 12, 7, 3, 10, 14] was taken 0 times.
Path_4: [0, 1, 5, 2, 8, 12, 10, 7, 3, 4, 14] was taken 282 times.
Path_5: [0, 1, 5, 2, 8, 12, 10, 14] was taken 744 times.
Path_6: [0, 1, 5, 2, 14] was taken 9734 times.
Path_7: [0, 1, 5, 4, 3, 10, 2, 14] was taken 1 times.
Path_8: [0, 1, 5, 4, 3, 10, 14] was taken 3 times.
Path_9: [0, 1, 5, 4, 14] was taken 112623 times.
Path_10: [0, 1, 5, 10, 2, 6, 4, 14] was taken 2397 times.
Path_11: [0, 1, 5, 10, 2, 8, 12, 7, 3, 4, 14] was taken 108 times.
Path_12: [0, 1, 5, 10, 2, 14] was taken 1725 times.
Path_13: [0, 1, 5, 10, 7, 3, 4, 14] was taken 1535 times.
Path_14: [0, 1, 5, 10, 14] was taken 1482 times.
Path_15: [0, 1, 12, 7, 3, 4, 14] was taken 6239 times.
Path_16: [0, 1, 12, 7, 3, 10, 2, 6, 4, 14] was taken 8 times.
Path_17: [0, 1, 12, 7, 3, 10, 2, 14] was taken 8 times.
Path_18: [0, 1, 12, 7, 3, 10, 14] was taken 2 times.
Path_19: [0, 1, 12, 7, 5, 2, 6, 4, 3, 10, 14] was taken 1 times.
Path_20: [0, 1, 12, 7, 5, 2, 6, 4, 14] was taken 2323 times.
Path_21: [0, 1, 12, 7, 5, 2, 14] was taken 1591 times.
Path_22: [0, 1, 12, 7, 5, 4, 3, 10, 2, 14] was taken 0 times.
Path_23: [0, 1, 12, 7, 5, 4, 3, 10, 14] was taken 2 times.
Path_24: [0, 1, 12, 7, 5, 4, 14] was taken 18905 times.
Path_25: [0, 1, 12, 7, 5, 10, 2, 6, 4, 14] was taken 577 times.
Path_26: [0, 1, 12, 7, 5, 10, 2, 14] was taken 387 times.
Path_27: [0, 1, 12, 7, 5, 10, 14] was taken 254 times.

Path.28: [0, 1, 12, 10, 2, 6, 4, 14] was taken 3995 times.
Path.29: [0, 1, 12, 10, 2, 14] was taken 5994 times.
Path.30: [0, 1, 12, 10, 7, 3, 4, 14] was taken 4748 times.
Path.31: [0, 1, 12, 10, 7, 5, 2, 6, 4, 14] was taken 2424 times.
Path.32: [0, 1, 12, 10, 7, 5, 2, 14] was taken 1491 times.
Path.33: [0, 1, 12, 10, 7, 5, 4, 14] was taken 14687 times.
Path.34: [0, 1, 12, 10, 14] was taken 4796 times.

3. Blue:_Vs_Red (With Resource Optimization)

Experiment Summary: Blue_Vs_Red

Description:

1. Agents were trained for a number of Episodes = 100000
2. System has 50% desirable states and 50% undesirable states
Total number of game simulations = 100000
Maximum Episodic length: 25
Average Episodic length: 22.75563
agent_blue was given 55.70398084279179 as initial resources
agent_red was given 358.7140770169328 as initial resources
Good Team wins 90532 times with 90.532 %
Bad Team wins 9367 times with 9.367 %
End with Draw 101 times with 0.101 %

Safety result: System is safe

Paths Records:

Path.0: [0, 1, 5, 2, 6, 4, 3, 10, 14] was taken 0 times.
Path.1: [0, 1, 5, 2, 6, 4, 14] was taken 2273 times.
Path.2: [0, 1, 5, 2, 8, 12, 7, 3, 4, 14] was taken 0 times.
Path.3: [0, 1, 5, 2, 8, 12, 7, 3, 10, 14] was taken 1 times.
Path.4: [0, 1, 5, 2, 8, 12, 10, 7, 3, 4, 14] was taken 0 times.
Path.5: [0, 1, 5, 2, 8, 12, 10, 14] was taken 201 times.
Path.6: [0, 1, 5, 2, 14] was taken 2922 times.
Path.7: [0, 1, 5, 4, 3, 10, 2, 14] was taken 0 times.
Path.8: [0, 1, 5, 4, 3, 10, 14] was taken 1 times.
Path.9: [0, 1, 5, 4, 14] was taken 34337 times.
Path.10: [0, 1, 5, 10, 2, 6, 4, 14] was taken 0 times.
Path.11: [0, 1, 5, 10, 2, 8, 12, 7, 3, 4, 14] was taken 0 times.
Path.12: [0, 1, 5, 10, 2, 14] was taken 510 times.
Path.13: [0, 1, 5, 10, 7, 3, 4, 14] was taken 0 times.
Path.14: [0, 1, 5, 10, 14] was taken 442 times.
Path.15: [0, 1, 12, 7, 3, 4, 14] was taken 0 times.
Path.16: [0, 1, 12, 7, 3, 10, 2, 6, 4, 14] was taken 0 times.
Path.17: [0, 1, 12, 7, 3, 10, 2, 14] was taken 0 times.
Path.18: [0, 1, 12, 7, 3, 10, 14] was taken 2 times.
Path.19: [0, 1, 12, 7, 5, 2, 6, 4, 3, 10, 14] was taken 0 times.
Path.20: [0, 1, 12, 7, 5, 2, 6, 4, 14] was taken 0 times.
Path.21: [0, 1, 12, 7, 5, 2, 14] was taken 503 times.
Path.22: [0, 1, 12, 7, 5, 4, 3, 10, 2, 14] was taken 0 times.
Path.23: [0, 1, 12, 7, 5, 4, 3, 10, 14] was taken 0 times.
Path.24: [0, 1, 12, 7, 5, 4, 14] was taken 0 times.
Path.25: [0, 1, 12, 7, 5, 10, 2, 6, 4, 14] was taken 0 times.
Path.26: [0, 1, 12, 7, 5, 10, 2, 14] was taken 104 times.
Path.27: [0, 1, 12, 7, 5, 10, 14] was taken 75 times.
Path.28: [0, 1, 12, 10, 2, 6, 4, 14] was taken 0 times.
Path.29: [0, 1, 12, 10, 2, 14] was taken 1721 times.
Path.30: [0, 1, 12, 10, 7, 3, 4, 14] was taken 0 times.
Path.31: [0, 1, 12, 10, 7, 5, 2, 6, 4, 14] was taken 0 times.

Path_32: [0, 1, 12, 10, 7, 5, 2, 14] was taken 452 times.
Path_33: [0, 1, 12, 10, 7, 5, 4, 14] was taken 0 times.
Path_34: [0, 1, 12, 10, 14] was taken 1420 times.

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e.g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Date and Signature: